OVERVIEW

Development Approach

How a team chooses the way it builds software, in four parts. Methodology concepts: process vs. process model vs. methodology, what a methodology addresses, its ten elements, and the two life cycles of an IT project (PLC for project requirements, SDLC for product requirements). Software development processes: plan-driven waterfall (when it fits and how it fails) vs. evolutionary/iterative development with its four phases (inception, elaboration, construction, transition). Agile: the 4 values and 12 principles, incremental development, XP practices and Scrum (roles, artifacts, sprints, backlog). DevOps: closing the gap between development and operations, and how project management adapts to a CI/CD pipeline.

The exam angle is choosing an approach for a situation and justifying it: match stable, known requirements to waterfall and evolving, uncertain ones to iterative or agile, and say why.

Process → model → methodology

Process = the activities; process model (life cycle) = an abstract shape like waterfall or iterative; methodology = the concrete "how we work around here".

PLC vs. SDLC

The project life cycle targets project requirements (time, cost); the SDLC targets product requirements. You need both, integrated.

Waterfall fits the known

Clear stable requirements, proven technology, low complexity, adequate time. Fails late: late integration, late risk resolution.

Iterative fits the uncertain

Unclear or evolving requirements, new technology, high complexity, short schedule. Phases: inception → elaboration (most critical) → construction → transition.

Agile = 4 values, 12 principles

Individuals over process, working software over documents, collaboration over contracts, responding to change over the plan. Waterfall fixes scope; agile fixes time and cost.

XP vs. Scrum

XP prescribes engineering practices (TDD, CI, refactoring, pairing); Scrum is a planning framework (backlog, sprints, Product Owner, ScrumMaster) with no technical practices.

HOW THE LECTURE IS ORGANIZED

Slides 1–14 Methodology concepts, elements of a methodology, PLC vs. SDLC

Slides 15–32 Plan-driven waterfall vs. evolutionary/iterative development

Slides 33–47 Agile: manifesto, principles, incremental development

Slides 48–55 Extreme Programming practices

Slides 56–74 Scrum: roles, backlog, sprints, agile vs. waterfall

Slides 75–98 DevOps and DevOps project management

EXAM FOCUS

Midterm 251 (both versions) asked when an iterative approach is most beneficial (MCQ 6) and, in the Edugate case, whether to recommend Agile, Waterfall or Hybrid and why (Case Q2, from the instructor's key: Agile/Scrum because requirements evolve). Final 251 Q1.4 asked for a PM tool or technique for a complex software project; Final 242 Q1.2 needs the milestone definition from slide 10.

HOW TO READ THIS PAGE

Every original slide is shown unchanged. Yellow marks on a slide point to a numbered note: hover or tap a mark to jump to its note, and hover a note to see its marks. Questions tagged **Past exam** appeared on a real paper; **Prediction** questions are likely angles, not confirmed. Pick an answer to each question: a wrong pick tells you why it is wrong, so you can try again.

SLIDE 1 / 98 **Slide 1**



Development Approach

SE423: Software Project Management

1

NOTES

1 The deck has four parts: **methodology concepts**, **software development processes** (waterfall vs. iterative), **agile** project management (XP and Scrum), and **DevOps**. The middle two carry most of the exam questions.

SLIDE 2 / 98 **Outline**

Outline

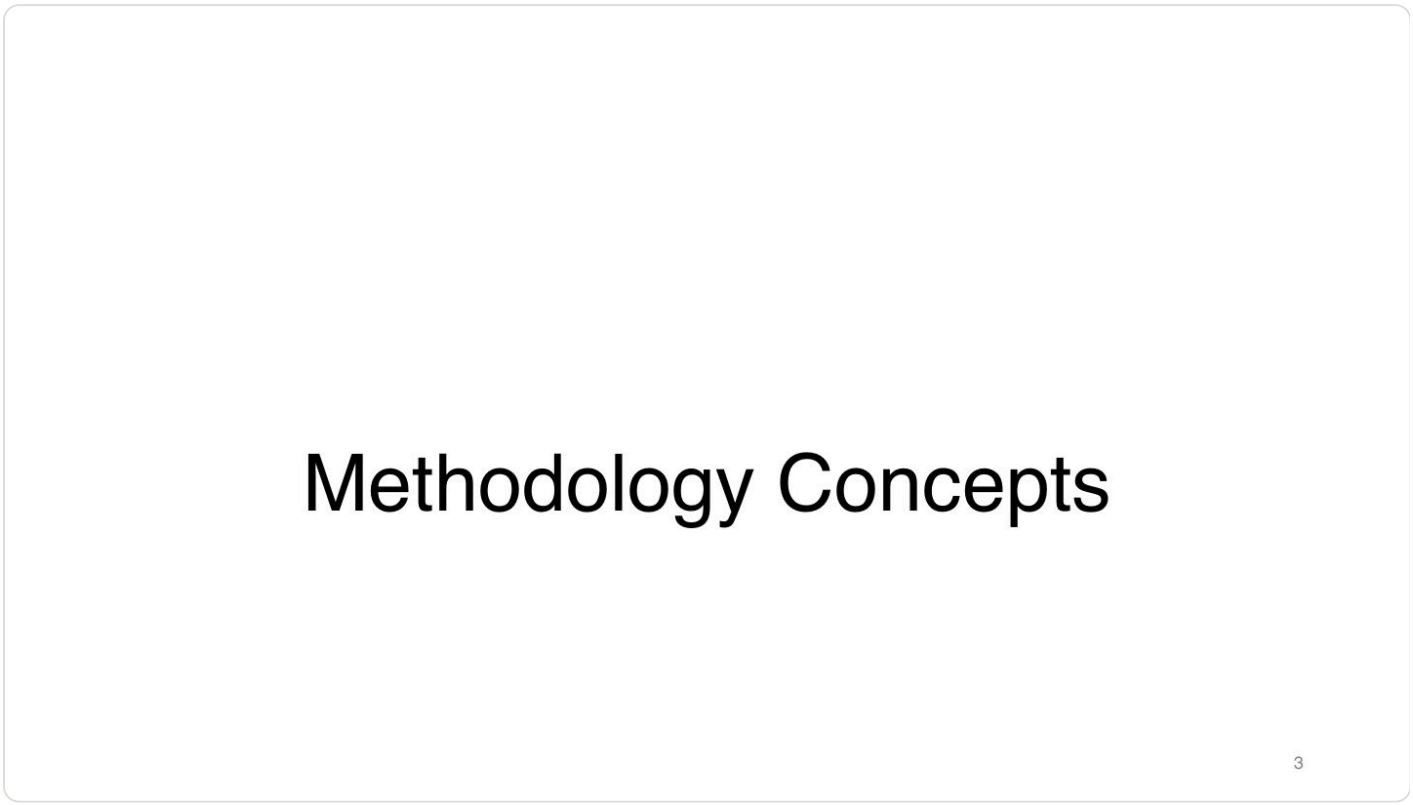
- Methodology Concepts
- Software Development Process
- Agile Project Management
- DevOps

2

NOTES

1. **Methodology concepts**: the vocabulary (process, model, methodology) and what a methodology contains.
2. **Software development process**: plan-driven waterfall vs. evolutionary/iterative.
3. **Agile project management**: the manifesto, XP and Scrum.
4. **DevOps**: joining development and operations, and what that means for the PM.

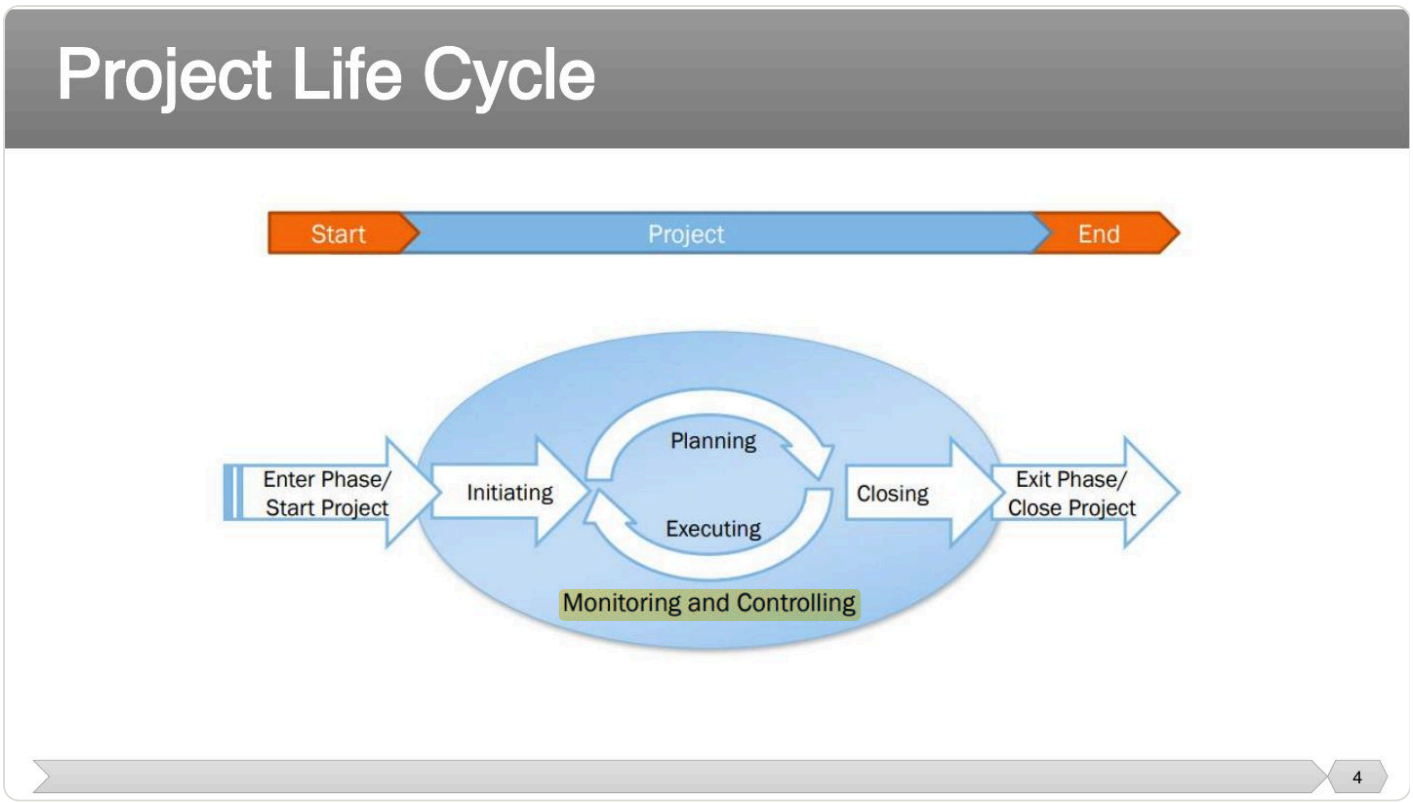
SLIDE 3 / 98 **Slide 3**



NOTES

1. Section divider: **Methodology Concepts**.

 Project Life Cycle



NOTES

1. The **project life cycle** as process groups. **Initiating** starts the phase or project (charter, stakeholders).
2. **Planning** and **executing** loop around each other, wrapped by **monitoring and controlling**, which compares progress with the plan and triggers re-planning.
3. **Closing** ends the phase or project (acceptance, lessons learned). Every development approach in this deck still sits inside this cycle.

 Methodology Concepts

Methodology Concepts

- Process
  - A collection of work activities, actions and tasks that are performed when some work product is to be created
- Process Model or Life Cycle
  - An abstract description of a software process that presents one view of that process
  - Waterfall, Iterative, Spiral, Evolutionary, etc.
- Process Methodology or Methodology
  - Instantiations of process models – tend to be prescriptive.
  - The conventions that a group agrees to – "How we work around here"

5

NOTES

1. A **process** is a collection of work activities, actions and tasks performed when some work product is to be created.
2. A **process model** (life cycle) is an **abstract description** of a process that shows one view of it: waterfall, iterative, spiral, evolutionary.
3. A **methodology** is an **instantiation** of a process model: concrete and prescriptive, the conventions a group agrees to, "how we work around here". So Scrum at your company is a methodology; "iterative" is the model behind it.

EXAM ANGLE

Prediction Q1

Waterfall, iterative and spiral are examples of:

A Tools

B Milestones

C Methodologies

D **Process models (life cycles)**

SLIDE 6 / 98 **What a Methodology Addresses?**

What a Methodology Addresses?

- Introducing new people to the process
- Substituting people
- Delineating responsibilities
- Demonstrating visible progress
  - McConnell's definition for visibility: "The ease & accuracy with which it is possible to assess the status of a project's cost, schedule, functionality, or other characteristic."

NOTES

1. A methodology addresses people issues: **introducing new people** to the process, **substituting** people, and **delineating responsibilities** (who does what).

2. It also **demonstrates visible progress**. McConnell defines **visibility** as the ease and accuracy with which you can assess the status of a project's cost, schedule, functionality or other characteristics.

SLIDE 7 / 98 **Evaluating a Methodology**

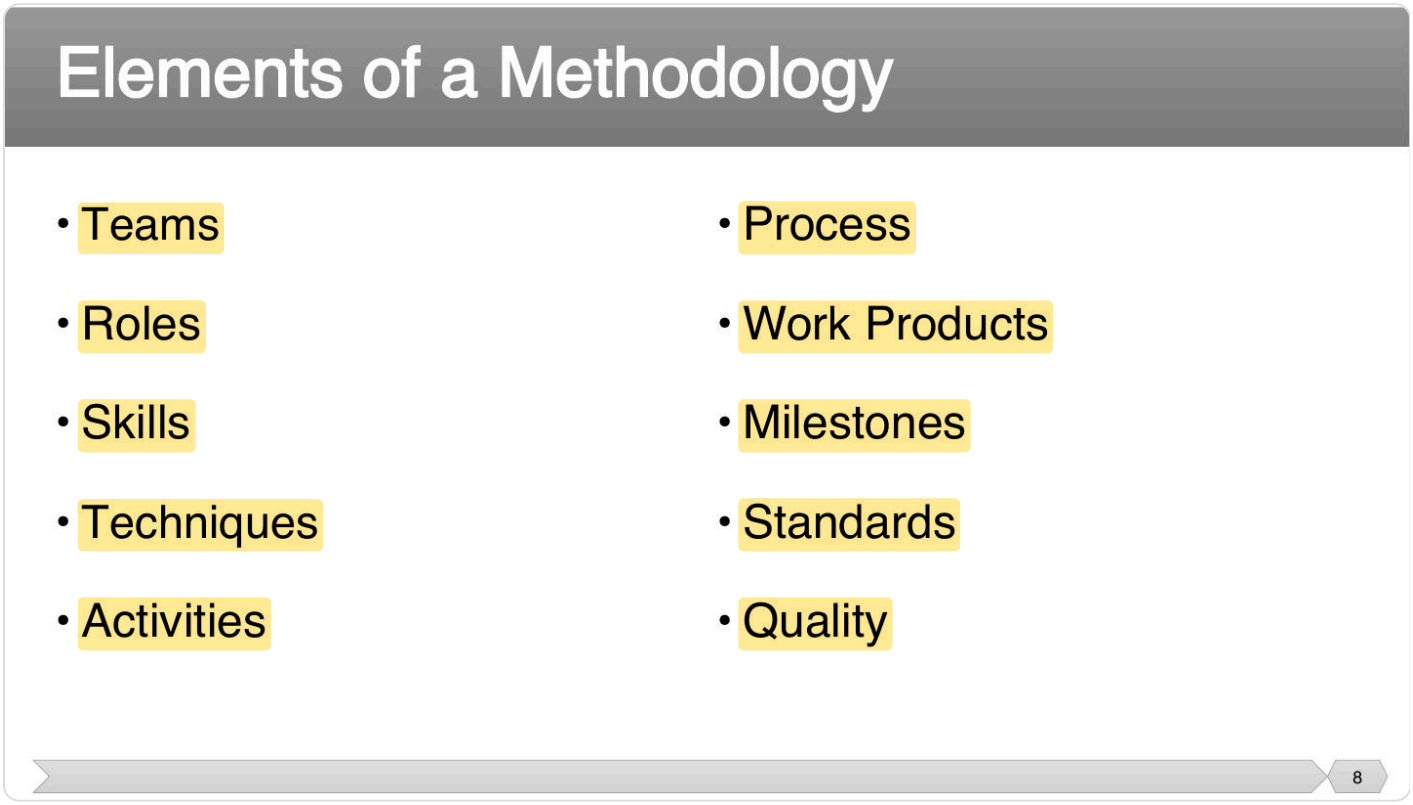
Evaluating a Methodology

- How rapidly you can substitute or train people.
- How great an effect it has on the sales process.
- How much freedom (or how constraining) it is to people on the project.
- How fast it allows people to respond to changing situations.
- How well it "protects" the organization – legally or from other damages.

NOTES

1. Judge a methodology by how quickly it lets you **substitute or train** people...
2. ...its **effect on the sales process** (can you show clients a credible way of working?)...
3. ...how much **freedom** it leaves people (or how constraining it is)...
4. ...how fast it lets people **respond to change**...
5. ...and how well it **protects** the organization, legally or from other damage (e.g. documented approvals).

SLIDE 8 / 98 Elements of a Methodology

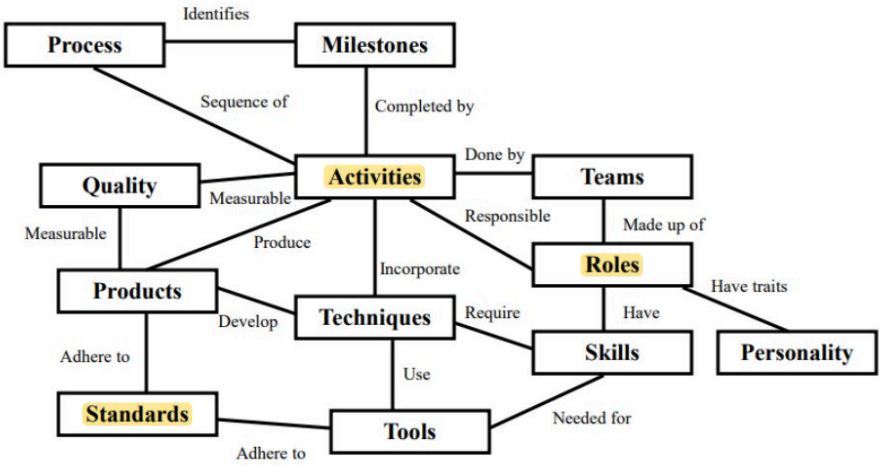


NOTES

1. Cockburn's **ten elements** of a methodology. The **people side**: teams, roles, skills, techniques, activities.
2. The **work side**: process, work products, milestones, standards, quality. Slides 10–13 define each.

 Cockburn Methodology Structure

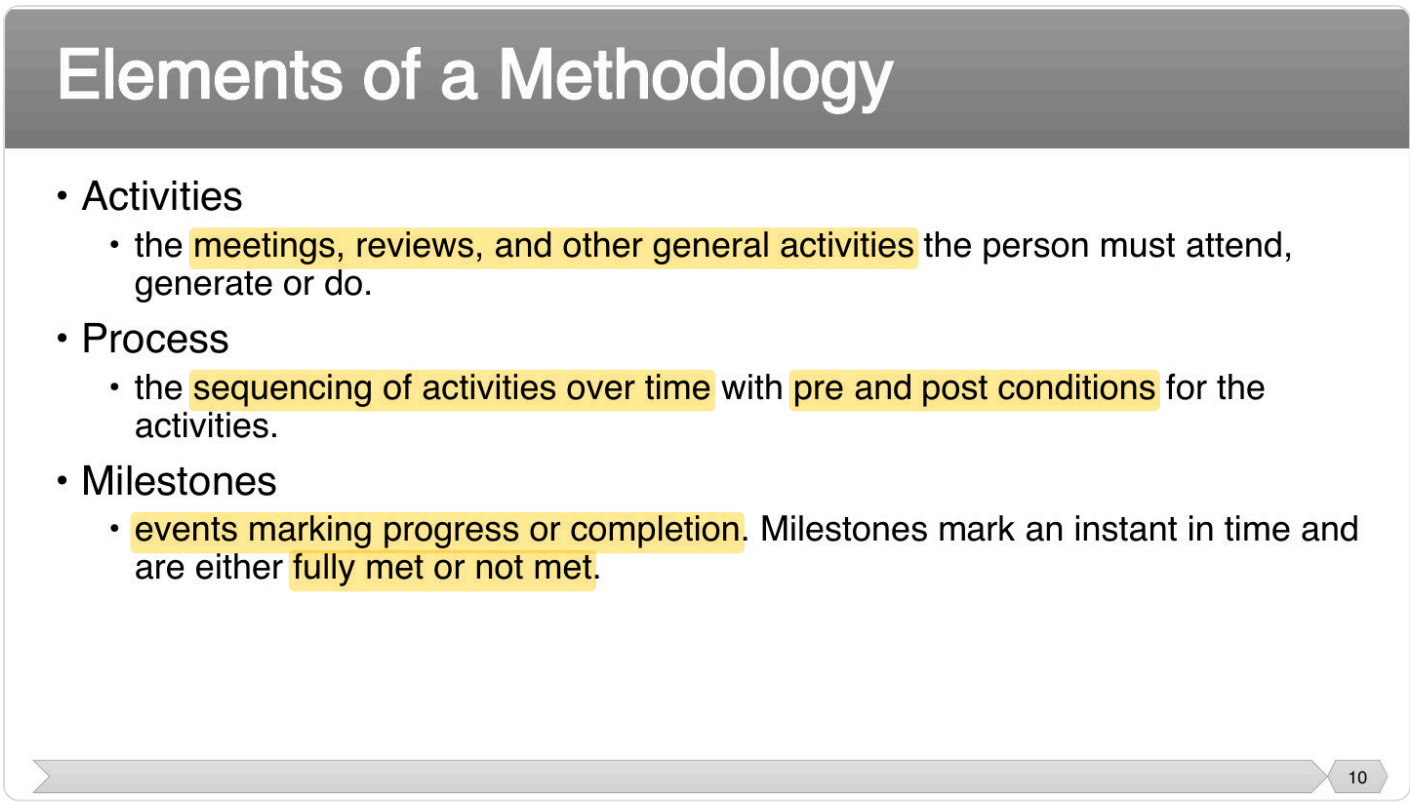
Cockburn Methodology Structure



NOTES

1. Read the diagram as sentences: the **process** is a sequence of **activities**, completed by **milestones**; activities produce **products** and incorporate **techniques**.
2. **Teams** are made of **roles**; roles have **skills** and are matched to **personalities**; techniques require skills and use **tools**.
3. Products adhere to **standards**, and **quality** is measurable on products and activities.

 Elements of a Methodology



NOTES

1. **Activities**: the meetings, reviews and other general activities a person must attend, generate or do.
2. **Process**: the sequencing of activities over time, with pre- and post-conditions for each.
3. **Milestones**: events marking progress or completion. A milestone marks an **instant** (zero duration) and is either **fully met or not met**; there is no “60% met”.

TIPS & TRICKS

- **MAP**: Milestones, Activities, Process.

EXAM ANGLE

Past exam · Final 242 · Q1.2 Q1

Explain what is a Gantt chart and describe its significance in planning and tracking project activities. In your answer, define the terms milestone and dependency, and provide one example of each as they would appear in a Gantt chart.

Answer: A Gantt chart is a bar chart with activities down the side and a time scale across, where each bar shows an activity's start, finish and duration; it makes the schedule easy to plan and to track, because actual progress is shaded against the planned bars. A milestone is a significant event with zero duration, either fully met or not, shown as a diamond, e.g. "Requirements signed off". A dependency is a logical relationship where one activity can't start (or finish) until another does, shown as an arrow, e.g. "Coding" can't start until "Design approved" finishes.

Prediction Q2

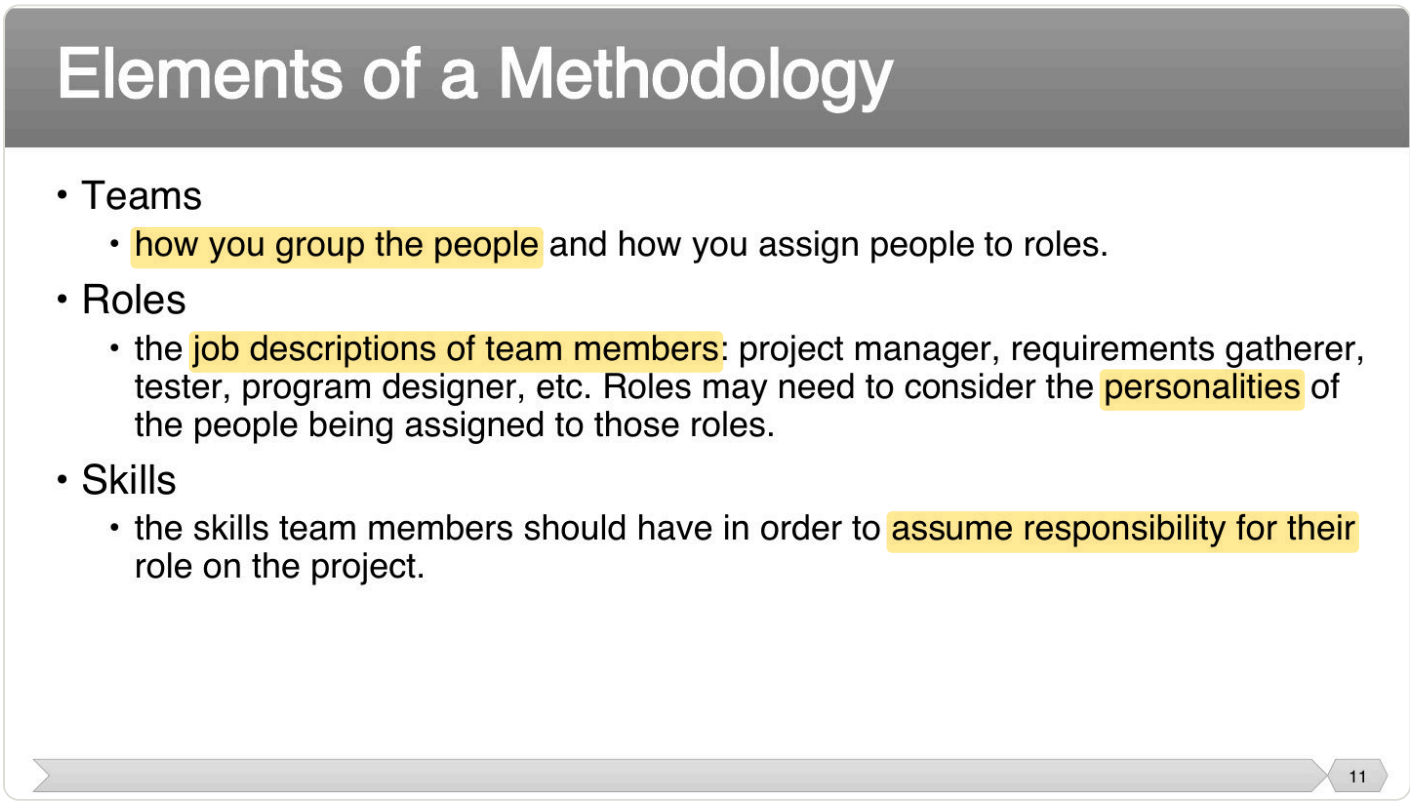
A milestone can be 60% met.

T True

F **False**

> Milestones mark an instant in time and are either fully met or not met.

SLIDE 11 / 98 **Elements of a Methodology**



NOTES

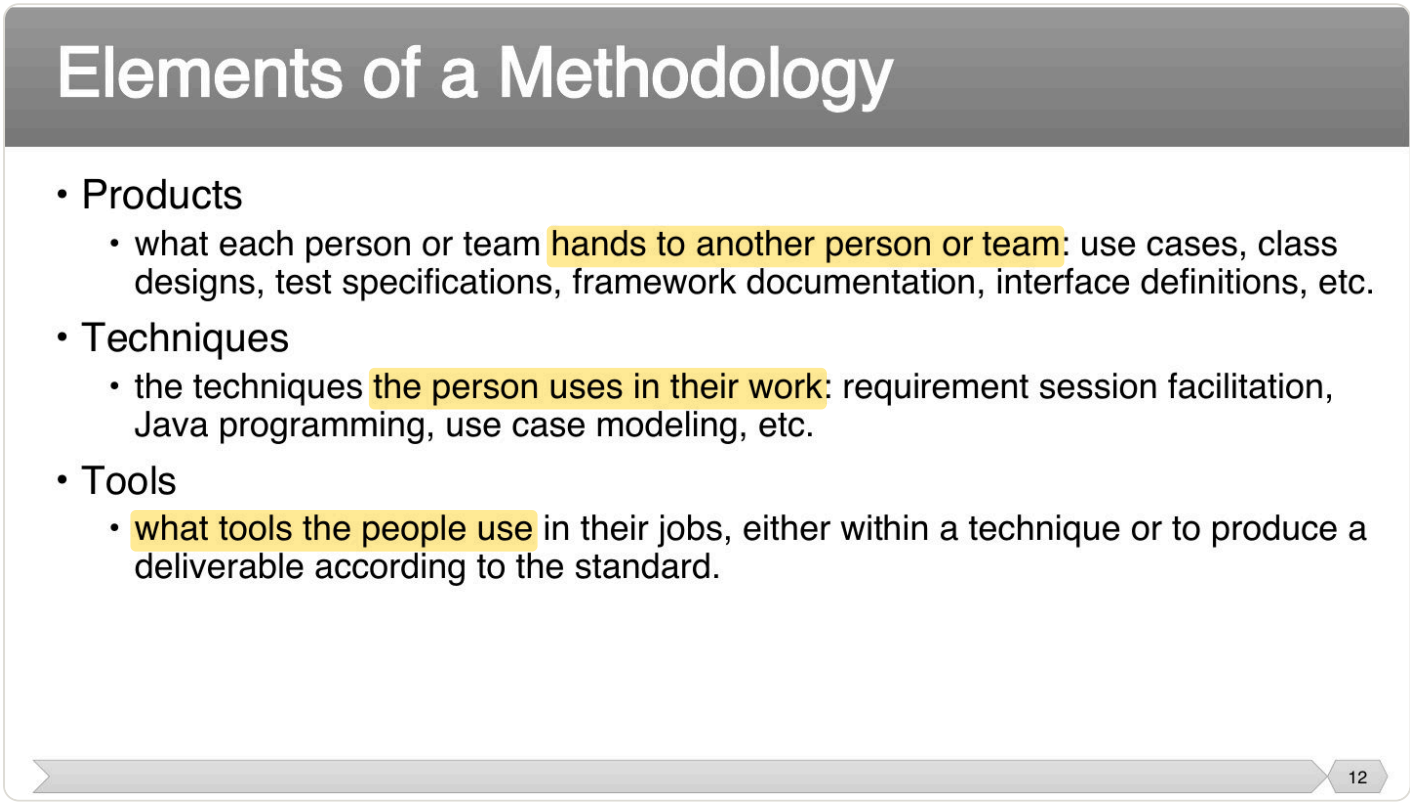
1 **Teams**: how you group people and assign them to roles.

2. **Roles**: the job descriptions (project manager, requirements gatherer, tester, designer). Assignments should consider the **personalities** of the people.
3. **Skills**: what team members need in order to **take responsibility** for their role.

TIPS & TRICKS

- **STaR**: Skills, Teams, Roles.

SLIDE 12 / 98 **Elements of a Methodology**



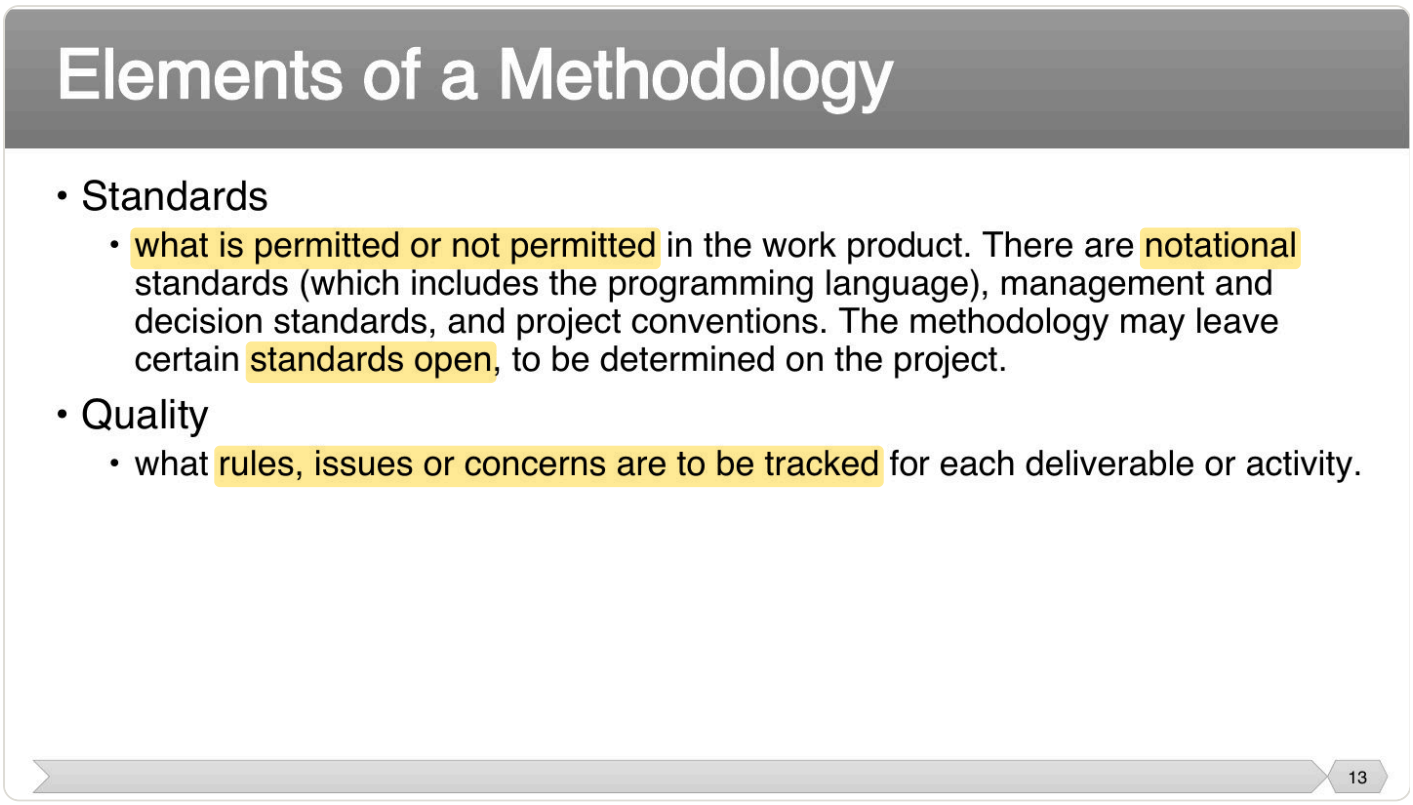
NOTES

1. **Products**: what each person or team **hands to another**: use cases, class designs, test specifications, interface definitions.
2. **Techniques**: how the person does the work: requirements facilitation, Java programming, use case modeling.
3. **Tools**: what they use, within a technique or to produce a deliverable to the standard (IDE, Jira, test frameworks).

TIPS & TRICKS

- **PuTT**: Products, Techniques, Tools.

 Elements of a Methodology



NOTES

1. **Standards**: what is permitted or not in the work product: **notational** standards (including the programming language), management and decision standards, and project conventions. Some may be **left open** to decide on the project.

2. **Quality**: which rules, issues or concerns are tracked for each deliverable or activity (e.g. code review required, test coverage).

 IT project life cycles

IT project life cycles

- IT projects have two concurrent life cycles:
  - *Project life* cycle (PLC) encompasses all activities of project, including the System/Software Development Life Cycle (SDLC)
  - PLC is directed toward achieving *project* requirements
  - SDLC is directed toward achieving *product* requirements
- Both life cycle models are needed to manage an IT project
  - PLC alone will not adequately address system development concerns
  - SDLC alone will not adequately address business and product integration concerns
  - Effective integration of the two life cycle models is essential to improving the likelihood of project success

14

NOTES

1. IT projects run **two life cycles at once**: the **project life cycle (PLC)** covers all project activities, including the **SDLC**.
2. The **PLC** targets **project** requirements (scope, time, cost, approvals); the **SDLC** targets **product** requirements (features, quality). A common trap: don't swap them.
3. Neither is enough alone: the PLC can't handle system development concerns, and the SDLC can't handle business and product integration.
4. **Integrating** the two is essential to project success.

EXAM ANGLE

Prediction Q1

The SDLC is directed toward achieving:

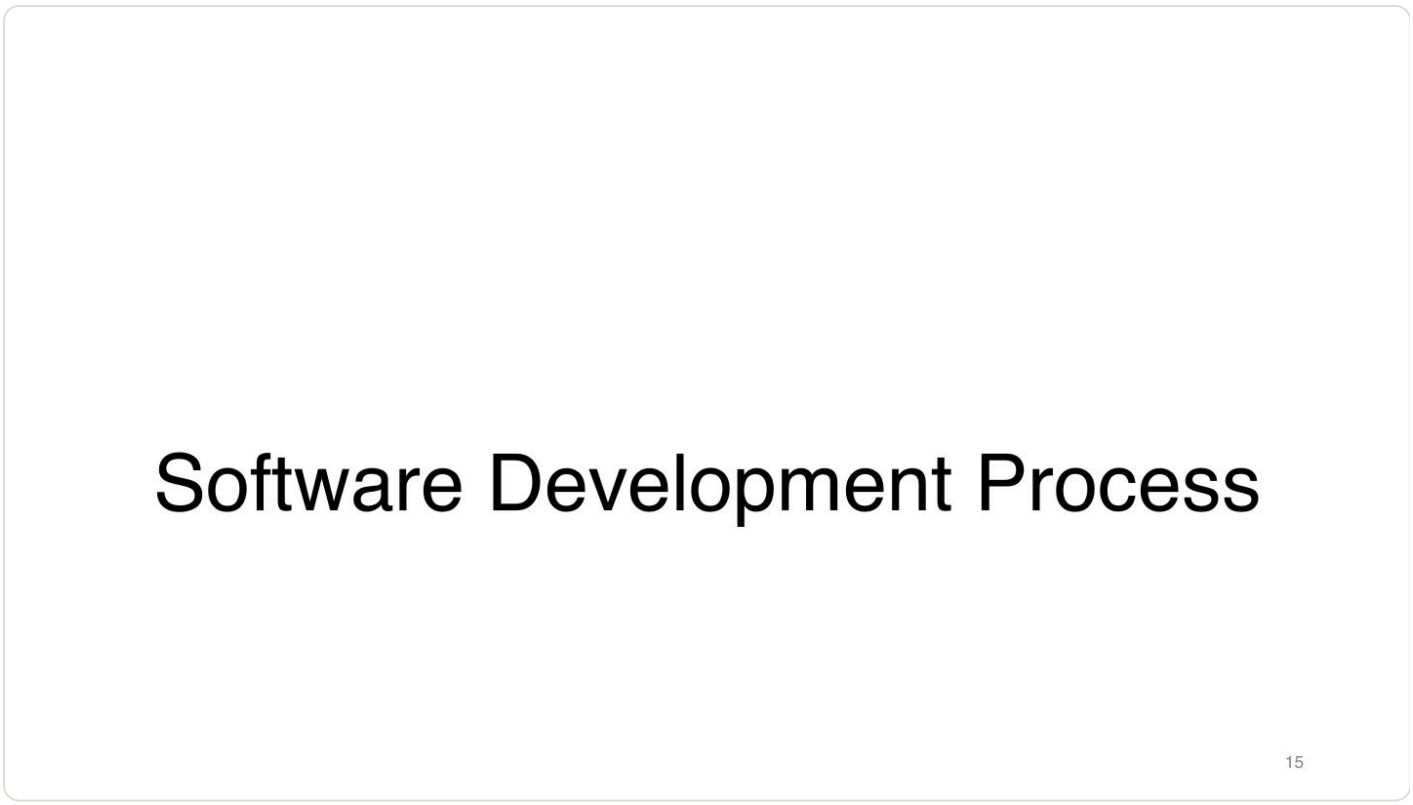
A Procurement

B Project requirements

C **Product requirements**

D Business case approval

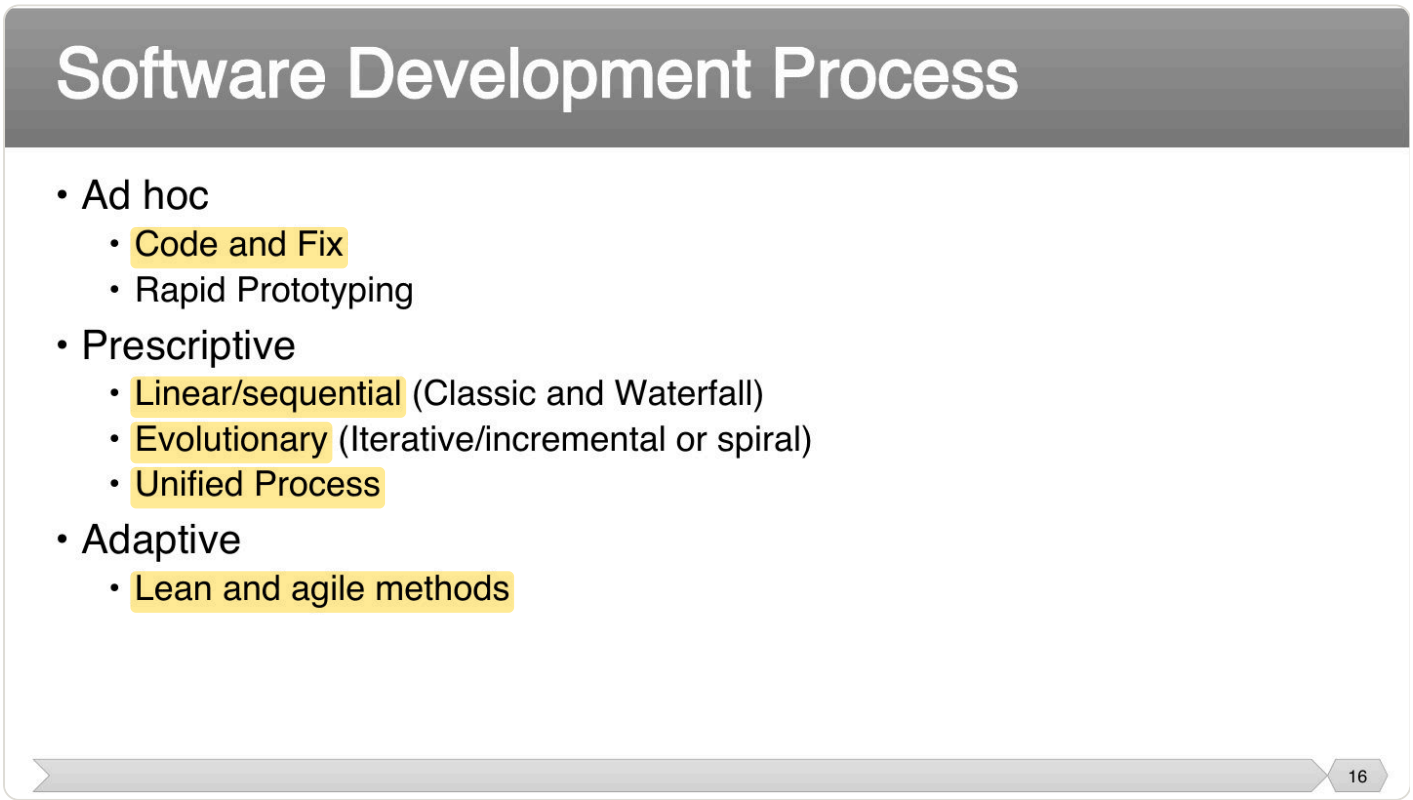
 Slide 15



NOTES

1. Section divider: **Software Development Process**.

 Software Development Process

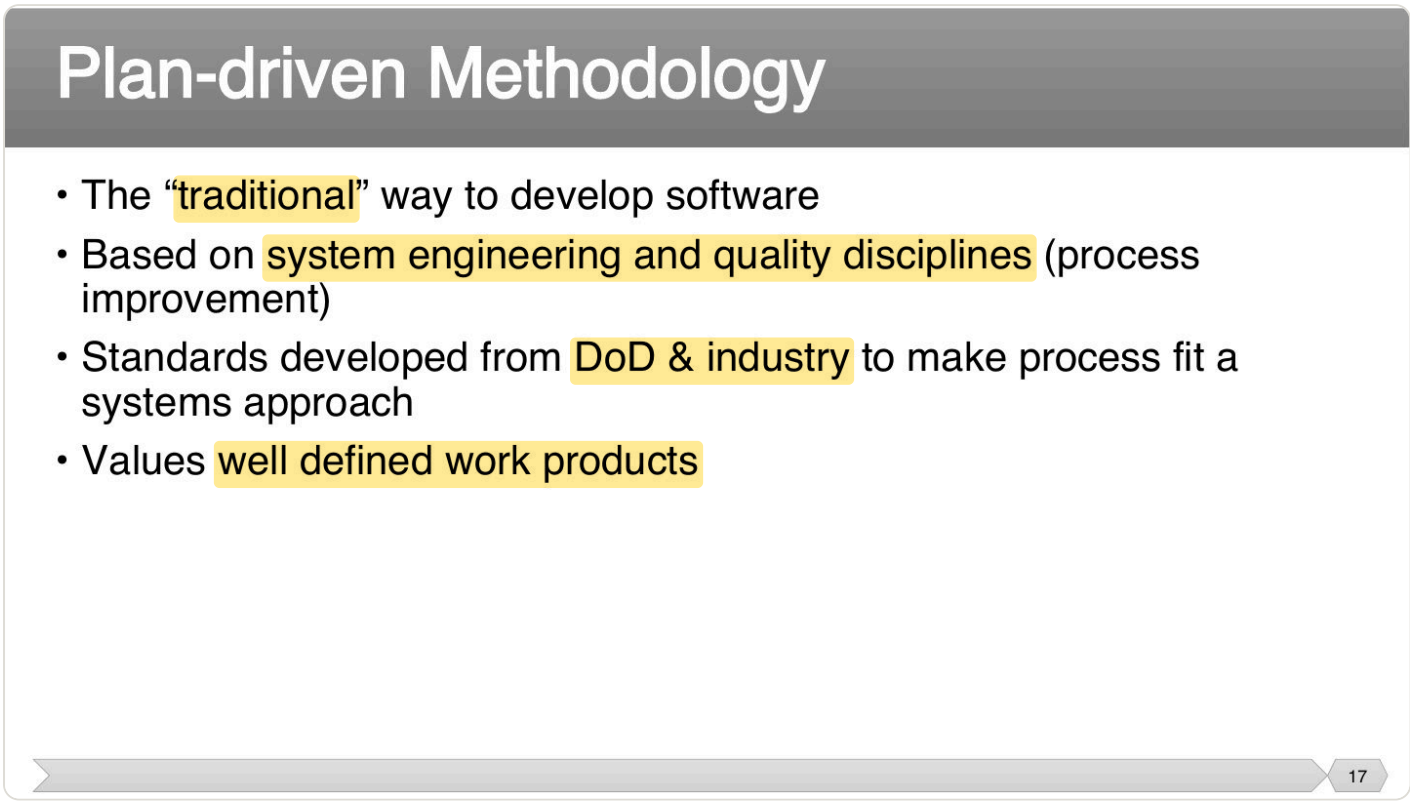


NOTES

1. Three families of process. **Ad hoc**: code and fix, rapid prototyping, with no defined process.

2. **Prescriptive**: linear/sequential (classic waterfall), evolutionary (iterative/incremental, spiral) and the Unified Process.
3. **Adaptive**: lean and agile methods.

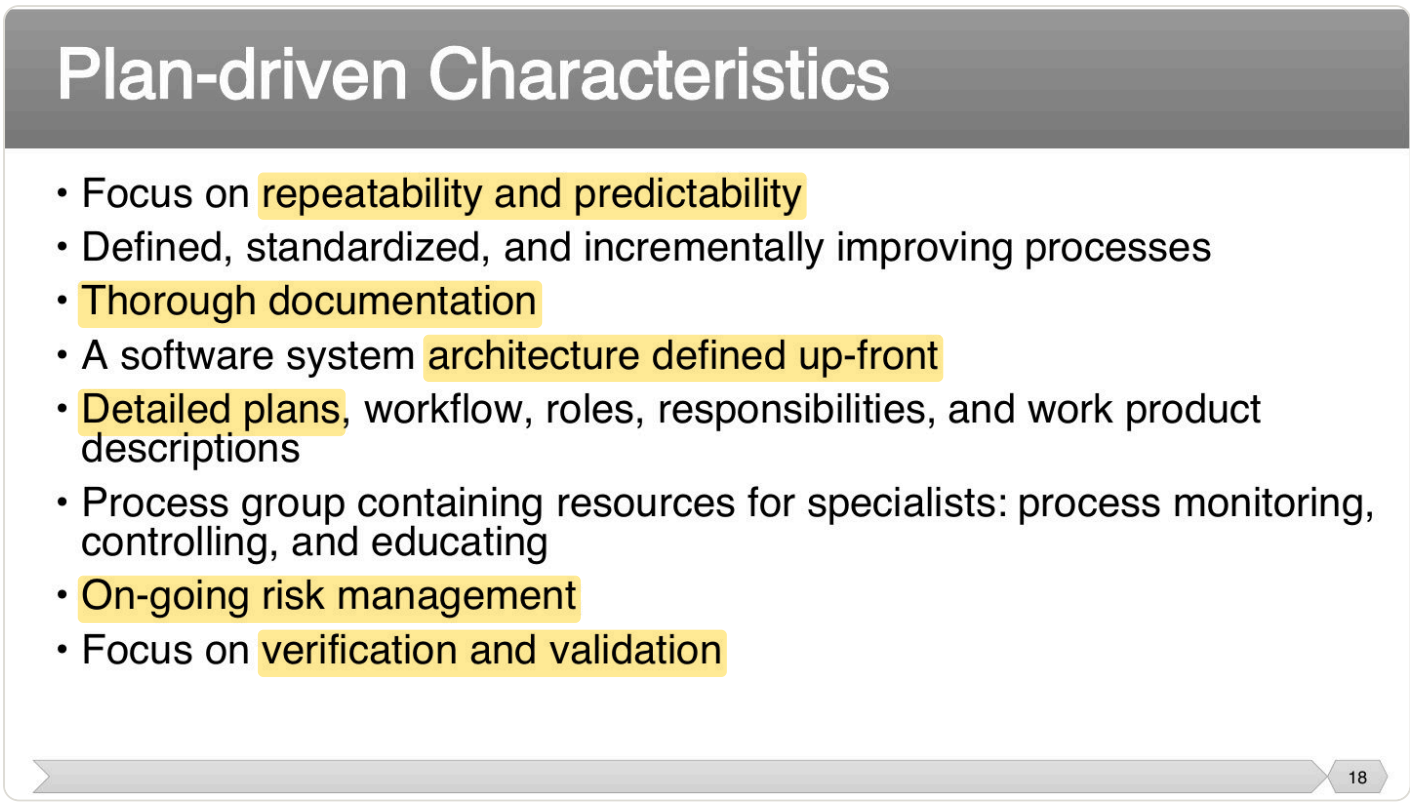
SLIDE 17 / 98 **Plan-driven Methodology**



NOTES

1. **Plan-driven** is the **traditional** way to develop software...
2. ...based on **systems engineering and quality** disciplines (process improvement)...
3. ...with standards from the **US Department of Defense and industry** that fit the process to a systems approach...
4. ...and it values **well-defined work products** (documents, specifications).

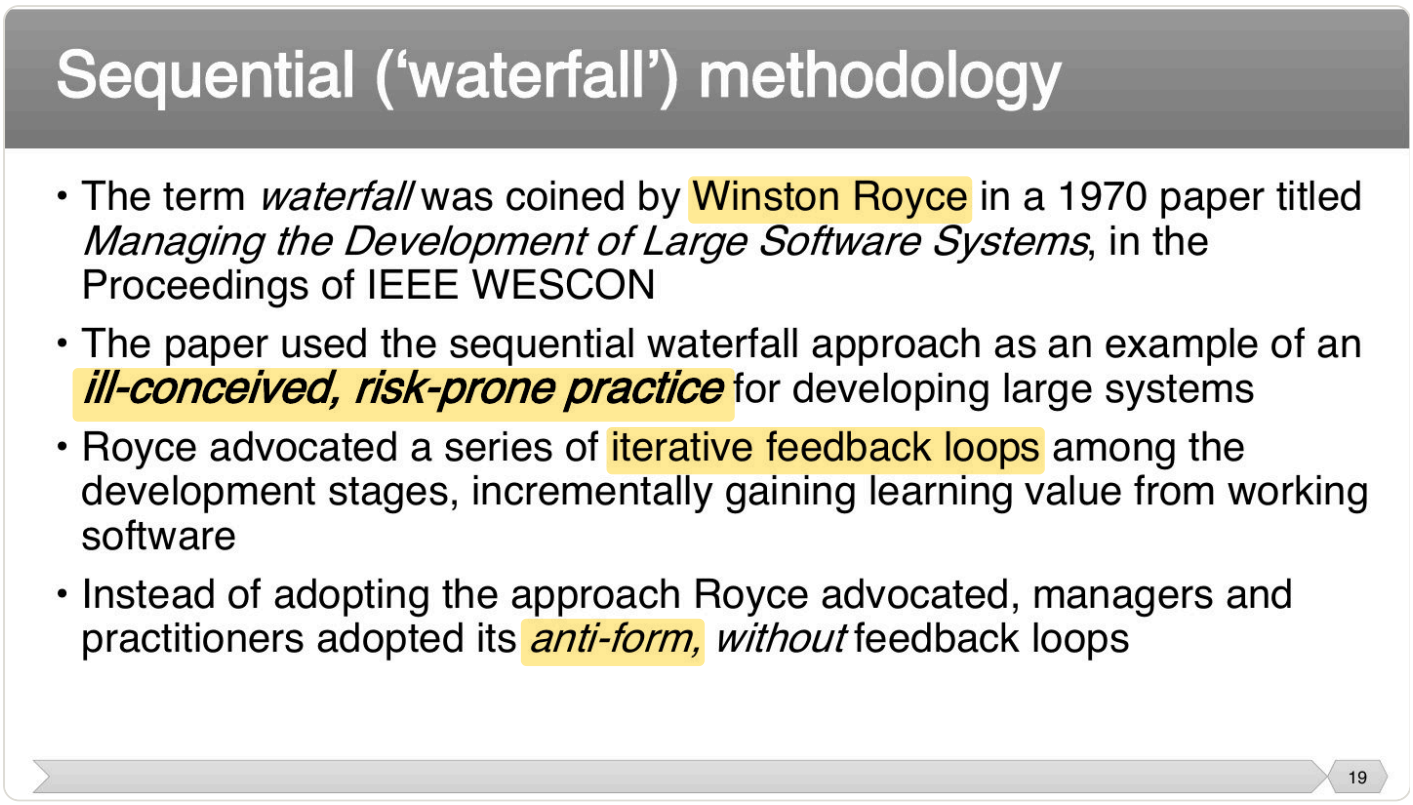
 Plan-driven Characteristics



NOTES

1. The aim is **repeatability and predictability**, through defined, standardized, continuously improved processes.
2. **Thorough documentation** and an **architecture defined up front**.
3. **Detailed plans**, workflows, roles and work-product descriptions, with a process group of specialists to monitor, control and educate.
4. **Ongoing risk management** and a focus on **verification and validation**.

 Sequential (‘waterfall’) methodology



NOTES

1. The term **waterfall** comes from Winston Royce’s 1970 paper on managing large software systems.
2. Royce used the pure sequential waterfall as an example of an **ill-conceived, risk-prone** practice.
3. He advocated **iterative feedback loops** between stages, learning incrementally from working software.
4. Industry adopted its **anti-form**, the version without feedback loops. A common trap: “Royce recommended the pure waterfall” is false.

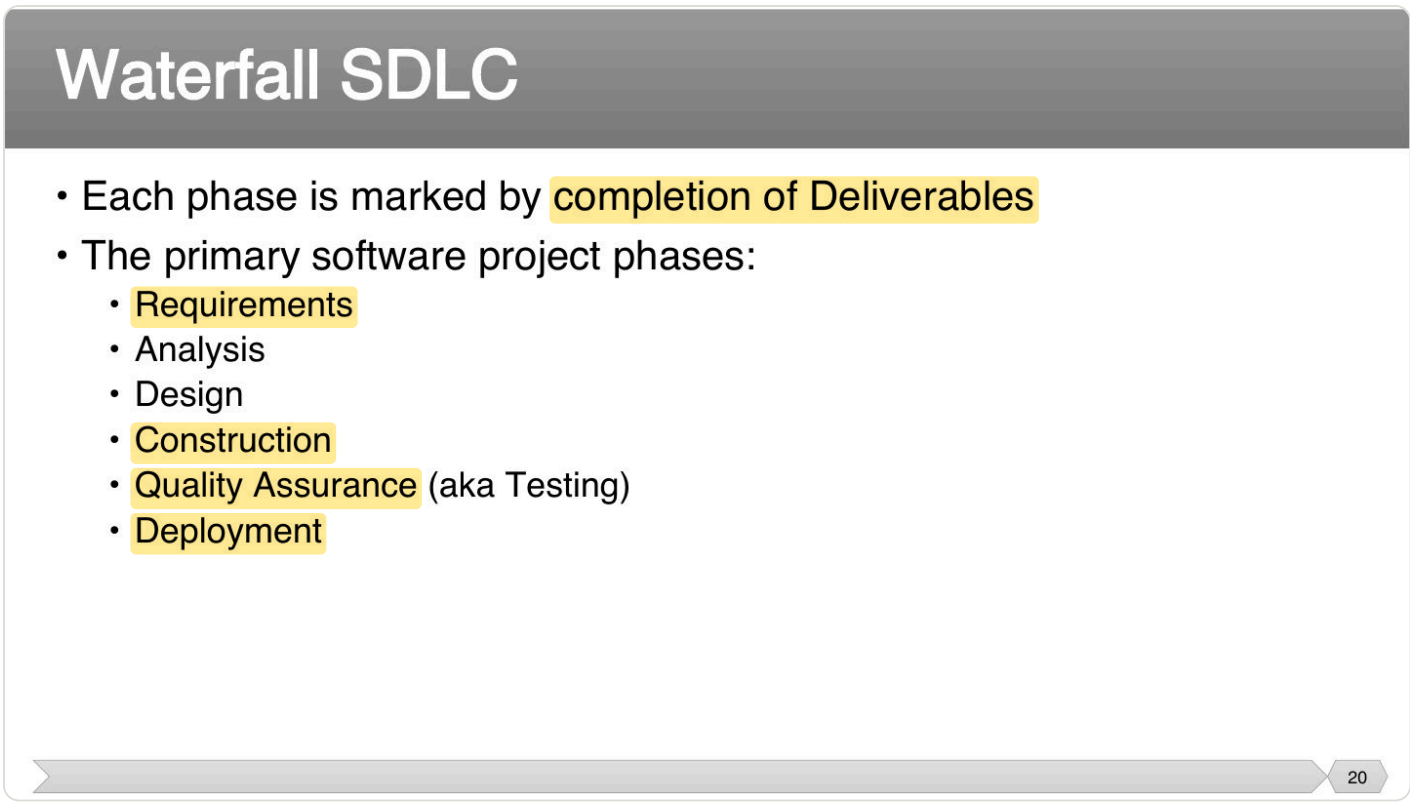
EXAM ANGLE

Prediction Q1

Winston Royce’s 1970 paper presented the pure sequential waterfall as an ill-conceived, risk-prone practice.

- T **True**
- F False

 Waterfall SDLC



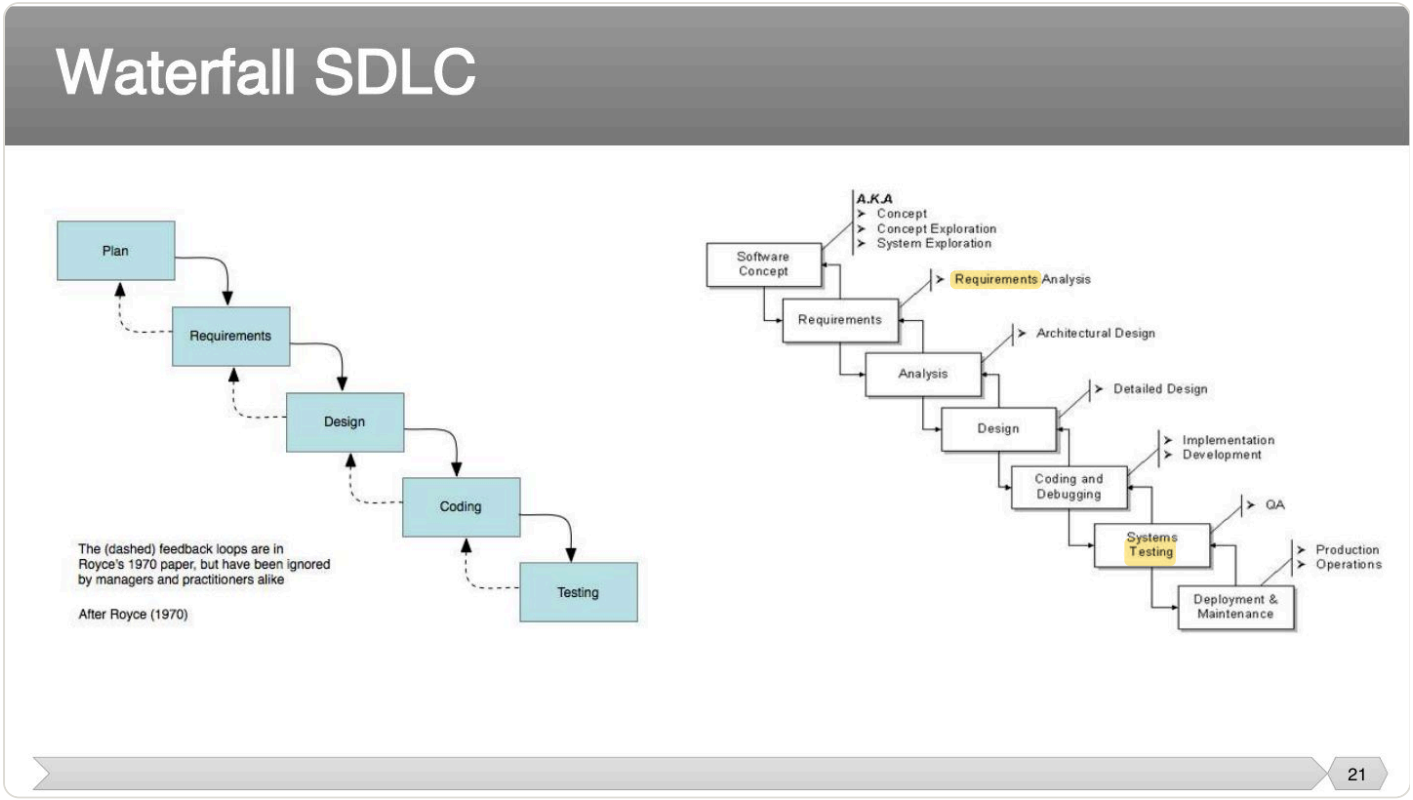
NOTES

1. In waterfall each phase ends with the **completion of deliverables** (a signed-off document or product) before the next starts.
2. The phases: **requirements → analysis → design → construction → QA (testing) → deployment**.

TIPS & TRICKS

- The order matters here: **R**eal **A**rchitects **D**raw, **C**onstruct, **Q**uality-check, **D**eploy. Requirements, Analysis, Design, Construction, Quality assurance, Deployment.

 Waterfall SDLC

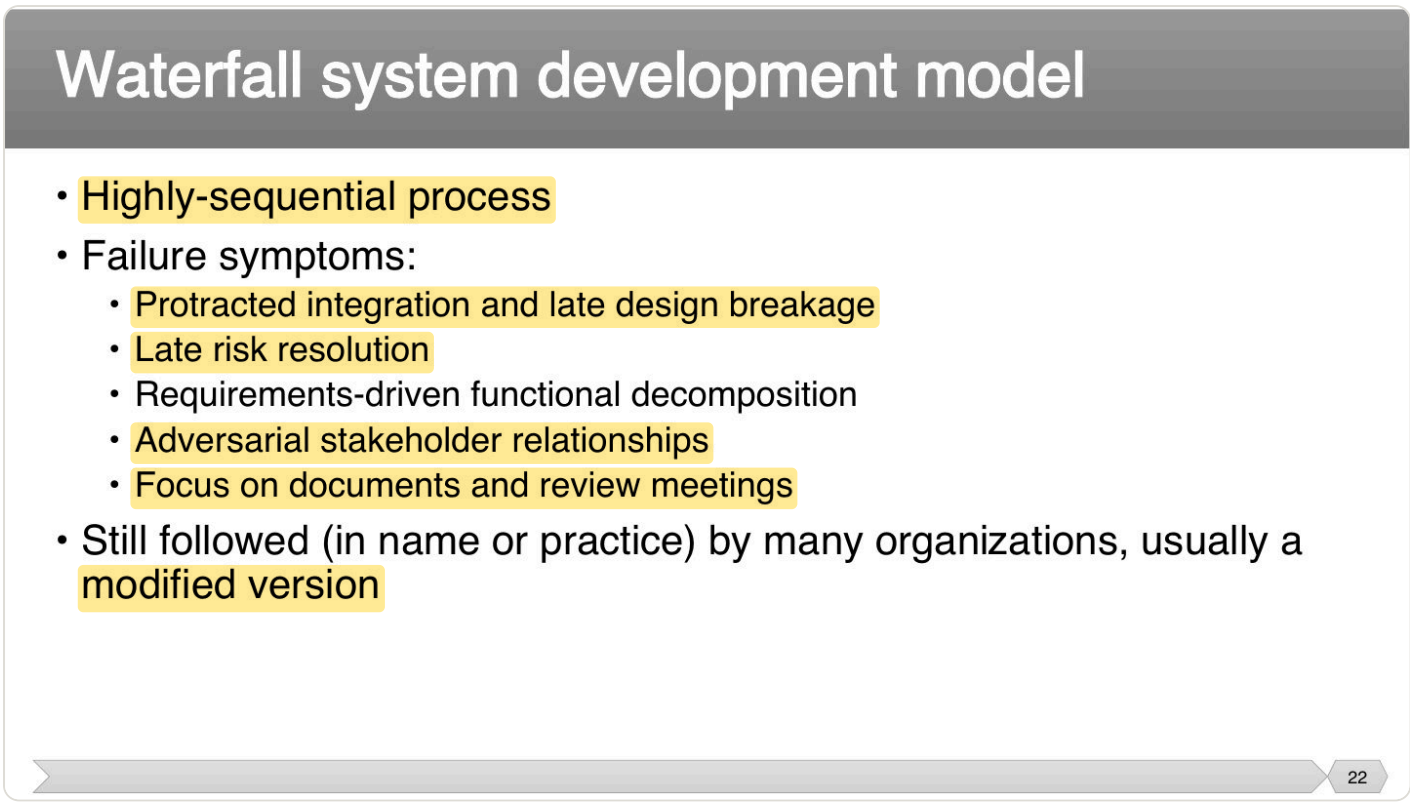


NOTES

1. Royce's waterfall: plan → **requirements** → design → coding → testing, with dashed **feedback loops** back to the previous step. The note on the left says those loops were in Royce's paper but have been ignored by managers and practitioners alike.

2. On the right, a more detailed version (concept → requirements → architecture → detailed design → coding → **testing** → deployment). Either way, testing comes last, which is why problems are found late.

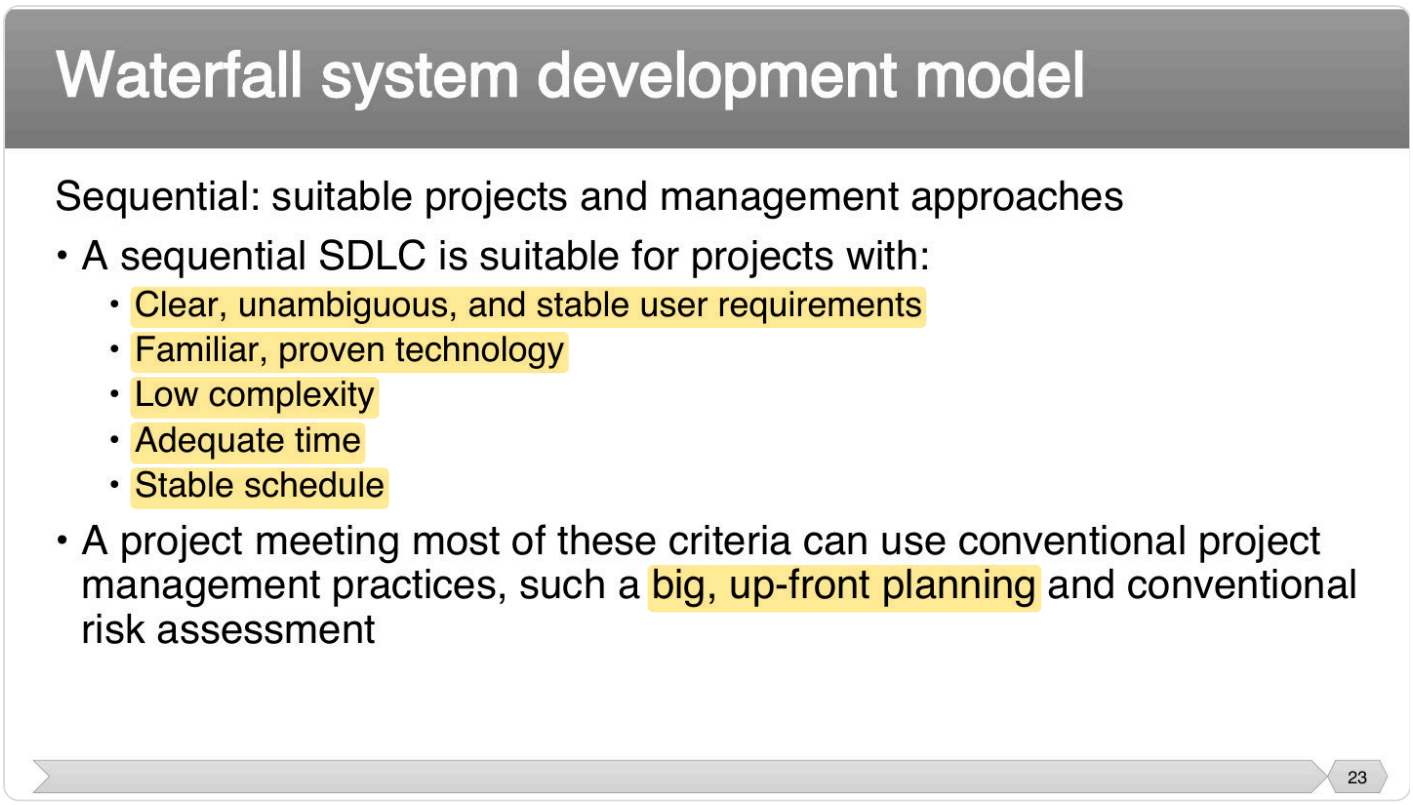
 Waterfall system development model



NOTES

1. Waterfall is a **highly sequential** process: no phase overlaps.
2. Failure symptoms: **protracted integration and late design breakage** (everything is integrated at the end, where design flaws appear) and **late risk resolution** (the big risks only show in testing). Also requirements-driven functional decomposition.
3. **Adversarial stakeholder relationships** (arguing over signed documents) and a **focus on documents and reviews** instead of working software.
4. Many organizations still follow it, usually a **modified version**.

 Waterfall system development model



NOTES

1. A **sequential SDLC** suits projects with **clear, unambiguous and stable requirements**, **familiar, proven technology**, **low complexity**, **adequate time** and a **stable schedule**. In short: everything is known.
2. Such a project can use conventional PM practices: **big up-front planning** and conventional risk assessment.

TIPS & TRICKS

- **FALConS**: Familiar technology, Adequate time, Low complexity, Clear requirements, Stable schedule.

EXAM ANGLE

Past exam · Midterm 251 · V1 · Case Q2 (also V2) Q1

Given the evolving requirements (student usability, advisor requests, IT integration rules), which development approach (Agile, Waterfall, or Hybrid) would you recommend, and why would it be the best fit for this project?

Answer: Agile (Scrum) (0.5), because it handles the evolving requirements from students, advisors and IT, delivers iteratively so each sprint gets feedback, and integrates with DevOps pipelines for frequent tested releases (0.5). Justify it against the slide criteria: the requirements are reasonably clear but not stable, so waterfall's "clear, unambiguous, stable requirements" condition fails.

Prediction Q2

Which project suits a sequential (waterfall) SDLC?

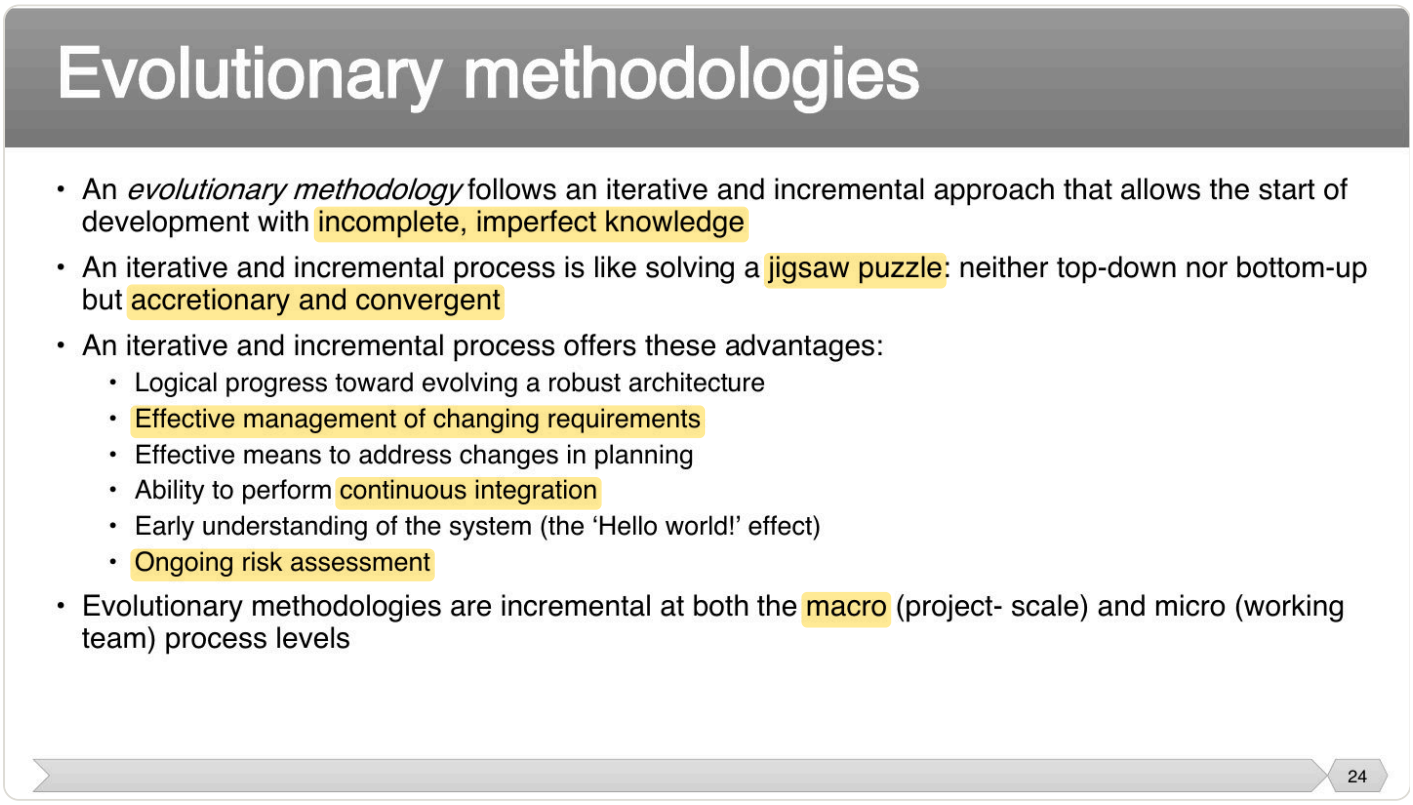
A Unclear requirements and new technology

B High complexity with a very short schedule

C **Stable requirements, familiar technology, low complexity**

D Requirements expected to change weekly

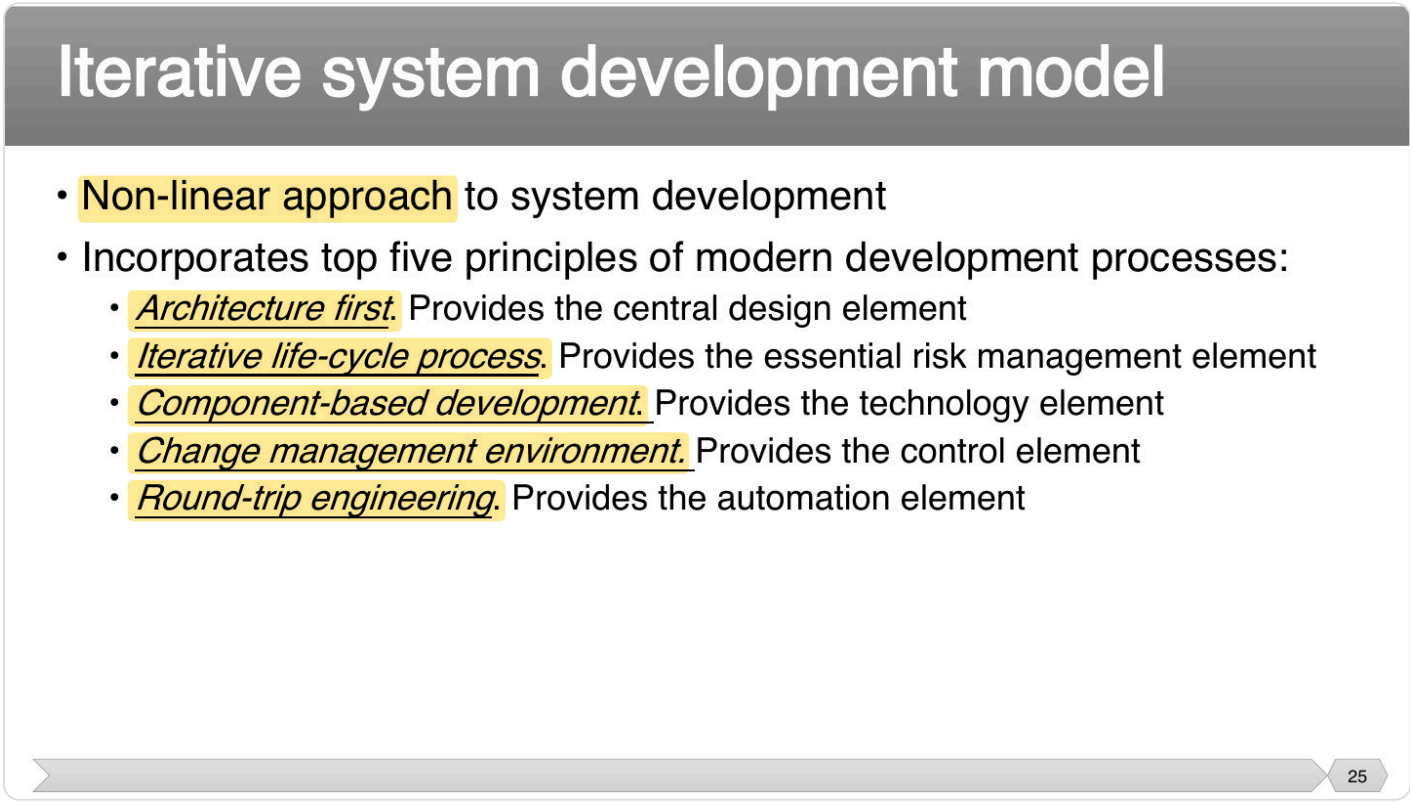
SLIDE 24 / 98 **Evolutionary methodologies**



NOTES

1. An **evolutionary** methodology is iterative and incremental: you can start with **incomplete, imperfect knowledge**.
2. It is like a **jigsaw puzzle**: neither top-down nor bottom-up, but **accretionary and convergent** (pieces accumulate until the picture emerges).
3. Advantages: an architecture that evolves logically, **effective management of changing requirements** and planning changes, **continuous integration**, early understanding (the "Hello world!" effect) and **ongoing risk assessment**.
4. It is incremental at both the **macro** (project) and micro (team) levels.

SLIDE 25 / 98 Iterative system development model



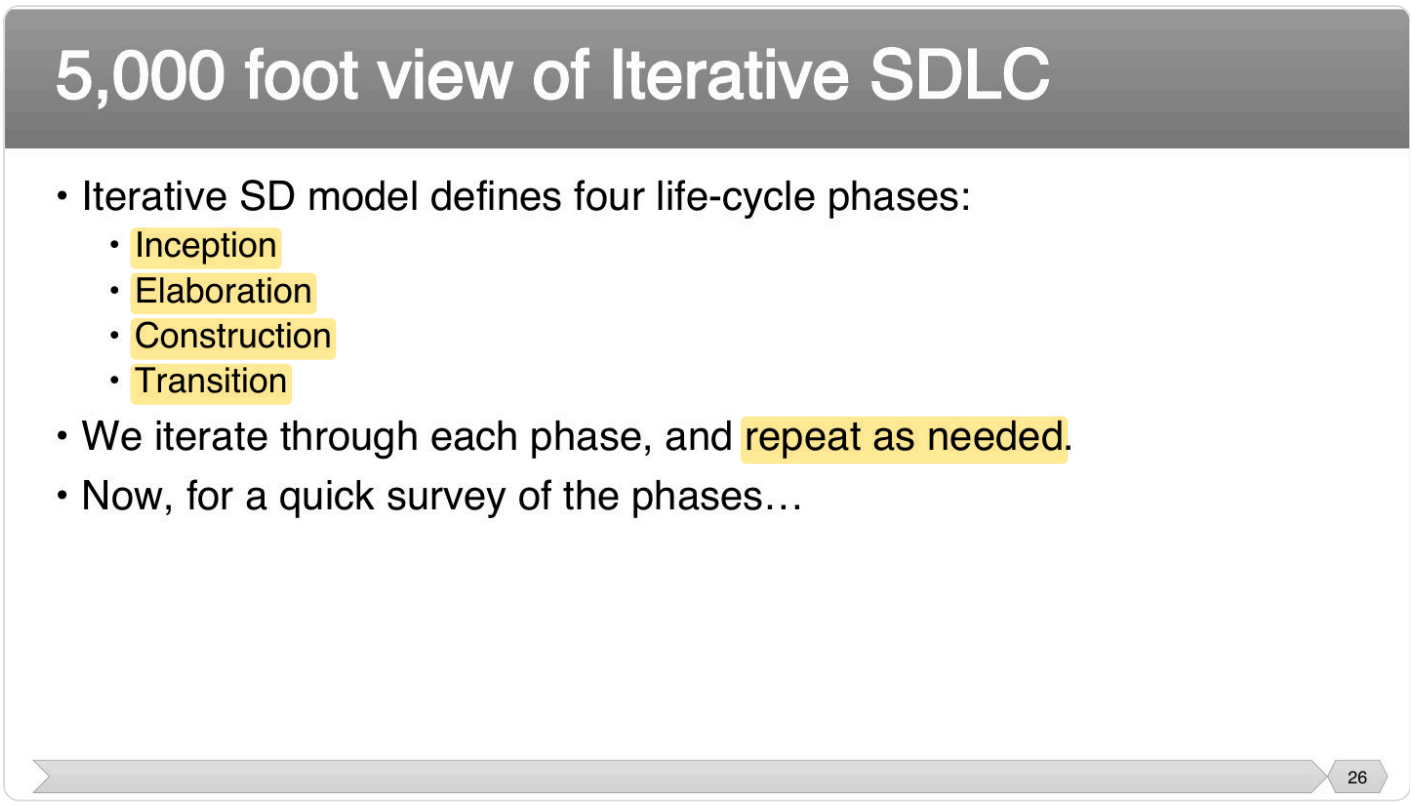
NOTES

1. The iterative model is a **non-linear** approach.
2. It uses five principles of modern development, each giving one element: **architecture first** (the design element), **iterative life cycle** (the risk-management element)...
3. ...**component-based development** (technology), **change management environment** (control) and **round-trip engineering** (automation: keeping models and code in sync).

TIPS & TRICKS

- **CIRCA**: Component-based development, Iterative life cycle, Round-trip engineering, Change management environment, Architecture first.

 5,000 foot view of Iterative SDLC



NOTES

1. The iterative SDLC has four phases: **inception → elaboration → construction → transition** (the Unified Process phases).
2. Each phase can contain several iterations, **repeated as needed**.

TIPS & TRICKS

- **InfECT**, already in order: Inception, Elaboration, Construction, Transition.

EXAM ANGLE

Prediction Q1

Which is the correct order of the iterative SDLC phases?

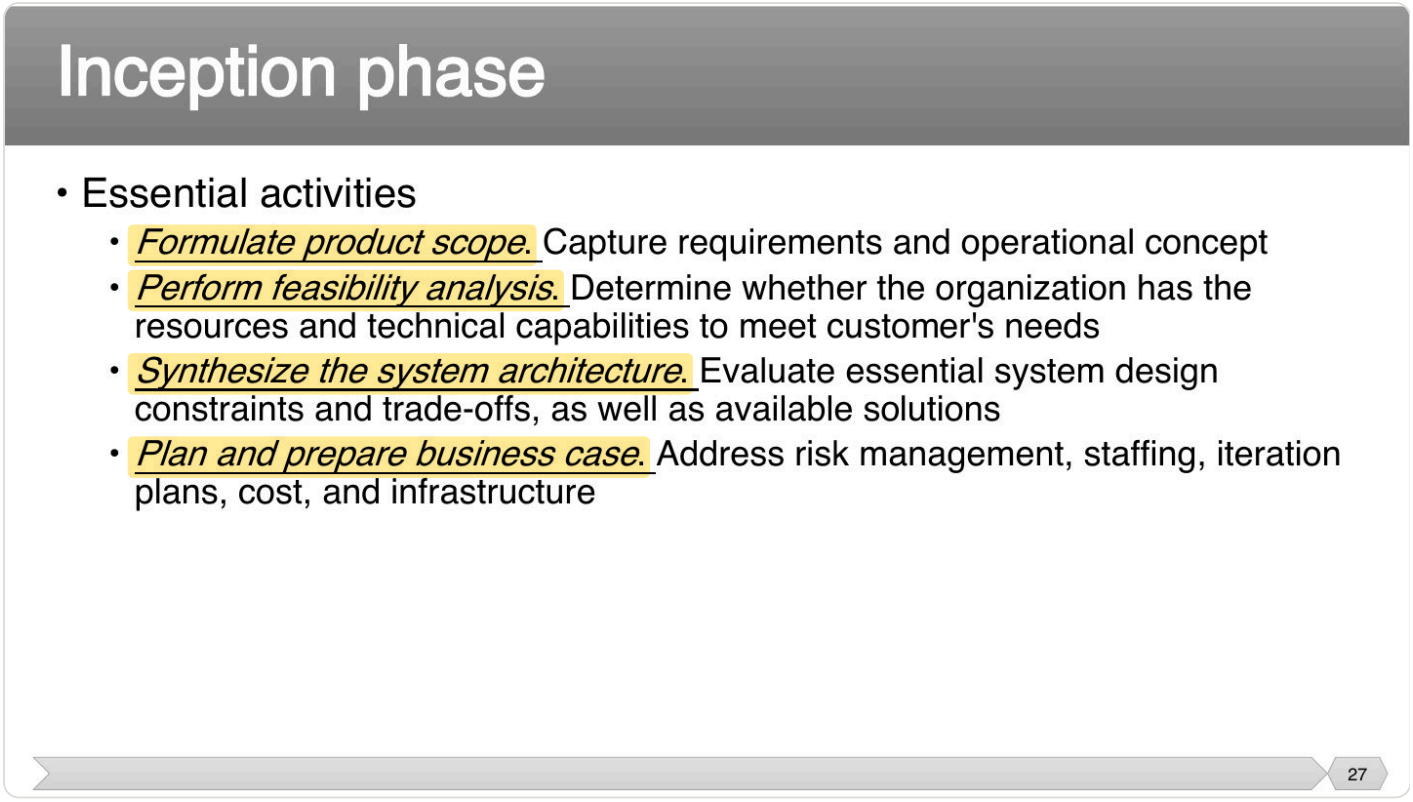
A Inception, construction, elaboration, transition

B Planning, design, build, deploy

C Elaboration, inception, construction, transition

D **Inception, elaboration, construction, transition**

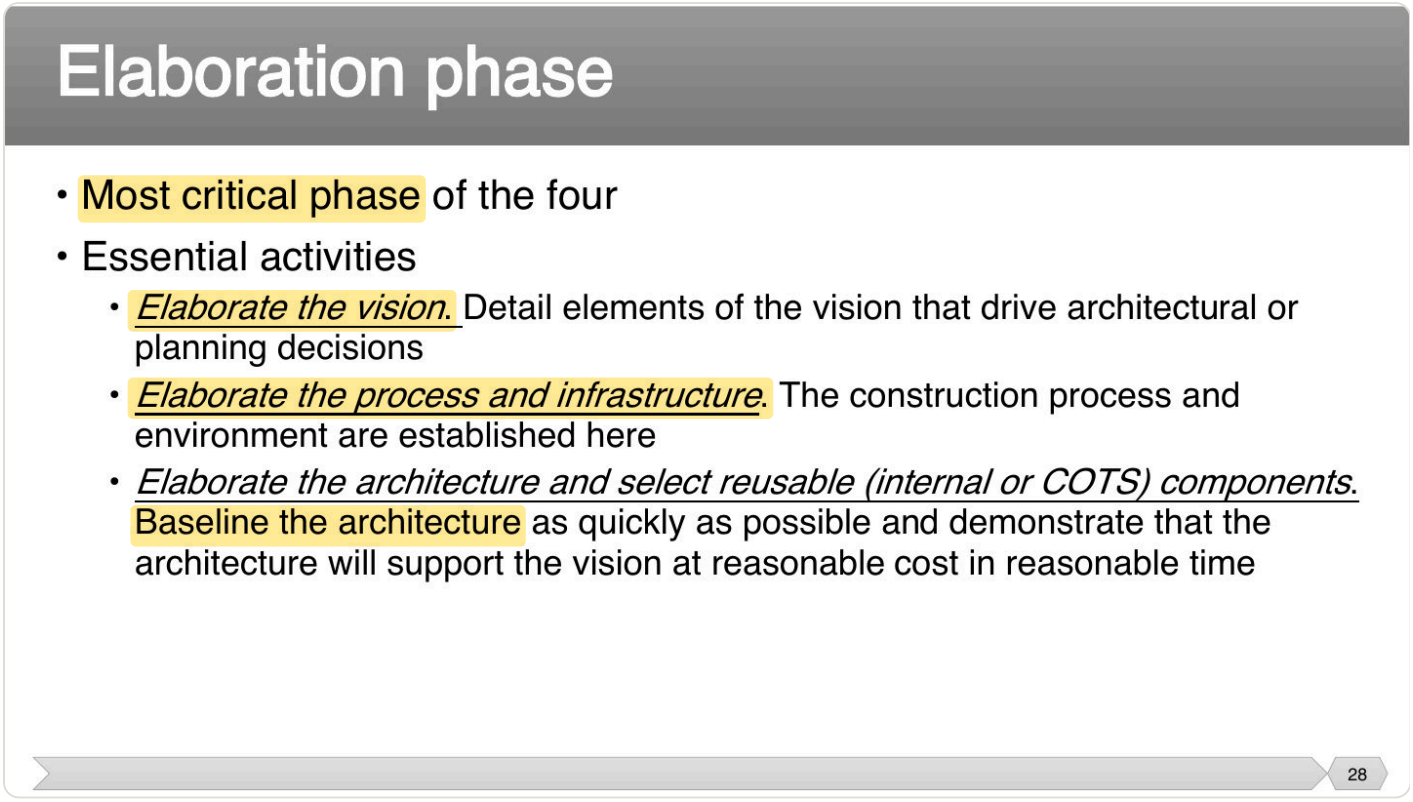
 Inception phase



NOTES

1. **Inception**: **formulate the product scope** (requirements and operational concept)...
2. ...**analyze feasibility** (does the organization have the resources and technical capability?)...
3. ...**synthesize a candidate architecture** (design constraints, trade-offs, available solutions)...
4. ...and **prepare the business case**: risk management, staffing, iteration plans, cost and infrastructure.

 Elaboration phase



NOTES

1. **Elaboration** is the **most critical** of the four phases: it removes the biggest risks before heavy spending starts.
2. **Elaborate the vision**: detail the parts of the vision that drive architecture or planning decisions.
3. **Elaborate the process and infrastructure**: the construction process and environment are set up here.
4. **Baseline the architecture** as quickly as possible and show it supports the vision at reasonable cost and time, selecting reusable or COTS components.

EXAM ANGLE

Prediction Q1

Which iterative phase is described as the most critical?

- A **Elaboration**
- B Transition
- C Construction
- D Inception

 Construction phase

Construction phase

- Essential activities
  - *Achieve useful versions* (intermediate, alpha, beta, and other test releases)
  - *Perform resource management, control, and process optimization*
  - *Complete component development and test*
  - *Assess product releases against acceptance criteria*

29

NOTES

1. **Construction**: produce **useful versions** (intermediate, alpha, beta and test releases)...
2. ...perform **resource management, control and process optimization**...
3. ...complete **component development and testing**...
4. ...and assess each release against **acceptance criteria**.

 Transition phase

Transition phase

- Essential activities
  - *Perform deployment-specific engineering tasks.* Commercial packaging and production, sales kit development, field personnel training
  - *Assess deployment baselines against complete vision and acceptance criteria.* Examine and compare what is being delivered to what was envisioned and delineated by acceptance criteria
  - Plan for next iteration

NOTES

1. **Transition**: **deployment-specific engineering**: packaging, production, sales kits and training of field staff.
2. **Assess the deployed baseline** against the full vision and acceptance criteria: compare what is delivered with what was envisioned.
3. **Plan the next iteration** or release.

 Comparative expenditure profiles

Comparative expenditure profiles

Waterfall		Iterative	
Activity	Cost	Cost	Activity
Management	5%	10%	Management
Requirements	5%	10%	Requirements
Design	10%	15%	Design
Code & Unit Testing	30%	25%	Implementation
Integration & Test	40%	25%	Assessment
Deployment	5%	5%	Deployment
Environment	5%	10%	Environment
Total	100%	100%	Total

NOTES

1. **Waterfall** spends most of its budget late: 30% on code and unit test and **40% on integration and test**, because problems surface only when everything comes together.
2. **Iterative** spreads the cost: more on **management** (10%), requirements (10%) and design (15%), with implementation and **assessment** at 25% each, since integration happens continuously.

EXAM ANGLE

Prediction Q1

In the waterfall expenditure profile, which activity takes the largest share of cost?

- A **Integration and test (40%)**
- B Requirements (5%)
- C Design (10%)
- D Management (5%)

 Suitable Projects And Management Approaches

Suitable Projects And Management Approaches

- An evolutionary SDLC is suitable for projects with:
  - Reasonably–but not perfectly–clear user requirements
  - Unfamiliar or unproven technology
  - High complexity
  - Short time schedule
  - Schedule variability
- Such a project would use rolling wave planning rather than big, up-front planning and use a continuous, adaptive approach to risk assessment and management

32

NOTES

1. An **evolutionary SDLC** suits projects with **reasonably but not perfectly clear requirements**, **unfamiliar technology**, **high complexity**, a **short schedule** and **schedule variability**. It is the mirror image of slide 23.
2. Such a project uses **rolling wave planning** (detailed plans for the near term, rough plans for later) instead of big up-front planning, and a **continuous, adaptive** approach to risk.

TIPS & TRICKS

- **RUSHeS**, the opposite of slide 23: Reasonably clear requirements, Unfamiliar technology, Short time, High complexity, Schedule variability.

EXAM ANGLE

Past exam · Midterm 251 · V1 · MCQ 6 (also V2) Q1

In which type of project would an iterative development approach be most beneficial?

A **A project where requirements are unclear and expected to evolve over time.**

> Iterative development refines ideas through feedback, which suits unclear or evolving requirements.

B A project with well-defined, unchanging requirements from the start.

C A project that must deliver the entire product only at final completion.

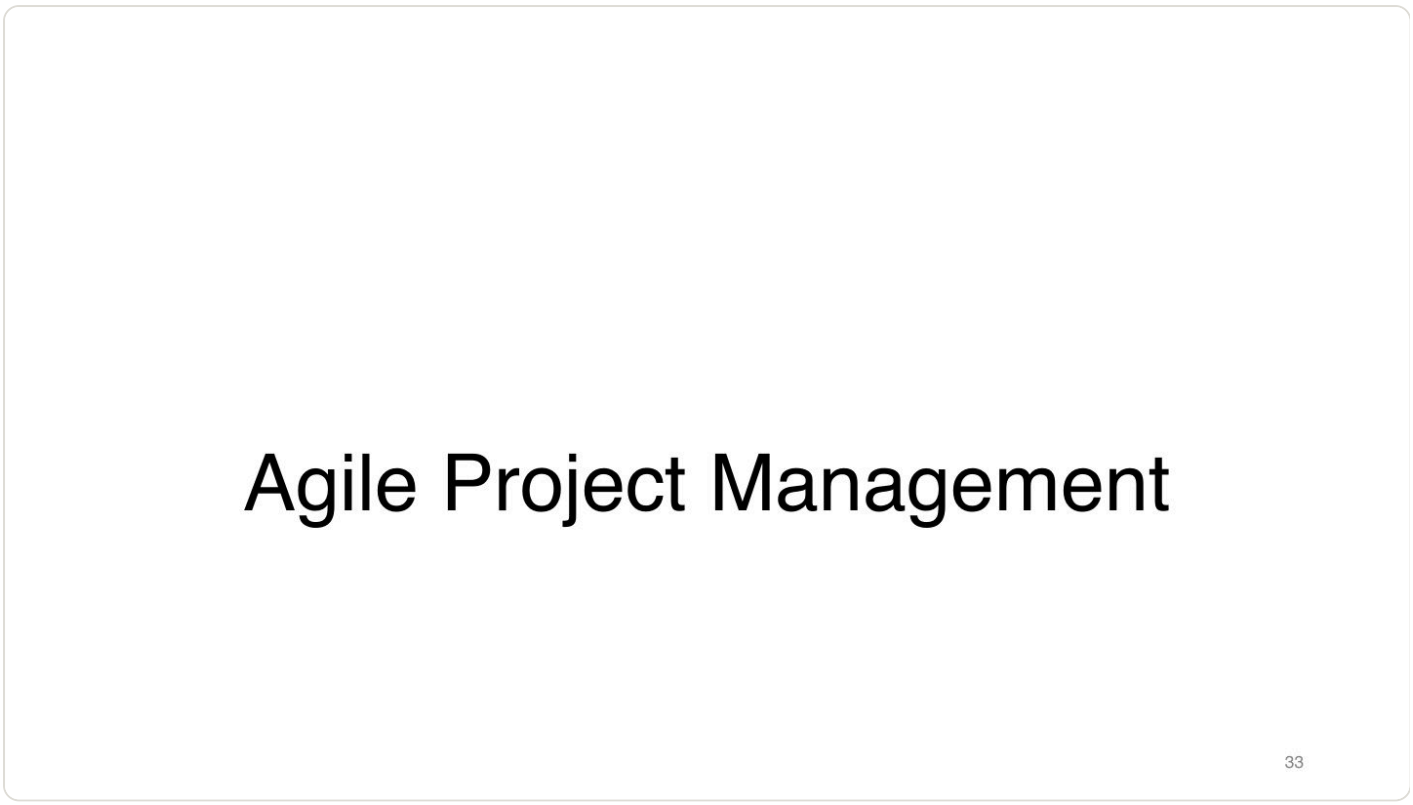
D A project that cannot accommodate stakeholder feedback during execution.

Past exam · Midterm 251 · V1 · Case Q2 (also V2) Q2

Given the evolving requirements (student usability, advisor requests, IT integration rules), which development approach (Agile, Waterfall, or Hybrid) would you recommend, and why would it be the best fit for this project?

Answer: Agile (Scrum) (0.5), because it handles the evolving requirements from students, advisors and IT, delivers iteratively so each sprint gets feedback, and integrates with DevOps pipelines for frequent tested releases (0.5). Justify it against the slide criteria: the requirements are reasonably clear but not stable, so waterfall's "clear, unambiguous, stable requirements" condition fails.

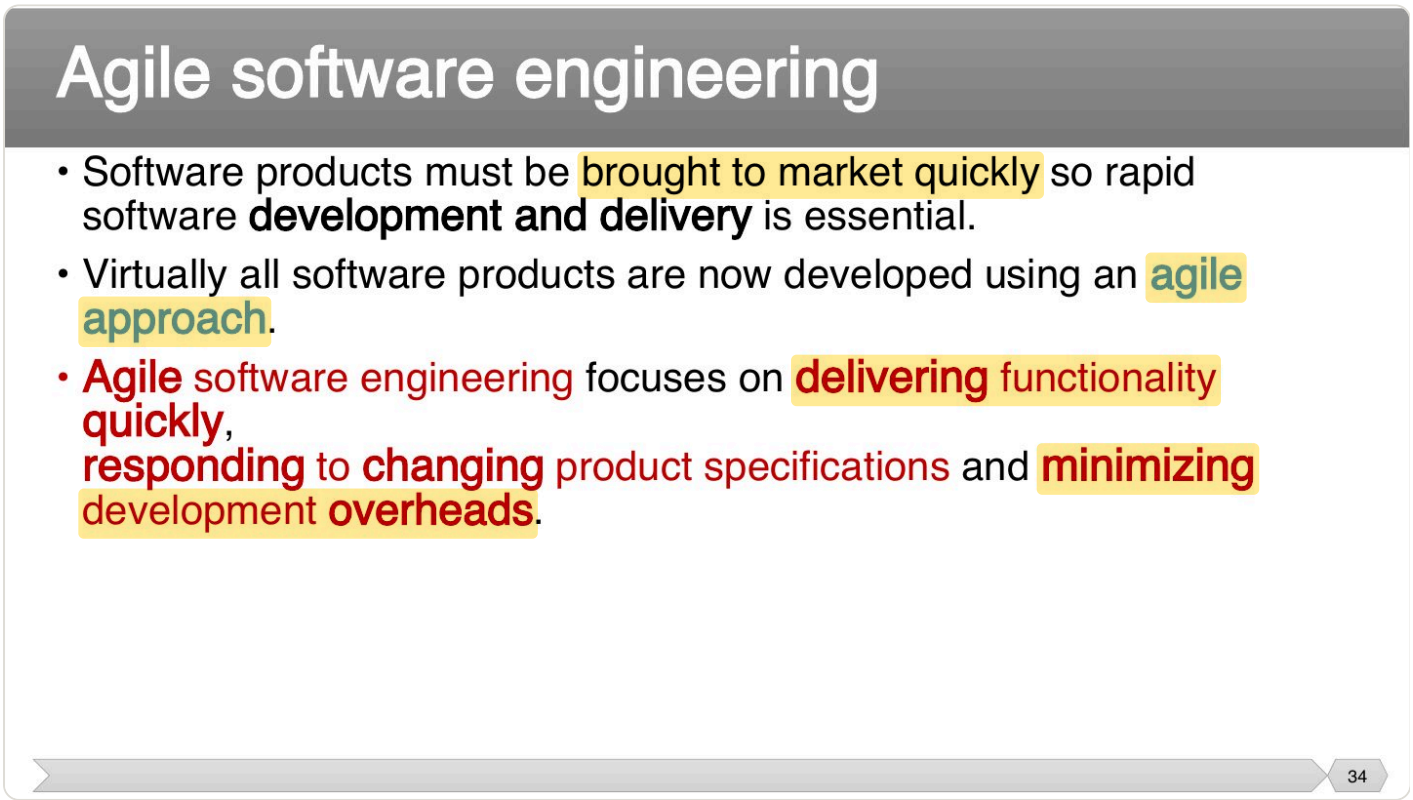
SLIDE 33 / 98 **Slide 33**



NOTES

1. Section divider: **Agile Project Management**.

SLIDE 34 / 98 **Agile software engineering**

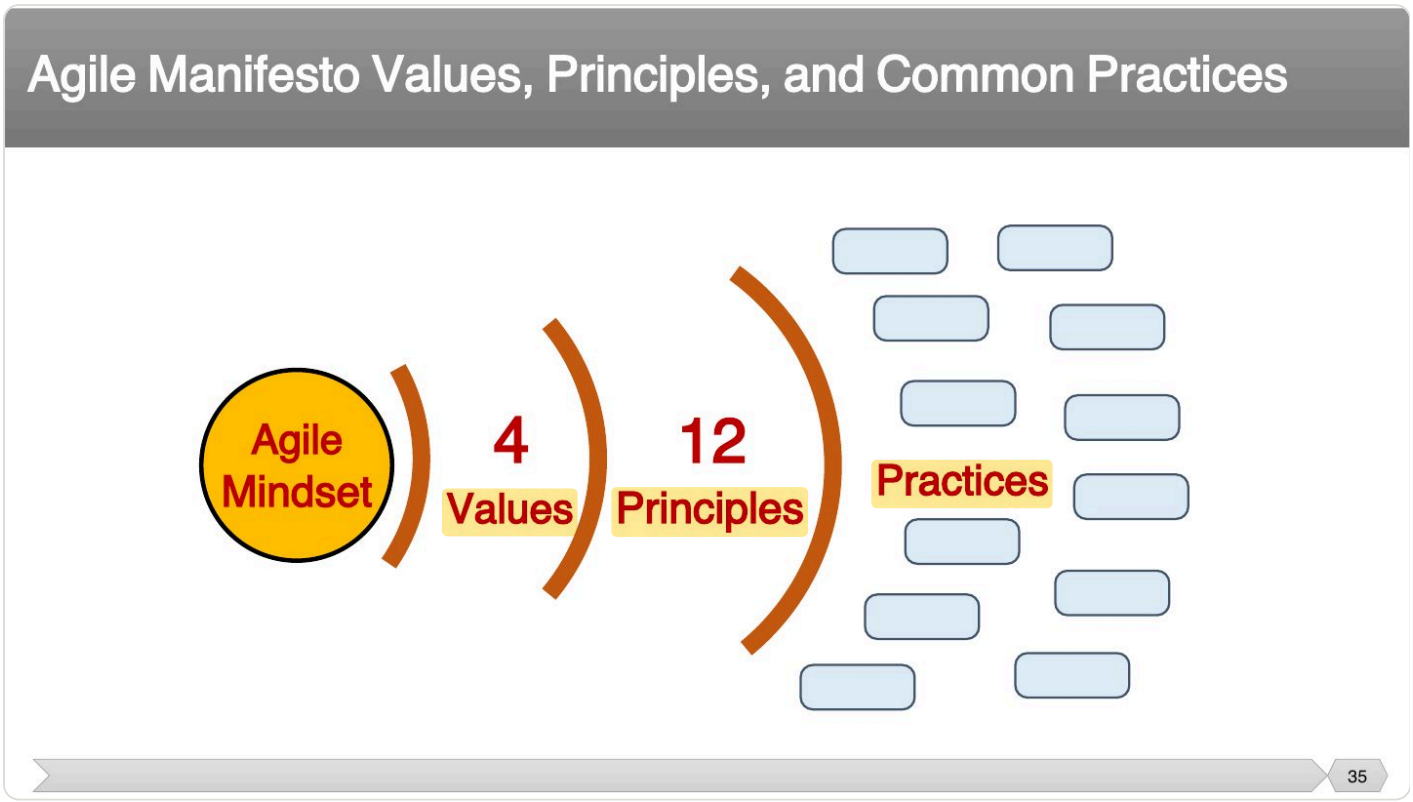


NOTES

1. Products must reach the market **quickly**, so rapid development and delivery are essential.

2. Virtually all software products are now developed with an **agile approach**.
3. Agile focuses on **delivering functionality quickly**, responding to changing specifications and **minimizing overhead**.

SLIDE 35 / 98 Agile Manifesto Values, Principles, and Common Practices

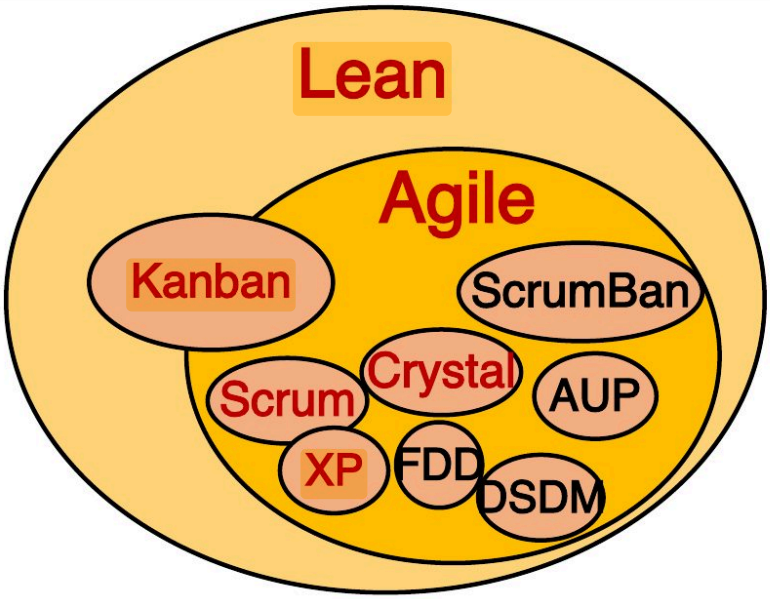


NOTES

1. Agile starts with a **mindset**, defined by **4 values**...
2. ...guided by **12 principles**...
3. ...and expressed through many **practices** (stand-ups, user stories, TDD), which a team chooses to fit its needs.

SLIDE 36 / 98 **Lean**

Agile is a Blanket Term for Many Approaches



36

NOTES

1. **Agile** is an umbrella term: Scrum, XP, Kanban, ScrumBan, Crystal, FDD, AUP and DSDM all sit inside it.
2. **Lean** is broader still, and agile sits inside it.

SLIDE 37 / 98 **Agile Manifesto and Mindset**

Agile Manifesto and Mindset

Agile is a mindset defined by 4 values, guided by 12 principles, and manifested through many different practices.

Agile practitioners select practices based on their needs.

37

NOTES

1. Agile is a mindset **defined by 4 values**, **guided by 12 principles** and expressed through **many practices**.
2. Practitioners **select practices** to fit their needs; there is no single “agile process”.

EXAM ANGLE

Prediction Q1

Agile is best described as:

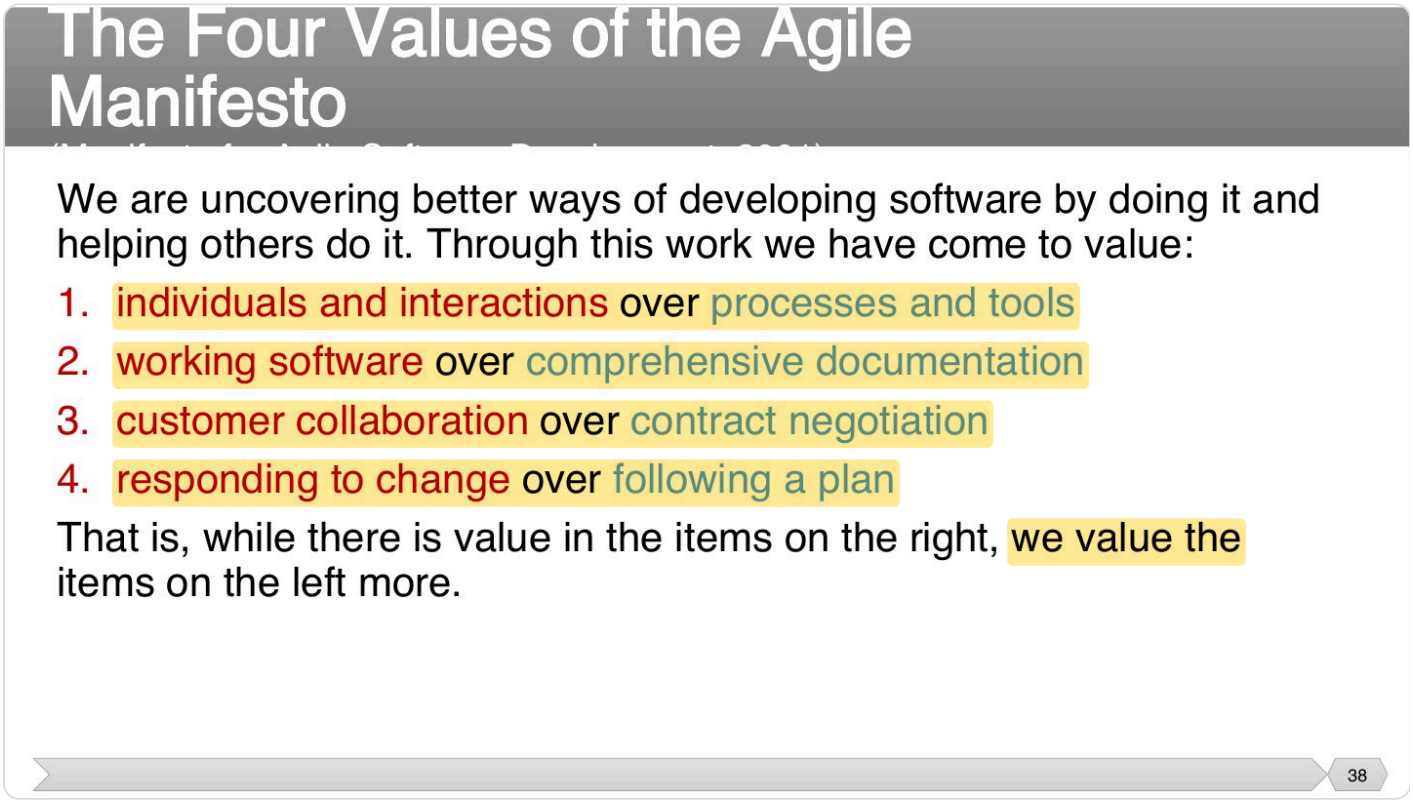
A A documentation standard

B A type of contract

C **A mindset defined by 4 values and 12 principles, expressed through many practices**

D A single fixed method

SLIDE 38 / 98 **Manifesto**



NOTES

1. **Individuals and interactions over processes and tools**: people talking solves more than a tool or process.

2. **Working software over comprehensive documentation**: software that runs is the real evidence of progress.
3. **Customer collaboration over contract negotiation**: work with the customer, rather than argue over the contract.
4. **Responding to change over following a plan**.
5. The items on the right still have value; the **left is valued more**. A common trap: agile does not say "no documentation".

TIPS & TRICKS

- The four values, in order: **I W**ill **C**hange **R**eadily. Individuals and interactions, Working software, Customer collaboration, Responding to change.

EXAM ANGLE

Prediction Q1

Which is one of the four values of the Agile Manifesto?

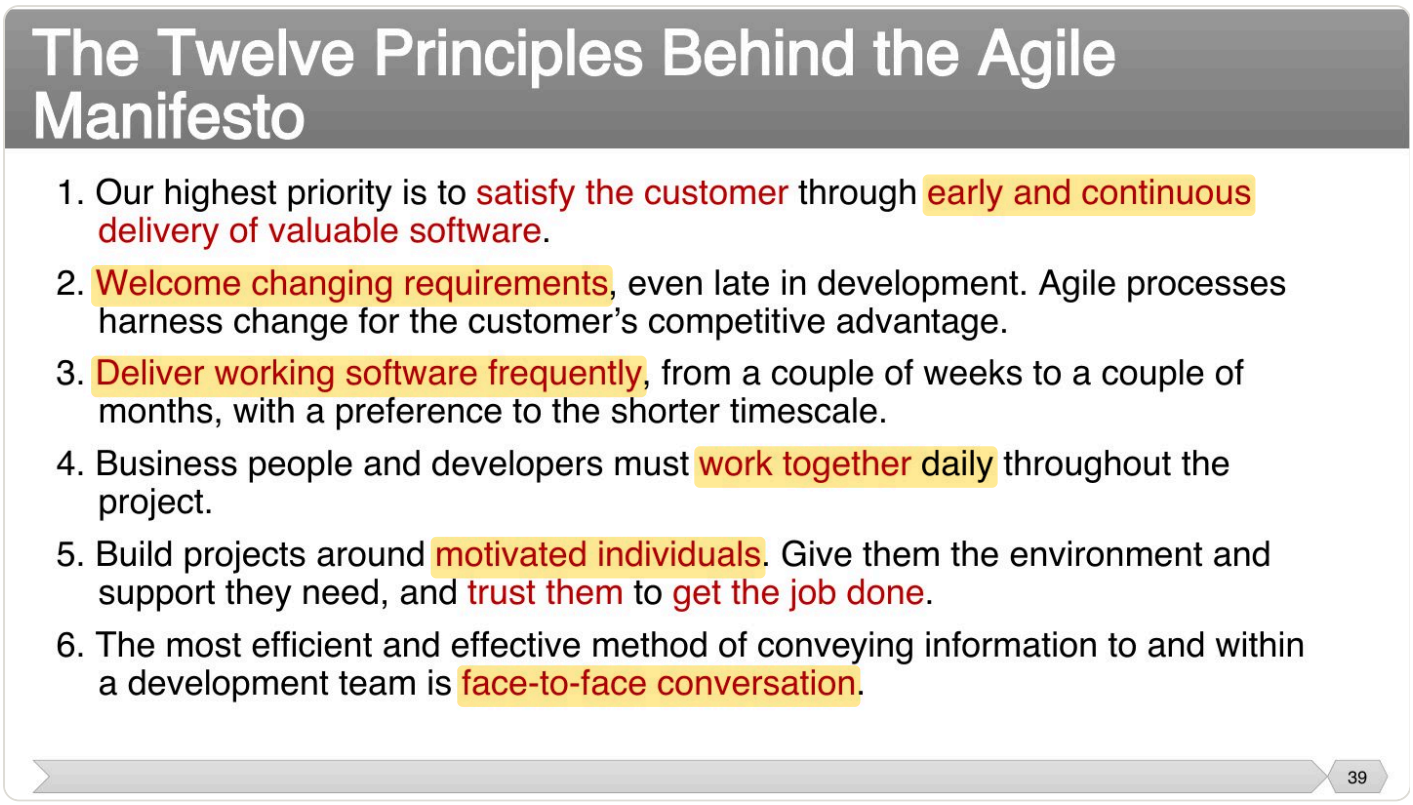
- **A Customer collaboration over contract negotiation**
- B Following a plan over responding to change
- C Processes and tools over individuals and interactions
- D Comprehensive documentation over working software

Prediction Q2

The Agile Manifesto says documentation has no value.

- T True
- **F False**

> There is value in the items on the right; the items on the left are valued more.



NOTES

1. 1. The highest priority is to satisfy the customer through **early and continuous delivery** of valuable software.
2. 2. **Welcome changing requirements**, even late, for the customer's competitive advantage.
3. 3. **Deliver working software frequently**, from a couple of weeks to a couple of months, preferring the shorter.
4. 4. Business people and developers **work together daily**.
5. 5. Build projects around **motivated individuals**, give them support and trust them.
6. 6. The most efficient and effective communication is **face-to-face conversation**.

 Manifesto

The Twelve Principles Behind the Agile Manifesto

7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity—the art of maximizing the amount of work not done—is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

NOTES

1. 7. **Working software is the primary measure of progress**, not documents or hours.
2. 8. **Sustainable development**: sponsors, developers and users can keep a constant pace indefinitely (no death marches).
3. 9. Continuous attention to **technical excellence** and good design enhances agility.
4. 10. **Simplicity**, the art of maximizing the amount of work not done, is essential.
5. 11. The best architectures, requirements and designs emerge from **self-organizing teams**.
6. 12. At regular intervals the team **reflects** and adjusts its behavior (the retrospective).

EXAM ANGLE

Prediction Q1

According to the agile principles, the primary measure of progress is:

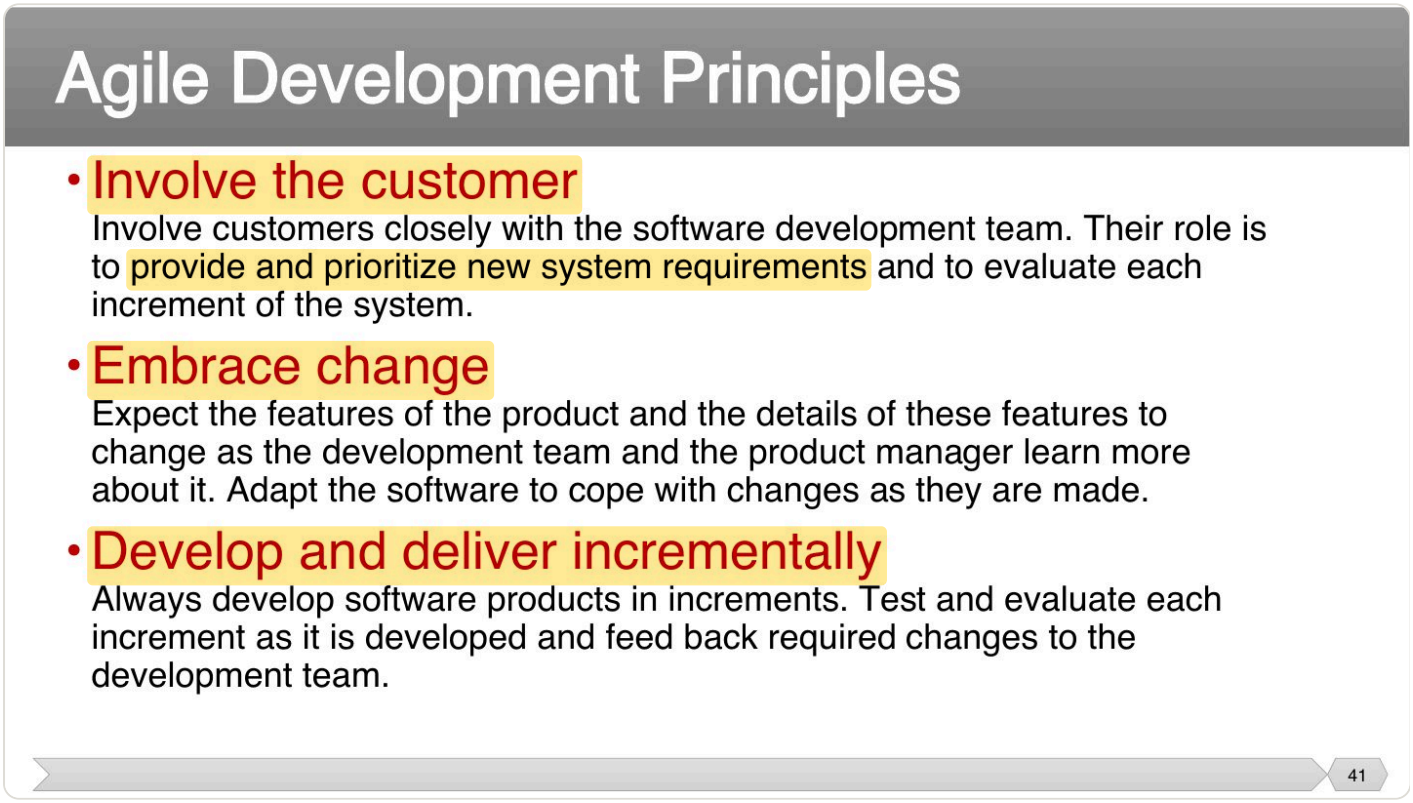
A Hours worked

B **Working software**

C Documents completed

D Lines of code

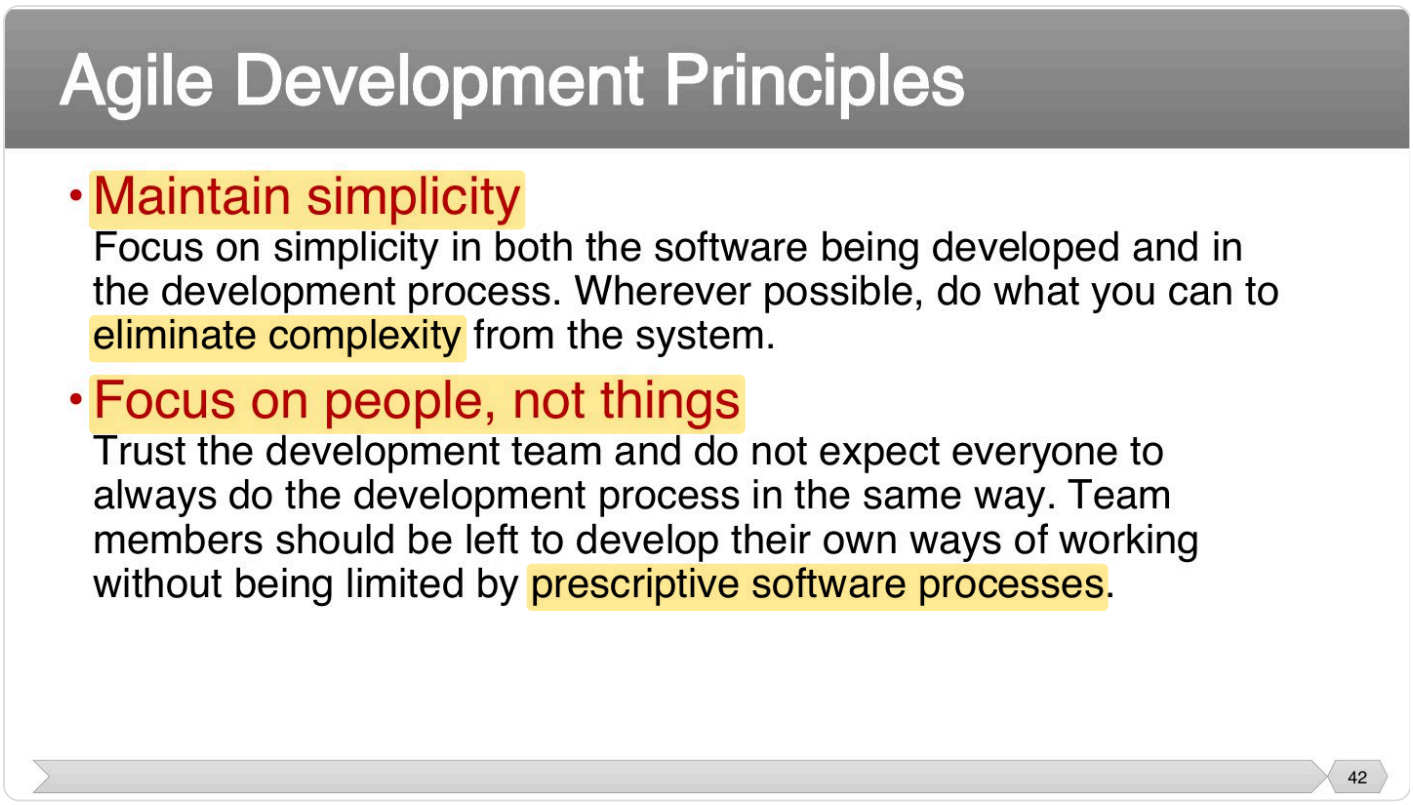
SLIDE 41 / 98 **Agile Development Principles**



NOTES

1. **Involve the customer** closely: they provide and **prioritize requirements** and evaluate each increment.
2. **Embrace change**: expect features and their details to change as the team and product manager learn; adapt the software as changes come.
3. **Develop and deliver incrementally**: test and evaluate each increment and feed back the changes needed.

 Agile Development Principles

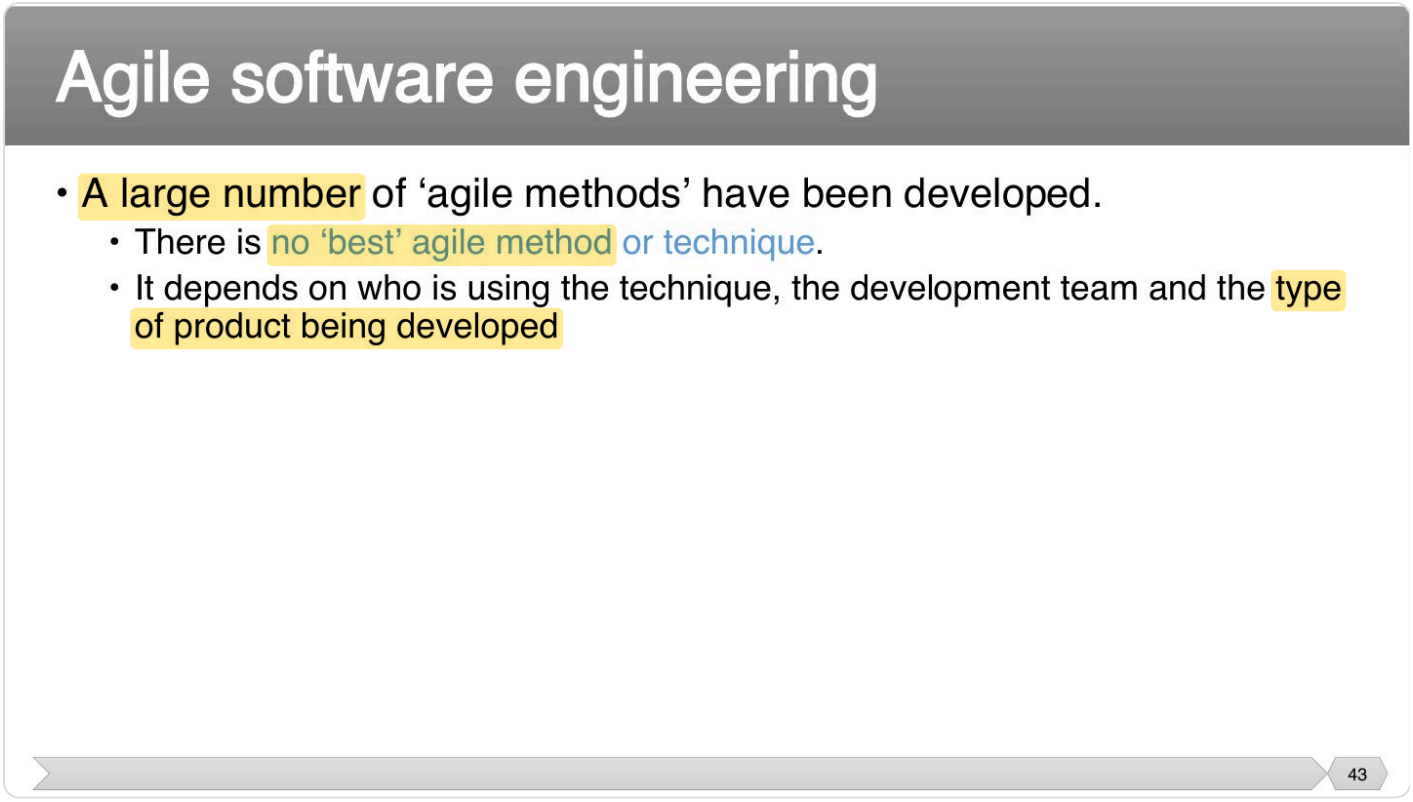


NOTES

1. **Maintain simplicity** in both the software and the process; **eliminate complexity** wherever possible.
2. **Focus on people, not things**: trust the team and let members develop their own ways of working instead of being limited by **prescriptive processes**.

TIPS & TRICKS

- Slides 41 and 42 together, **MIdFiEID**: Maintain simplicity, Involve the customer, Focus on people, Embrace change, Develop incrementally.



NOTES

1. **Many** agile methods have been developed...
2. ...and there is **no single best one**...
3. ...it depends on who uses it, the team, and the **type of product** (tailoring again).

 Agile methods

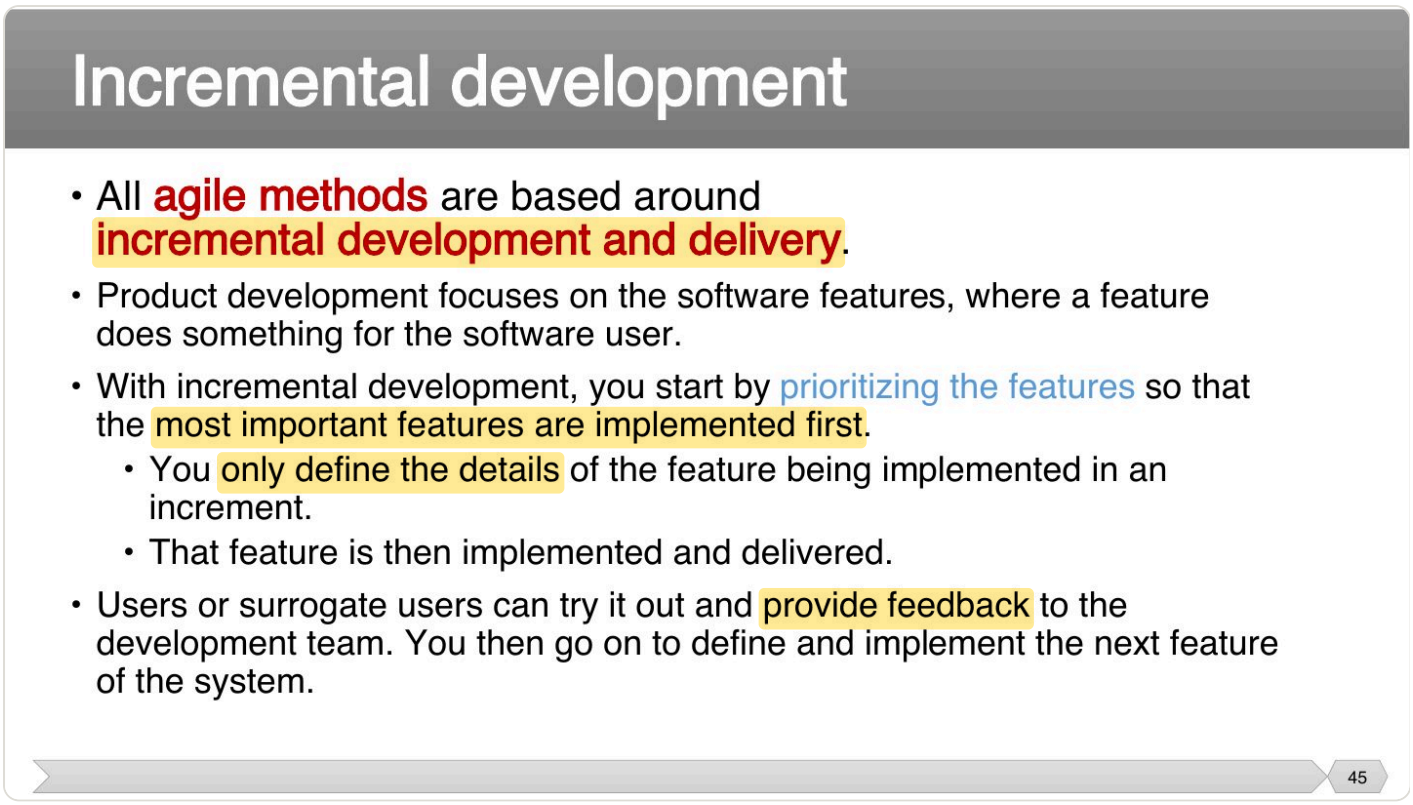
Agile methods

- Plan-driven development evolved to support the engineering of large, long-lifetime systems
  - This approach is based on controlled and rigorous software development processes that include detailed project planning, requirements specification and analysis and system modelling.
  - However, plan-driven development involves significant overheads and documentation and it does not support the rapid development and delivery of software.
- Agile methods were developed in the 1990s to address this problem.
  - These methods focus on the software rather than its documentation, develop software in a series of increments and aim to reduce process bureaucracy as much as possible.

NOTES

1. **Plan-driven** development evolved for **large, long-lifetime** systems...
2. ...using controlled, rigorous processes with **detailed planning**, requirements specification and system modeling.
3. But it carries **significant overhead** and documentation and doesn't support rapid delivery.
4. Agile methods appeared in the **1990s** to fix this.
5. They focus on the software rather than its documentation, develop in increments and **reduce bureaucracy** as much as possible.

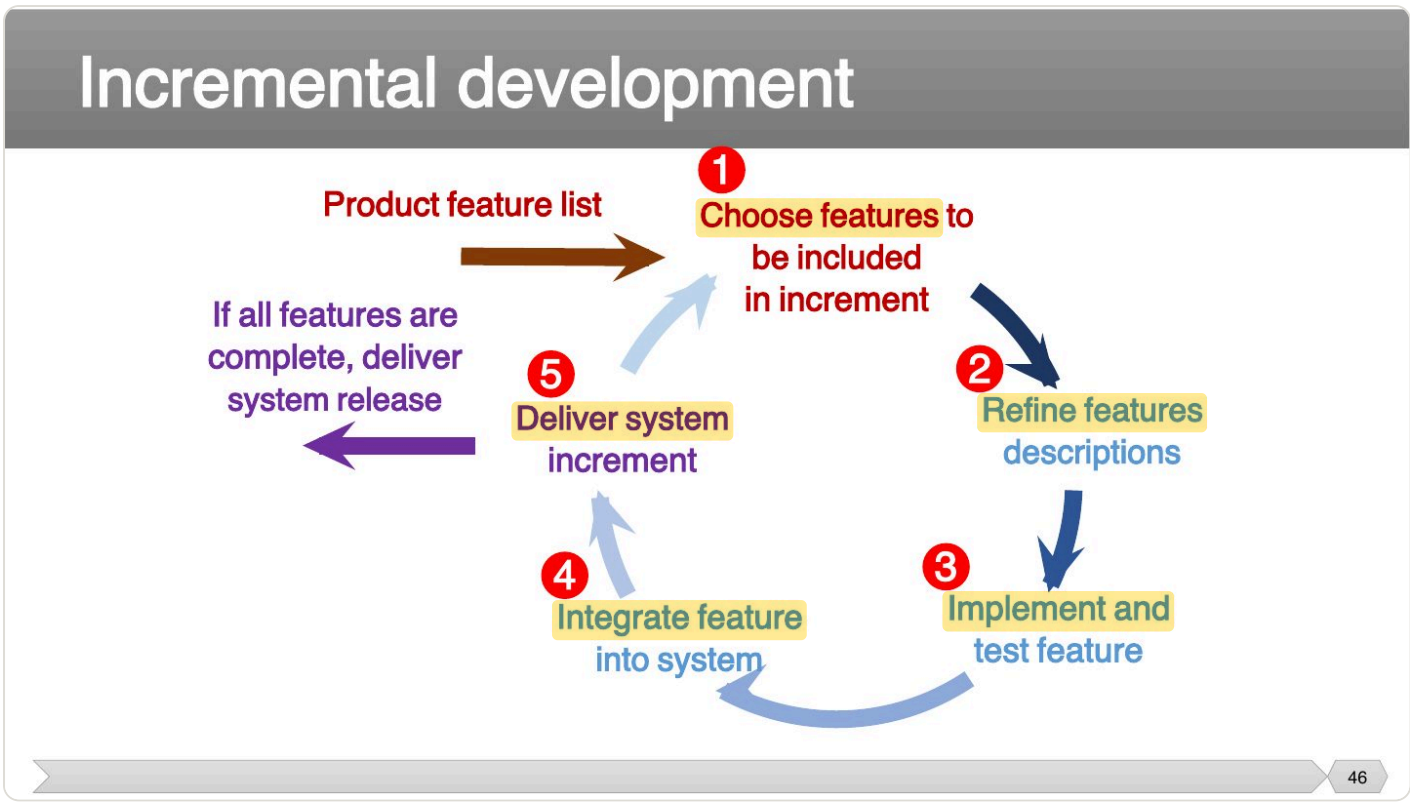
 Incremental development



NOTES

1. All agile methods are based on **incremental development and delivery**, organized around features (something useful to the user).
2. Prioritize features so the **most important are implemented first**.
3. Only define the **details of the feature being built in this increment**; the rest stays rough until it is needed.
4. Implement and deliver it, let users **give feedback**, then move to the next feature.

 Incremental development



NOTES

1. The incremental cycle, starting from the product feature list: (1) **choose** features for the increment...
2. (2) **refine** their descriptions...
3. (3) **implement and test** each feature...
4. (4) **integrate** it into the system...
5. (5) **deliver** the increment; when all features are complete, deliver the system release.

 activities

Incremental development activities

1. **Choose features to be included in an increment**
   Using the list of features in the planned product, select those features that can be implemented in the next product increment.
2. **Refine feature descriptions**
   Add detail to the feature descriptions so that the team have a common understanding of each feature and there is sufficient detail to begin implementation.
3. **Implement and test**
   Implement the feature and develop automated tests for that feature that show that its behaviour is consistent with its description.
4. **Integrate feature and test**
   Integrate the developed feature with the existing system and test it to check that it works in conjunction with other features.
5. **Deliver system increment**
   Deliver the system increment to the customer or product manager for checking and comments. If enough features have been implemented, release a version of the system for customer use.

47

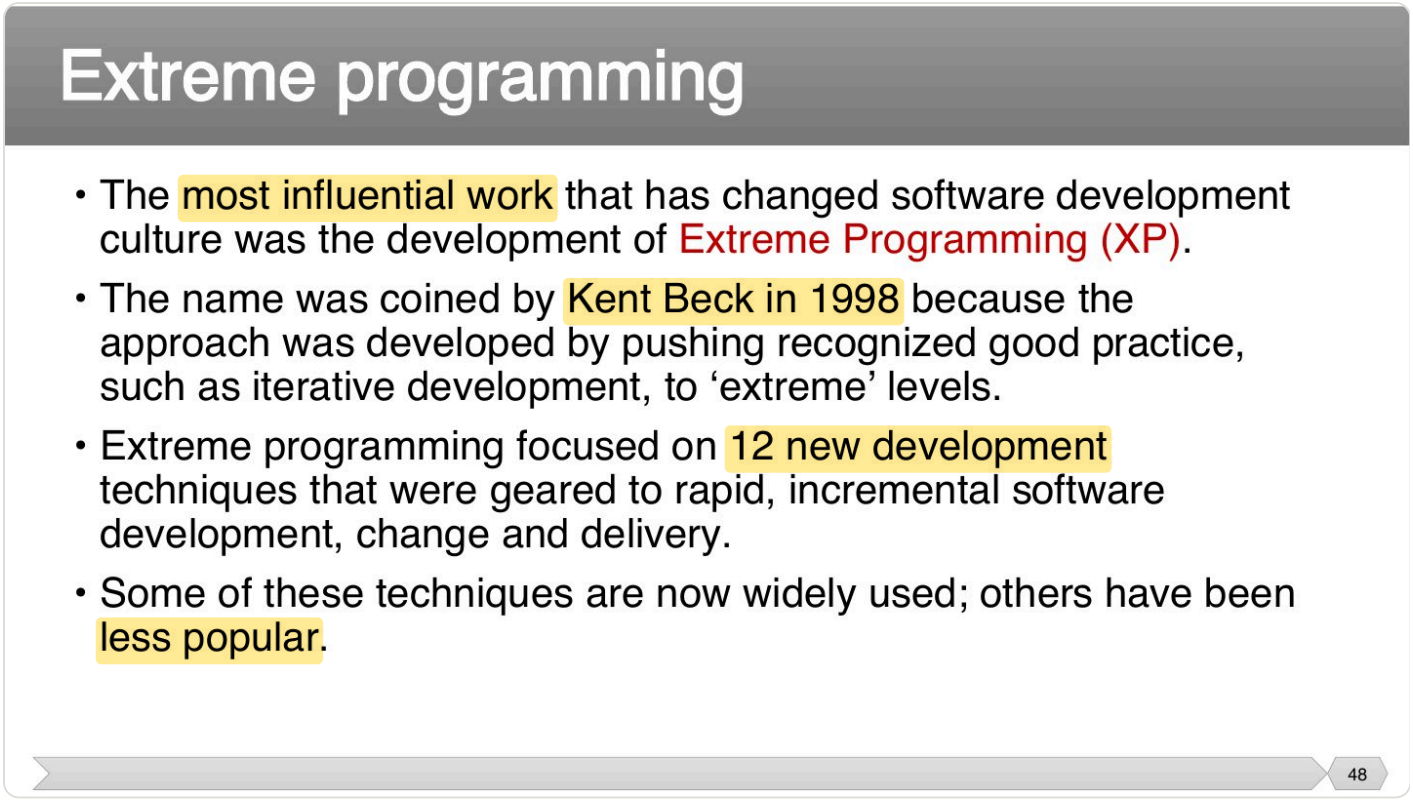
NOTES

1. **Choose features**: from the planned feature list, select what fits the next increment.
2. **Refine descriptions**: add enough detail for a common understanding and to start work.
3. **Implement and test**: build the feature with **automated tests** showing it behaves as described.
4. **Integrate and test**: add it to the existing system and check it works with the other features.
5. **Deliver**: give the increment to the customer or product manager for comments; release a version when enough features are in.

TIPS & TRICKS

- The order matters here: **C**ooks **R**oast **I**n **I**ron **D**ishes. Choose features, Refine them, Implement and test, Integrate and test, Deliver.

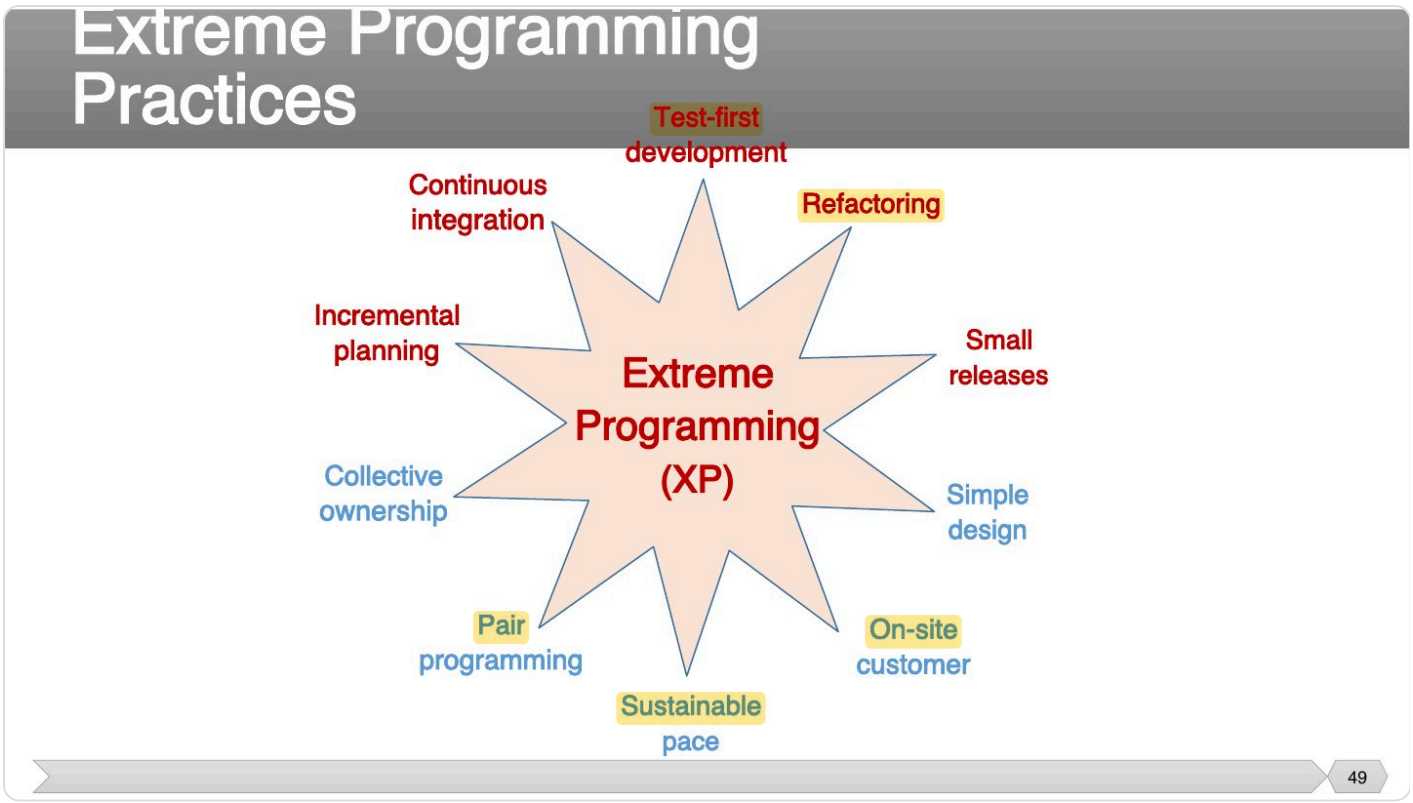
 Extreme programming



NOTES

1. **Extreme Programming (XP)** was the **most influential** change to development culture.
2. Named by **Kent Beck in 1998**, because it pushed recognized good practice (like iterative development) to "extreme" levels.
3. It introduced **12 development techniques** for rapid, incremental development and delivery...
4. ...some now everywhere, others **less popular** (e.g. pair programming all day).

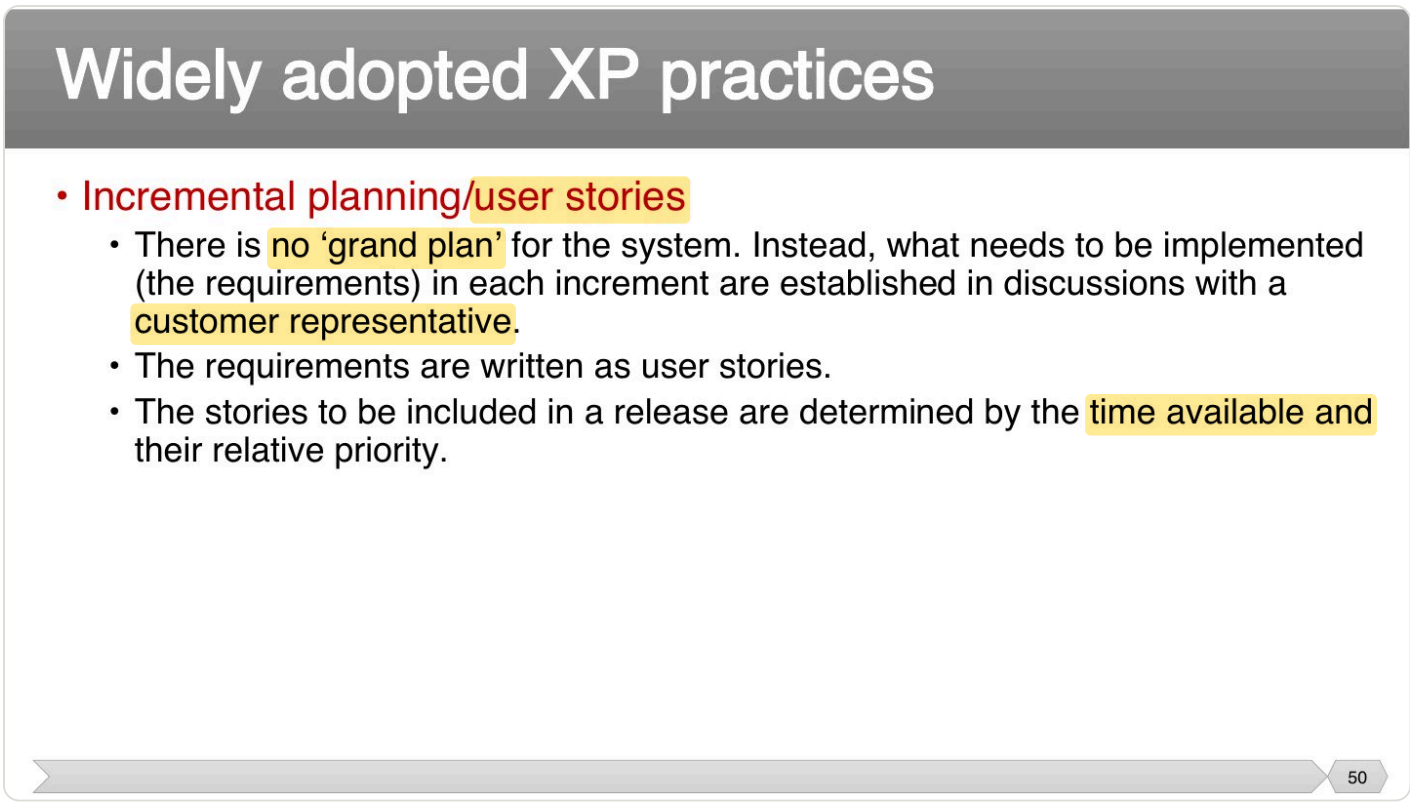
 Practices



NOTES

1. XP practices on the star: **test-first development**, continuous integration, **refactoring**, incremental planning, small releases, simple design.
2. People practices: collective ownership, **pair programming** and an **on-site customer**.
3. **Sustainable pace**: no long overtime. The next slides cover the widely adopted ones.

 Widely adopted XP practices



NOTES

1. **Incremental planning**: there is **no grand plan**; each increment's requirements are agreed in discussion with a **customer representative**.
2. Requirements are written as **user stories** ("As a ..., I want ... so that ...").
3. Which stories go into a release depends on the **time available** and their relative priority.

TIPS & TRICKS

- Slides 50 to 54, **STRICt**: Small releases, Test-driven development, Refactoring, Incremental planning, Continuous integration.

Widely adopted XP practices

- Small releases
  - The minimal useful set of functionality that provides business value is developed first.
  - Releases of the system are frequent and incrementally add functionality to the previous release.

NOTES

1. **Small releases**: the **minimal useful set** of functionality with business value is developed first (the MVP idea).
2. Releases are **frequent**, each **adding** functionality to the previous one.

Widely adopted XP practices

- Test-driven development
  - Instead of writing code then tests for that code, developers write the tests first.
  - This helps clarify what the code should actually do and that there is always a ‘tested’ version of the code available.
  - An automated unit test framework is used to run the tests after every change.
  - New code should not ‘break’ code that has already been implemented.

NOTES

1. **Test-driven development**: developers **write the tests first**, then the code.
2. This **clarifies what the code should do** and means a tested version always exists.
3. An **automated unit test framework** runs all tests after every change...
4. ...so new code doesn't **break** what already works.

EXAM ANGLE

Prediction Q1

In test-driven development, developers:

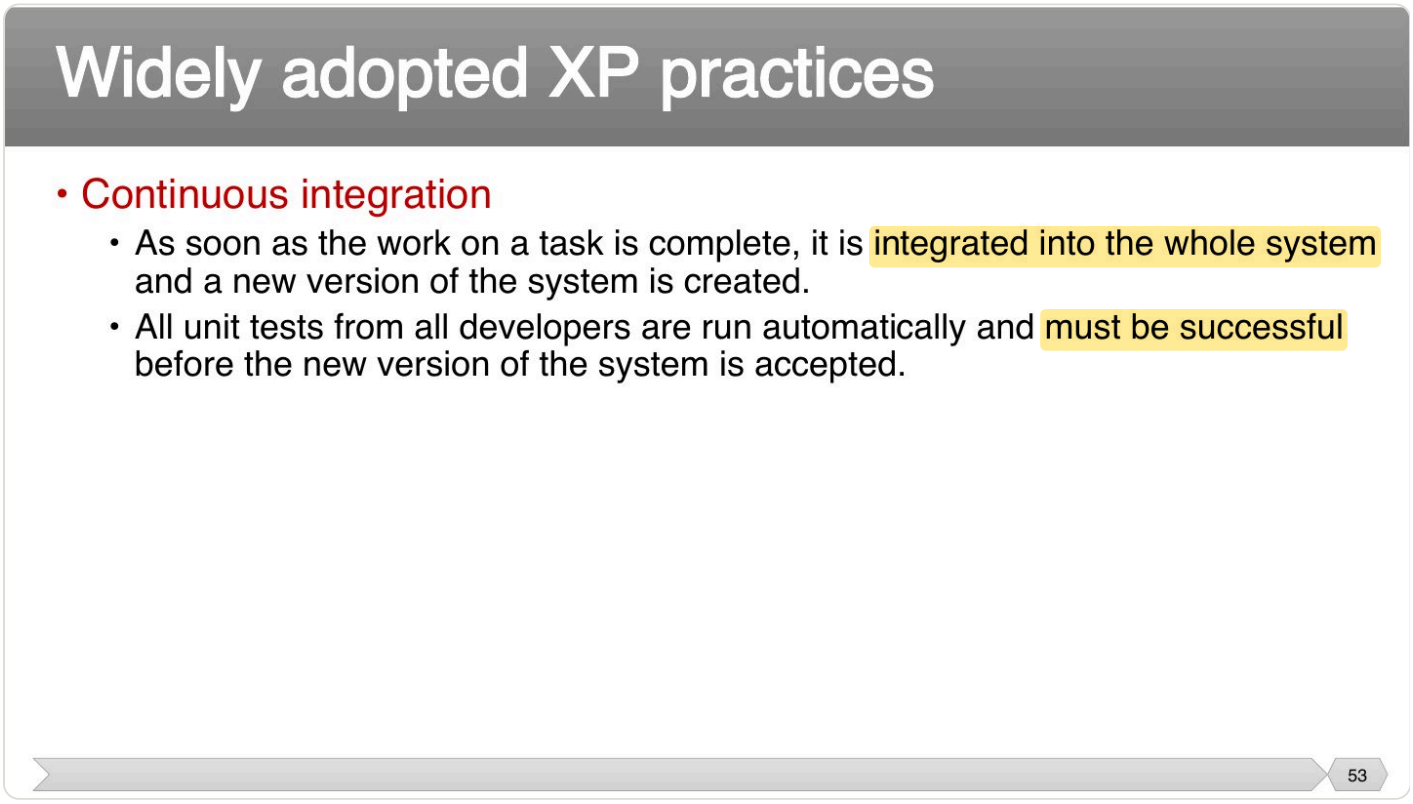
A Only test manually

B Write tests after release

C **Write tests before writing the code**

D Skip unit tests

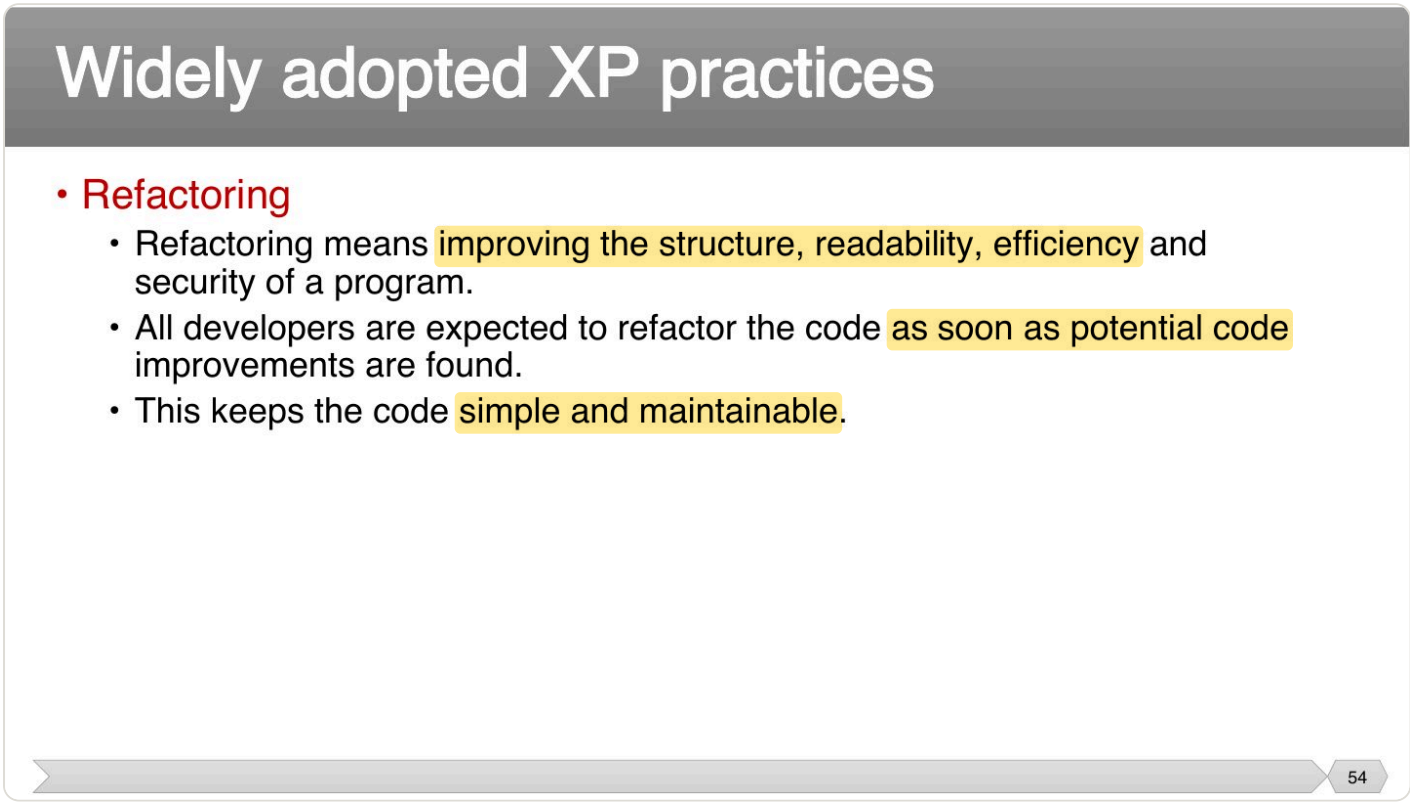
SLIDE 53 / 98 **Widely adopted XP practices**



NOTES

1. **Continuous integration**: as soon as a task is finished it is **integrated into the whole system** and a new version is built.
2. All unit tests from all developers run automatically and **must pass** before the new version is accepted, so integration problems appear in minutes instead of at the end.

SLIDE 54 / 98 **Widely adopted XP practices**



NOTES

1. **Refactoring** means **improving the structure**, readability, efficiency and security of code without changing what it does.
2. Everyone refactors **as soon as** an improvement is spotted...
3. ...which keeps the code **simple and maintainable**.

SLIDE 55 / 98 **Widely adopted XP practices**

Widely adopted XP practices

- Incremental planning/user stories
  - There is no ‘grand plan’ for the system. Instead, what needs to be implemented (the requirements) in each increment are established in discussions with a customer representative.
  - The requirements are written as user stories.
  - The stories to be included in a release are determined by the time available and their relative priority.

NOTES

1 A repeat of slide 50: **incremental planning** with **user stories** instead of a grand plan.

SLIDE 56 / 98 **Scrum**

Scrum

- Software company managers need information that will help them understand how much it costs to develop a software product, how long it will take and when the product can be brought to market.
- Plan-driven development provides this information through long-term development plans that identify deliverables - items the team will deliver and when these will be delivered.
- Plans always change so anything apart from short-term plans are unreliable.
- Scrum is an agile method that provides a framework for agile project organization and planning. It does not mandate any specific technical practices.

NOTES

1 Managers need to know **how much** a product will cost, how long it will take and when it can ship.

2. Plan-driven development answers with **long-term plans** of deliverables and dates.
3. But **plans always change**, so anything beyond short-term plans is unreliable.
4. **Scrum** is an agile framework for **organization and planning**. It **mandates no technical practices**, unlike XP. A common trap.

EXAM ANGLE

Prediction Q1

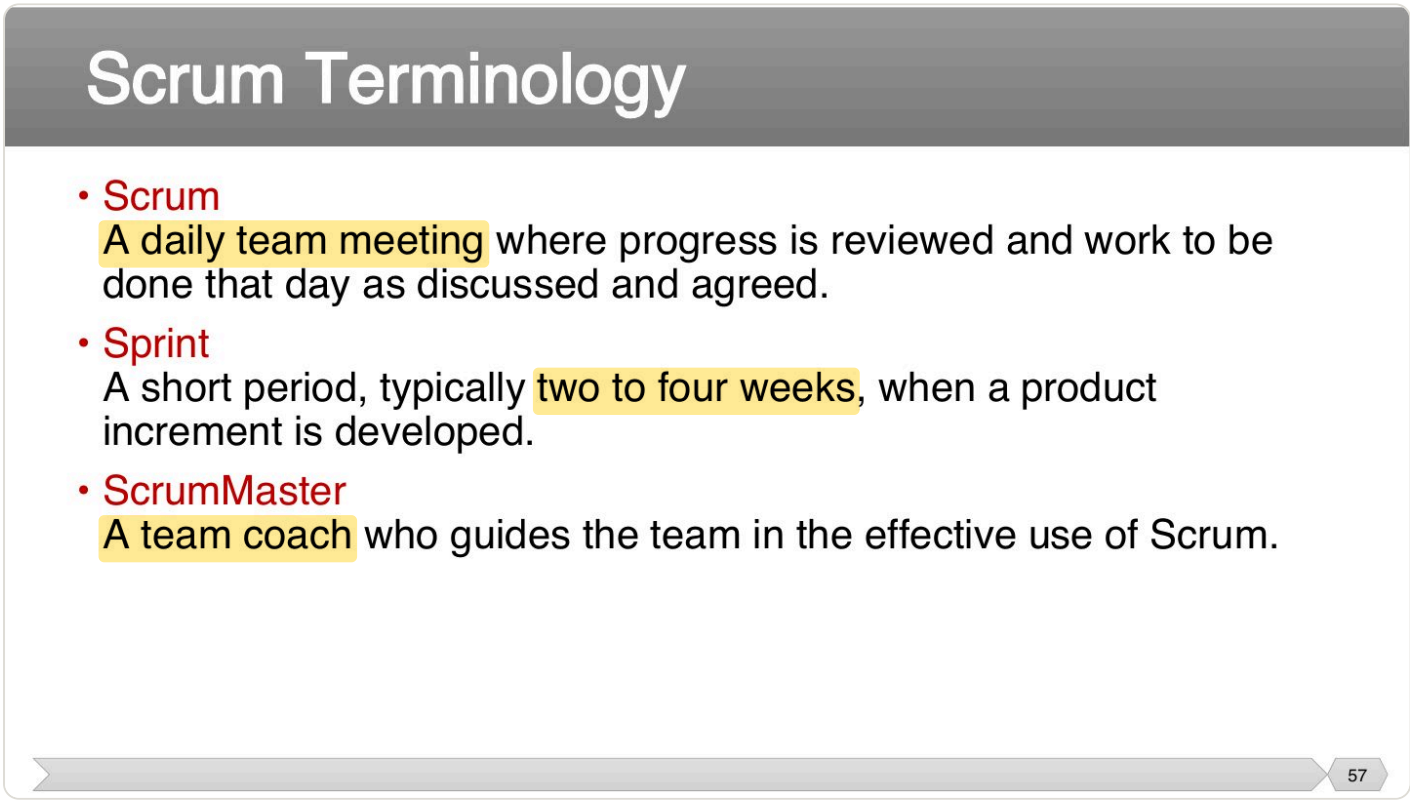
Scrum mandates specific technical practices such as pair programming.

T True

F **False**

> Scrum provides a framework for organization and planning only; XP defines the engineering practices.

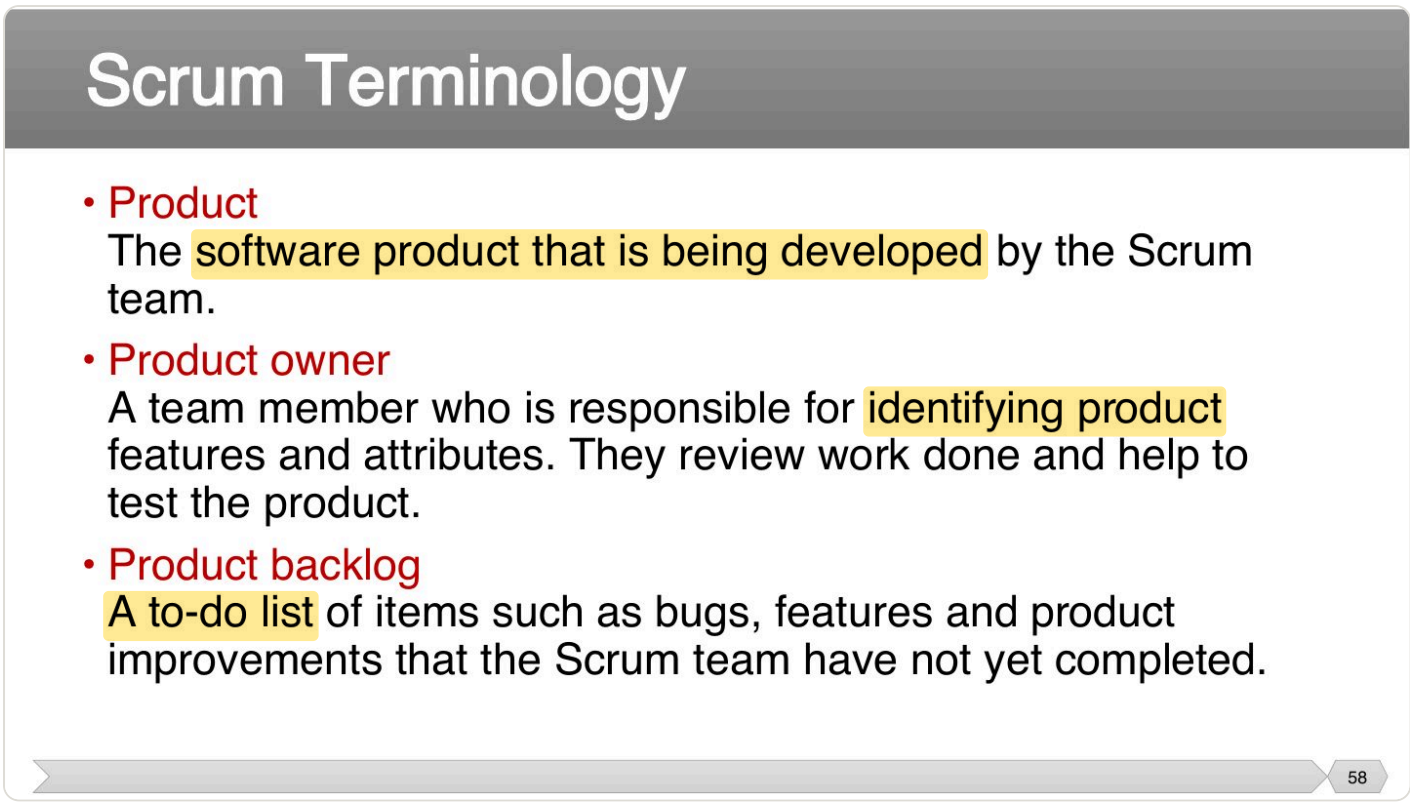
SLIDE 57 / 98 **Scrum Terminology**



NOTES

1. **Scrum** (the meeting): a **daily team meeting** where progress is reviewed and the day's work agreed.
2. **Sprint**: a short period, typically **two to four weeks**, in which a product increment is developed.
3. **ScrumMaster**: a **team coach** who guides the team in using Scrum effectively.

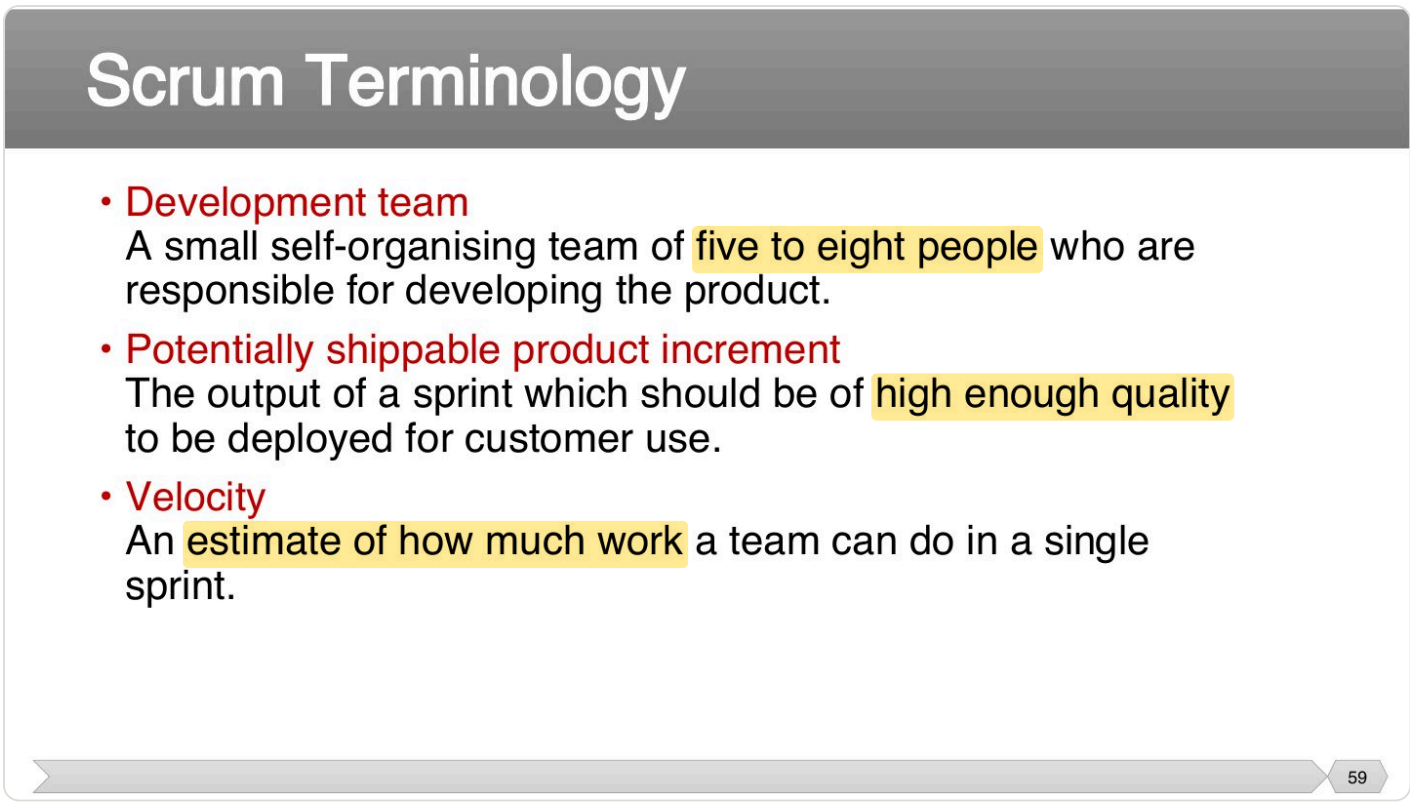
 Scrum Terminology



NOTES

1. **Product**: the software being developed by the Scrum team.
2. **Product owner**: responsible for **identifying product features and attributes**; reviews work and helps test.
3. **Product backlog**: a **to-do list** of bugs, features and improvements not yet completed.

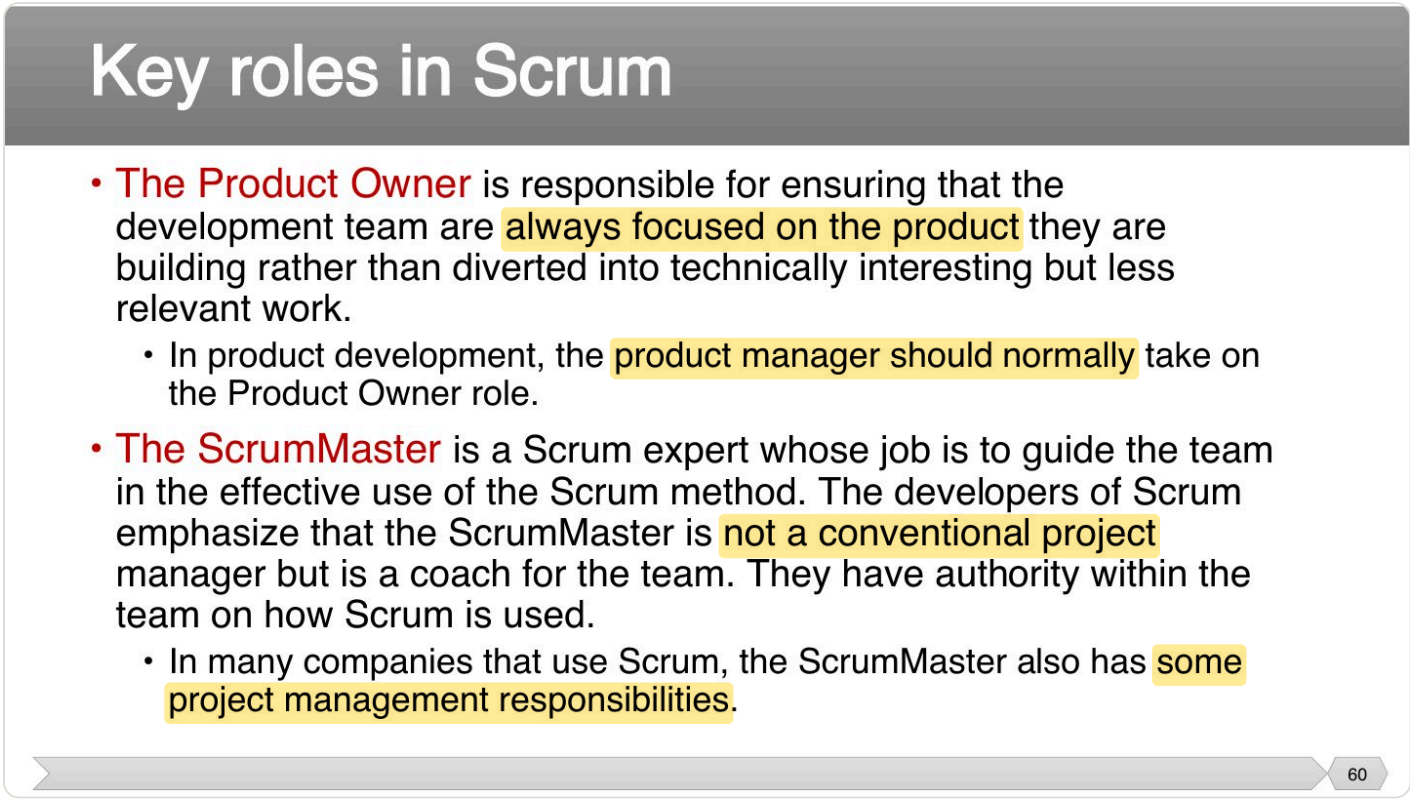
 Scrum Terminology



NOTES

1. **Development team**: small and self-organizing, **five to eight people**.
2. **Potentially shippable product increment**: a sprint's output, of **high enough quality** to deploy.
3. **Velocity**: an **estimate of how much work** the team can do in one sprint, used to plan the next sprints.

 Key roles in Scrum



NOTES

1. The **Product Owner** keeps the team **focused on the product**, not diverted into technically interesting but less relevant work.
2. In product development the **product manager** normally takes the Product Owner role.
3. The **ScrumMaster** is a Scrum expert and a **coach**, **not a conventional project manager**; they have authority on how Scrum is used.
4. In many companies the ScrumMaster also carries **some project management** duties.

TIPS & TRICKS

- **PoST**: Product owner, ScrumMaster, Team.

EXAM ANGLE

Prediction Q1

Who keeps the team focused on the product instead of technically interesting but less relevant work?

A Sponsor

B Tester

C ScrumMaster

D **Product Owner**

Prediction Q2

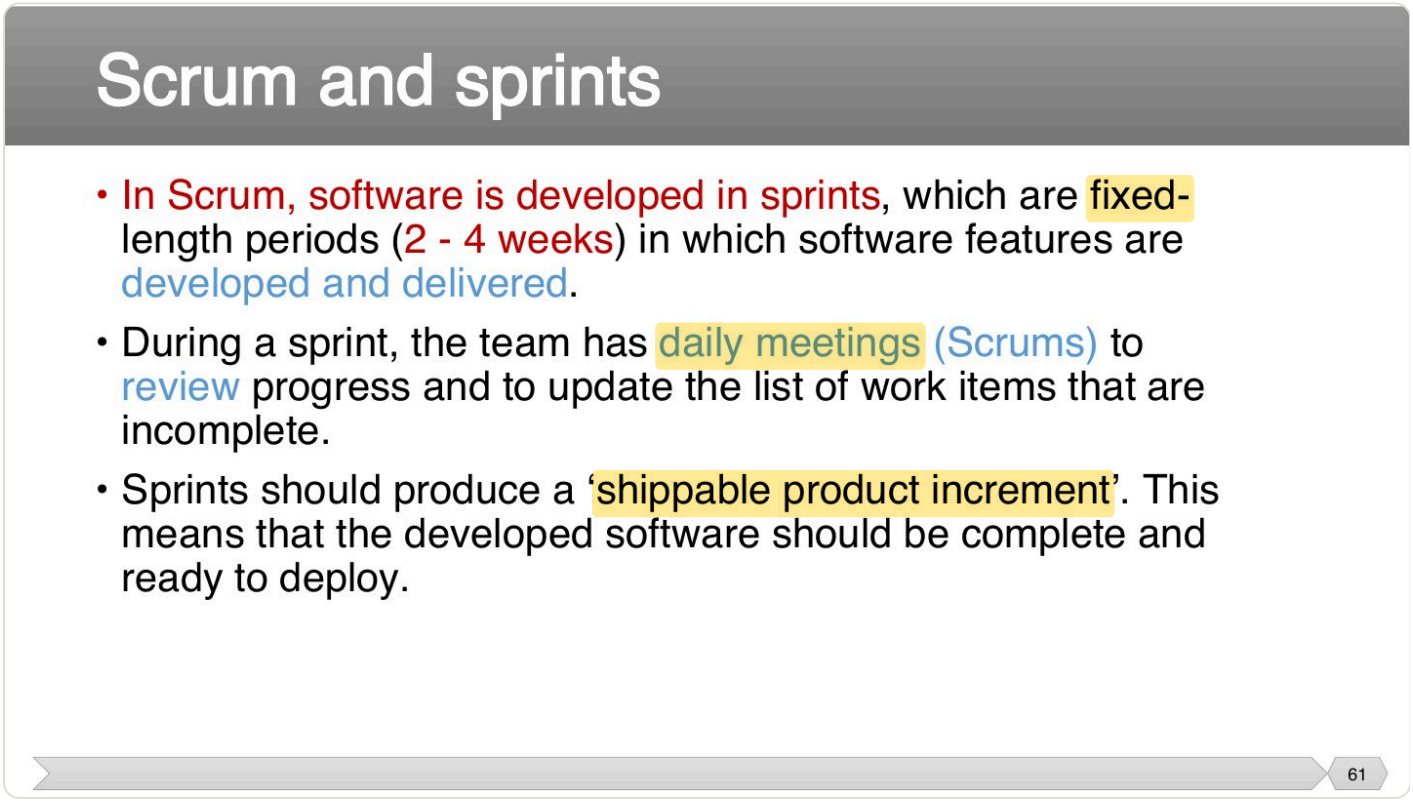
The ScrumMaster is a conventional project manager.

T True

F **False**

> The ScrumMaster is a coach for the team.

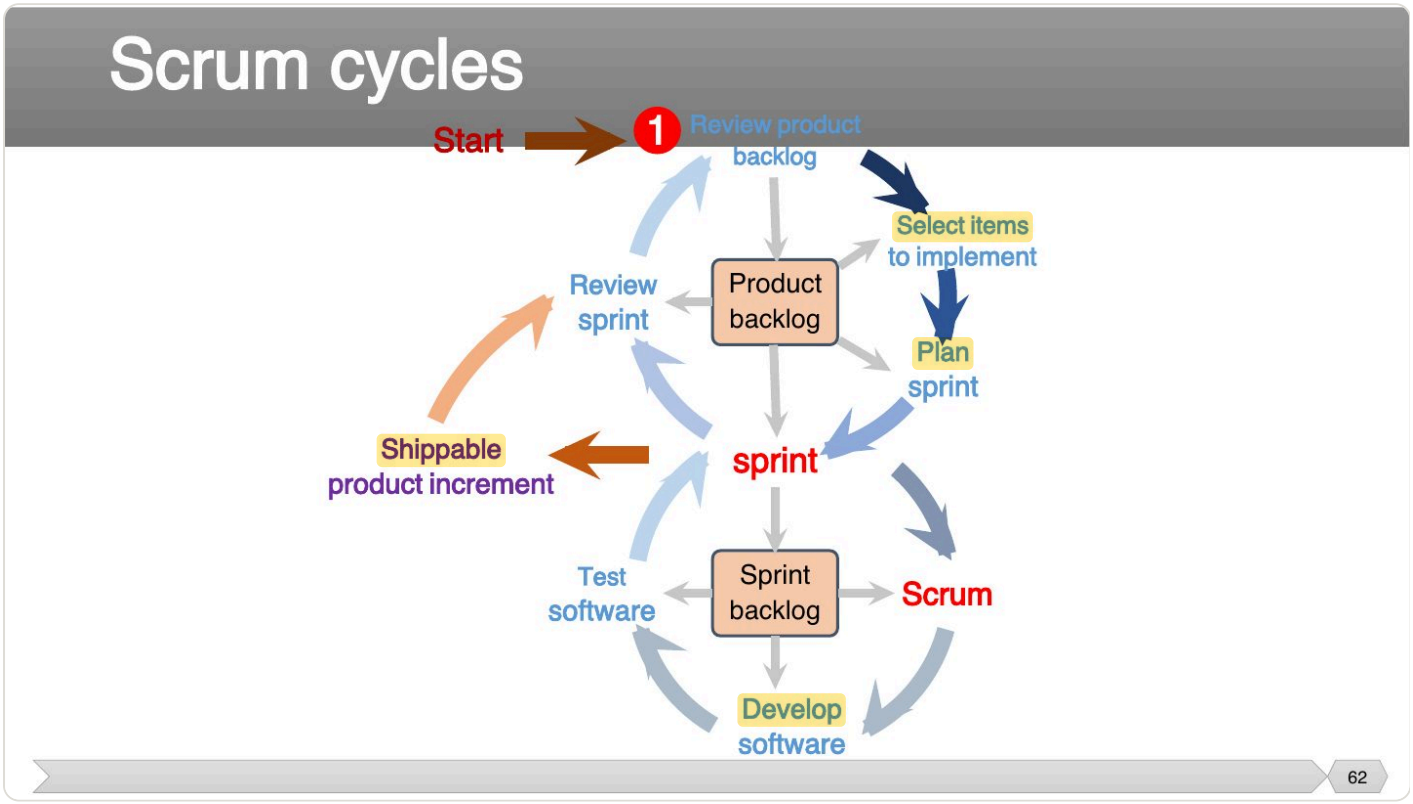
SLIDE 61 / 98 **Scrum and sprints**



NOTES

1. Work happens in **sprints**: **fixed-length** periods (2–4 weeks).
2. **Daily meetings** (Scrums) review progress and update the list of incomplete work.
3. Each sprint should produce a **shippable product increment**: complete and ready to deploy.

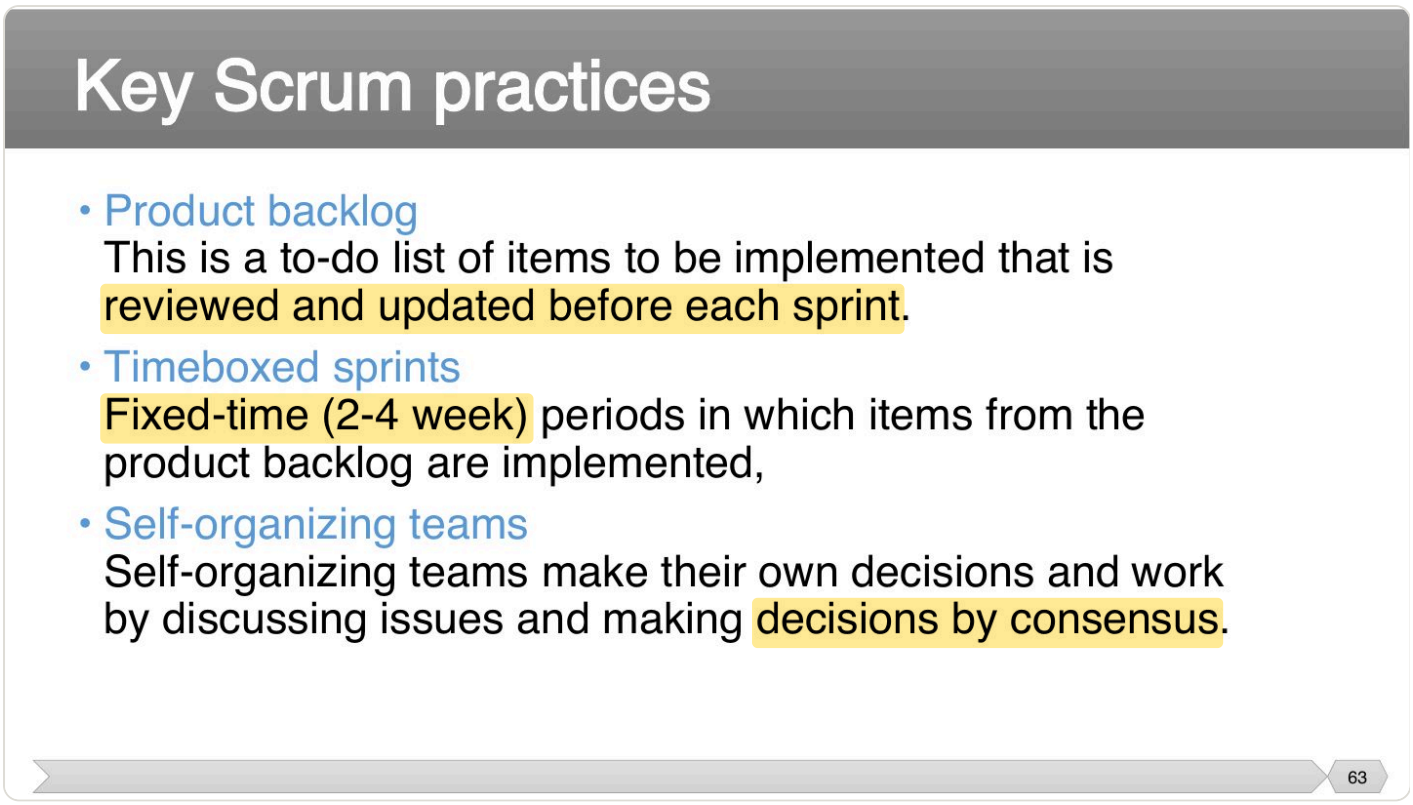
SLIDE 62 / 98 **Scrum cycles**



NOTES

1. The Scrum cycle: review the product backlog, **select items** for the sprint and **plan the sprint** (creating the sprint backlog).
2. During the sprint the team **develops and tests** software, with a daily Scrum.
3. The sprint ends with a **shippable increment** and a sprint review, then the cycle restarts.

 Key Scrum practices



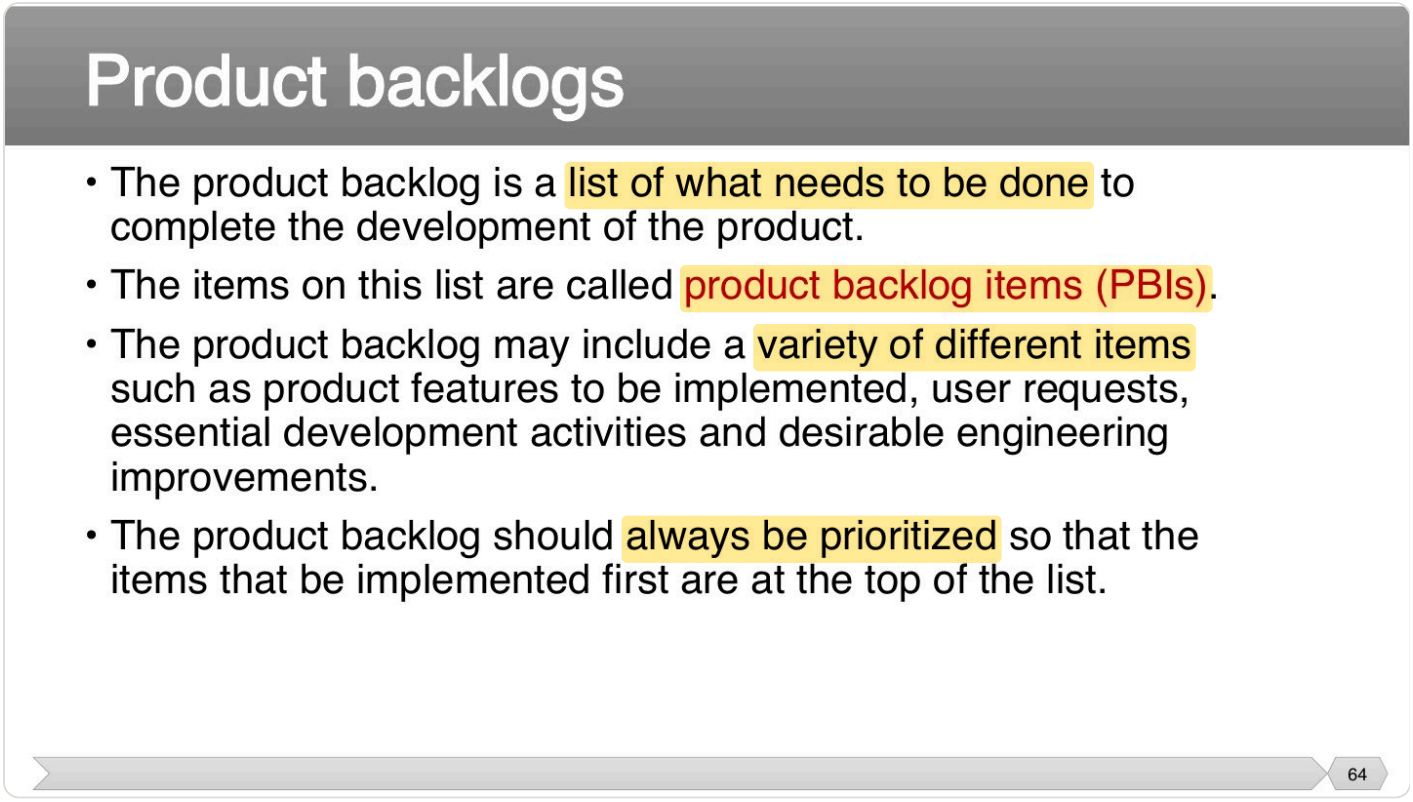
NOTES

1. **Product backlog**: the to-do list, **reviewed and updated before each sprint**.
2. **Timeboxed sprints**: **fixed-time** (2–4 week) periods implementing backlog items.
3. **Self-organizing teams** make their own decisions by discussion and **consensus**.

TIPS & TRICKS

- **STeP**: Self-organizing teams, Timeboxed sprints, Product backlog.

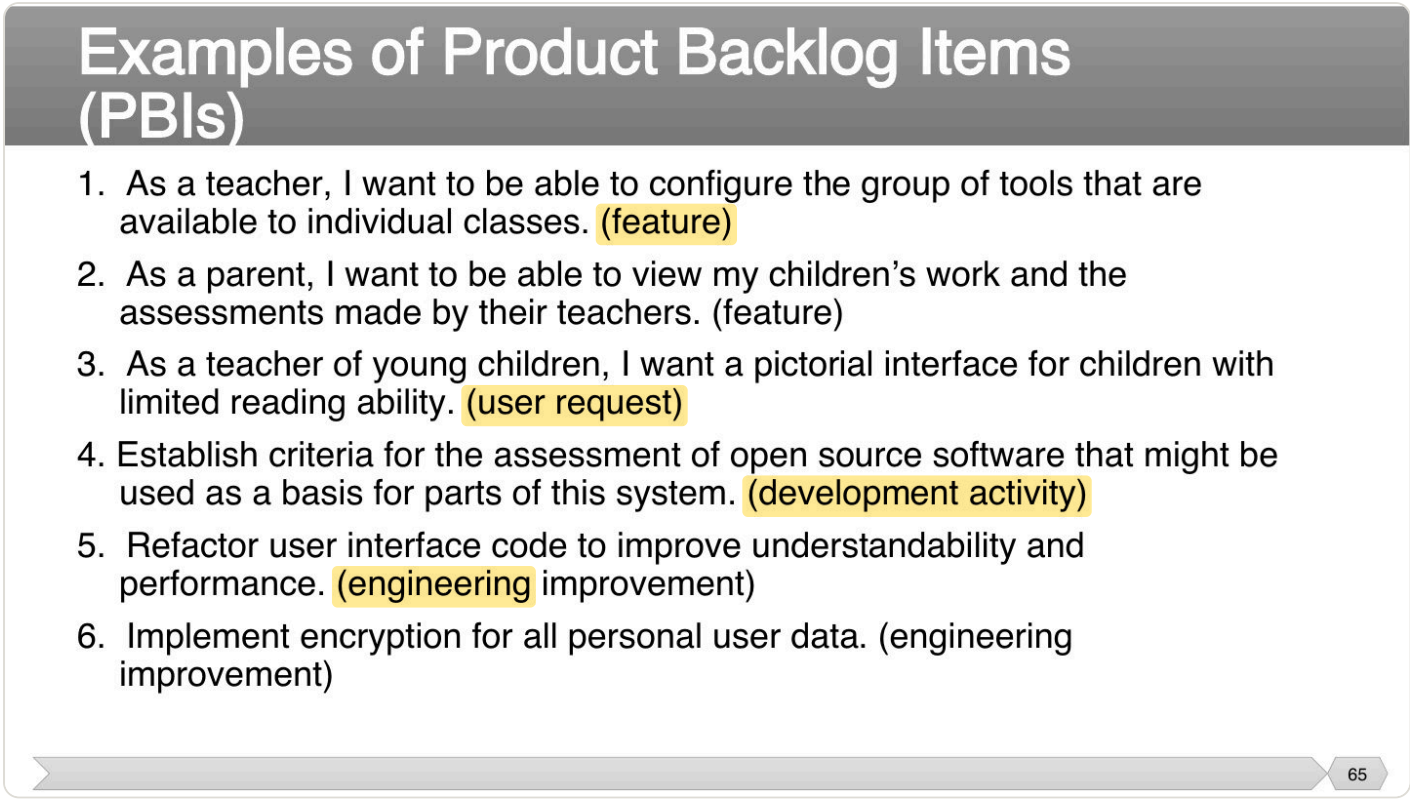
 Product backlogs



NOTES

1. The **product backlog** is the **list of what must be done** to complete the product.
2. Its entries are **product backlog items (PBIs)**.
3. They can be **varied**: features, user requests, development activities, engineering improvements.
4. It is **always prioritized**, with the next items to implement at the top.

 Examples of Product Backlog Items



NOTES

1. PBI types by example. **Features**: "As a teacher, I want to configure the tools available to each class."
2. **User request**: a pictorial interface for children with limited reading ability.
3. **Development activity**: establish criteria for assessing open-source software.
4. **Engineering improvement**: refactor UI code, or encrypt all personal data.

EXAM ANGLE

Prediction Q1

"Refactor user interface code to improve performance" is which kind of PBI?

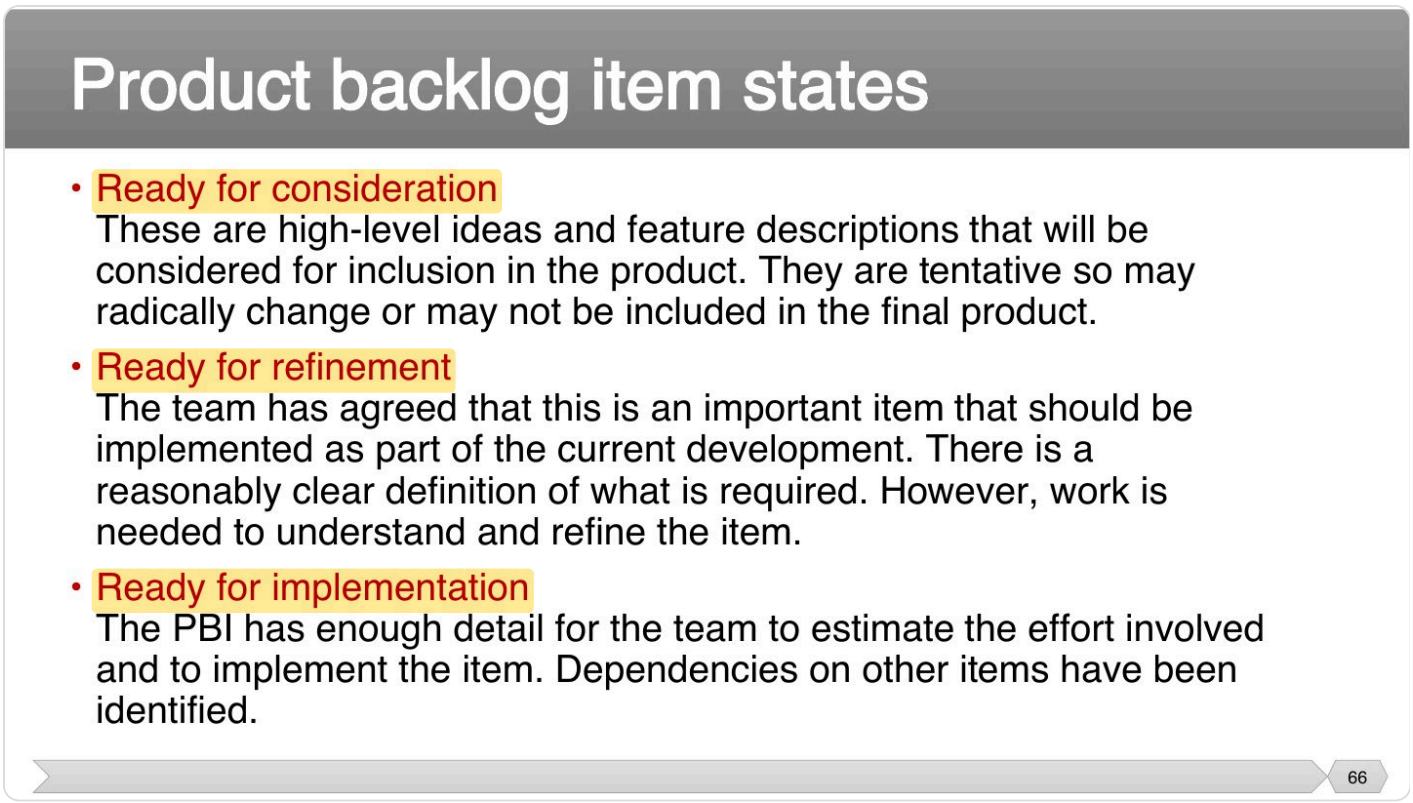
A Development activity

B **Engineering improvement**

C User request

D Feature

 Product backlog item states



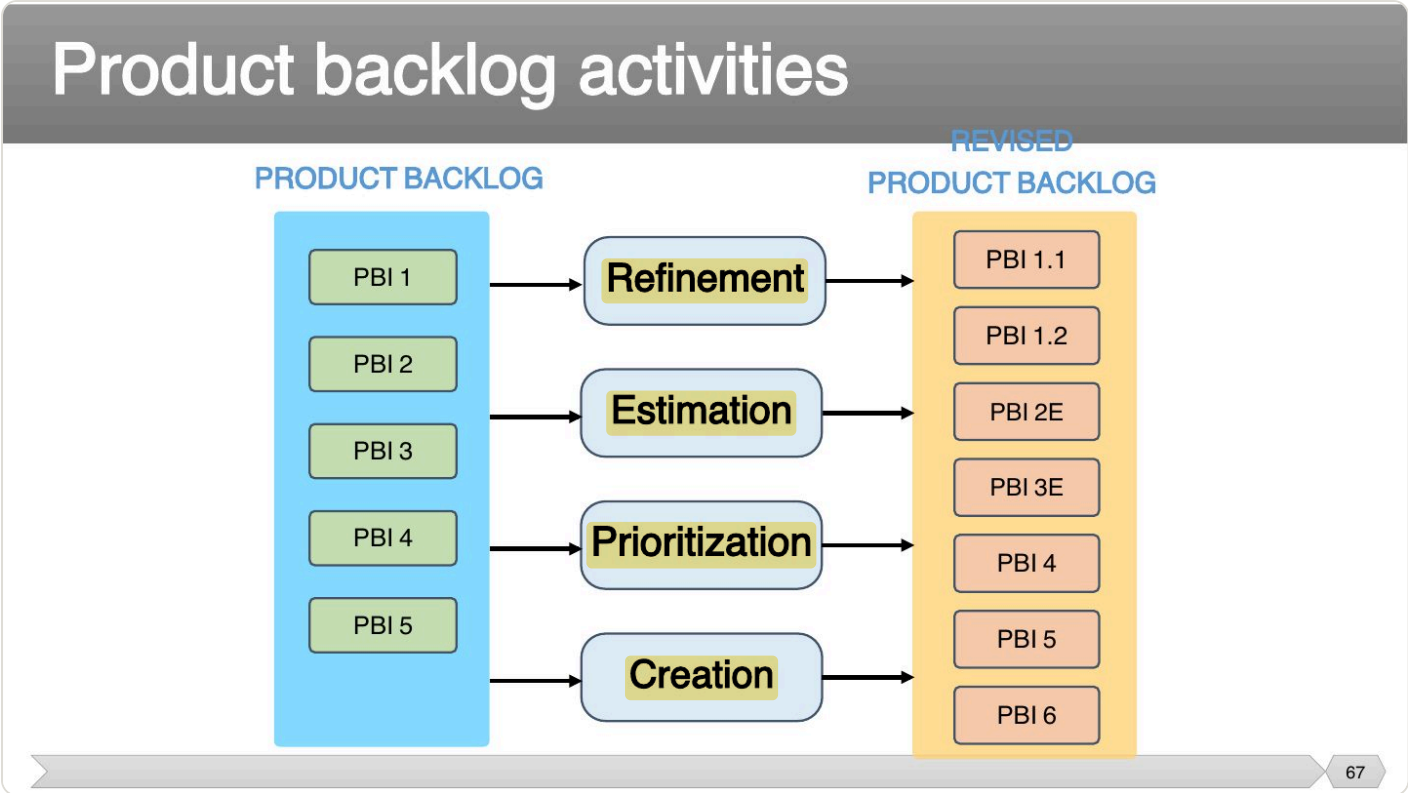
NOTES

1. **Ready for consideration**: tentative, high-level ideas that may change radically or be dropped.
2. **Ready for refinement**: agreed as important, with a reasonably clear definition, but needing work to understand and refine.
3. **Ready for implementation**: detailed enough to **estimate** and build, with dependencies identified.

TIPS & TRICKS

- **CRIb**, already in order: Consideration, Refinement, Implementation.

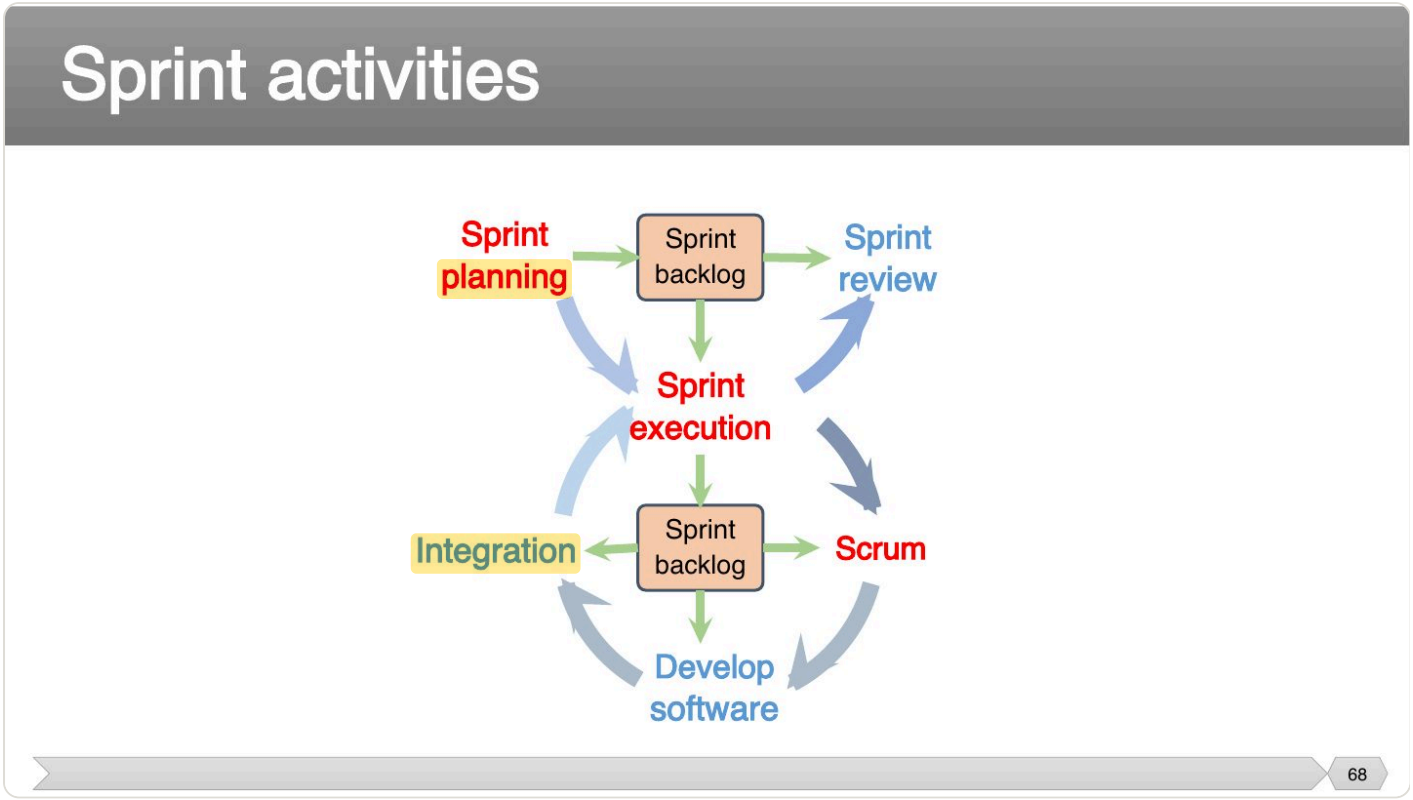
 Product backlog activities



NOTES

1. **Refinement**: break PBIs into smaller, clearer items (PBI 1 becomes 1.1 and 1.2).
2. **Estimation**: estimate the effort for items (PBI 2E, 3E).
3. **Prioritization**: reorder the backlog by value.
4. **Creation**: add new items as they are discovered (PBI 6). The result is a revised backlog.

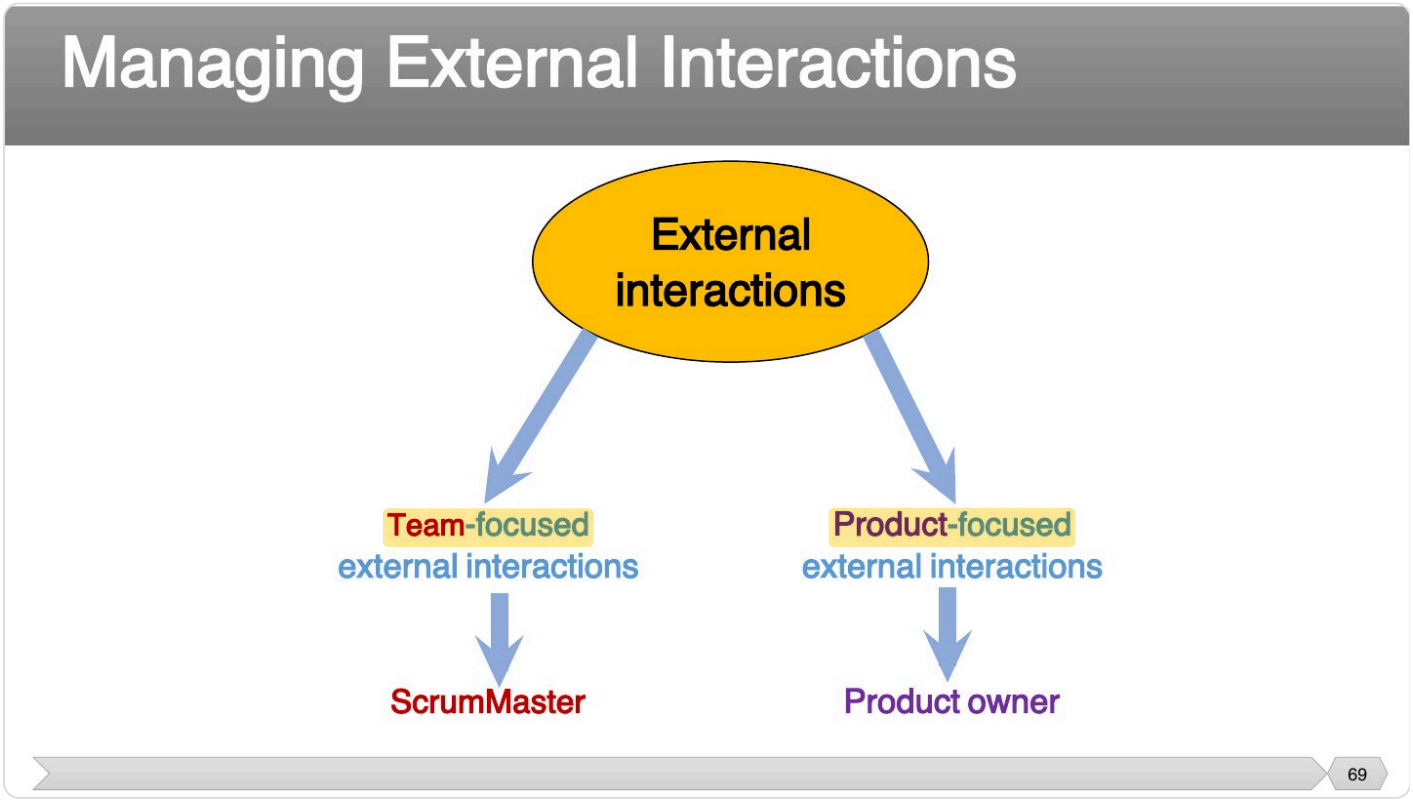
SLIDE 68 / 98 **Sprint activities**



NOTES

1. Sprint activities: **sprint planning** produces the sprint backlog; **sprint execution** develops software with daily Scrums.
2. Work is continuously **integrated**, and the **sprint review** closes the sprint.

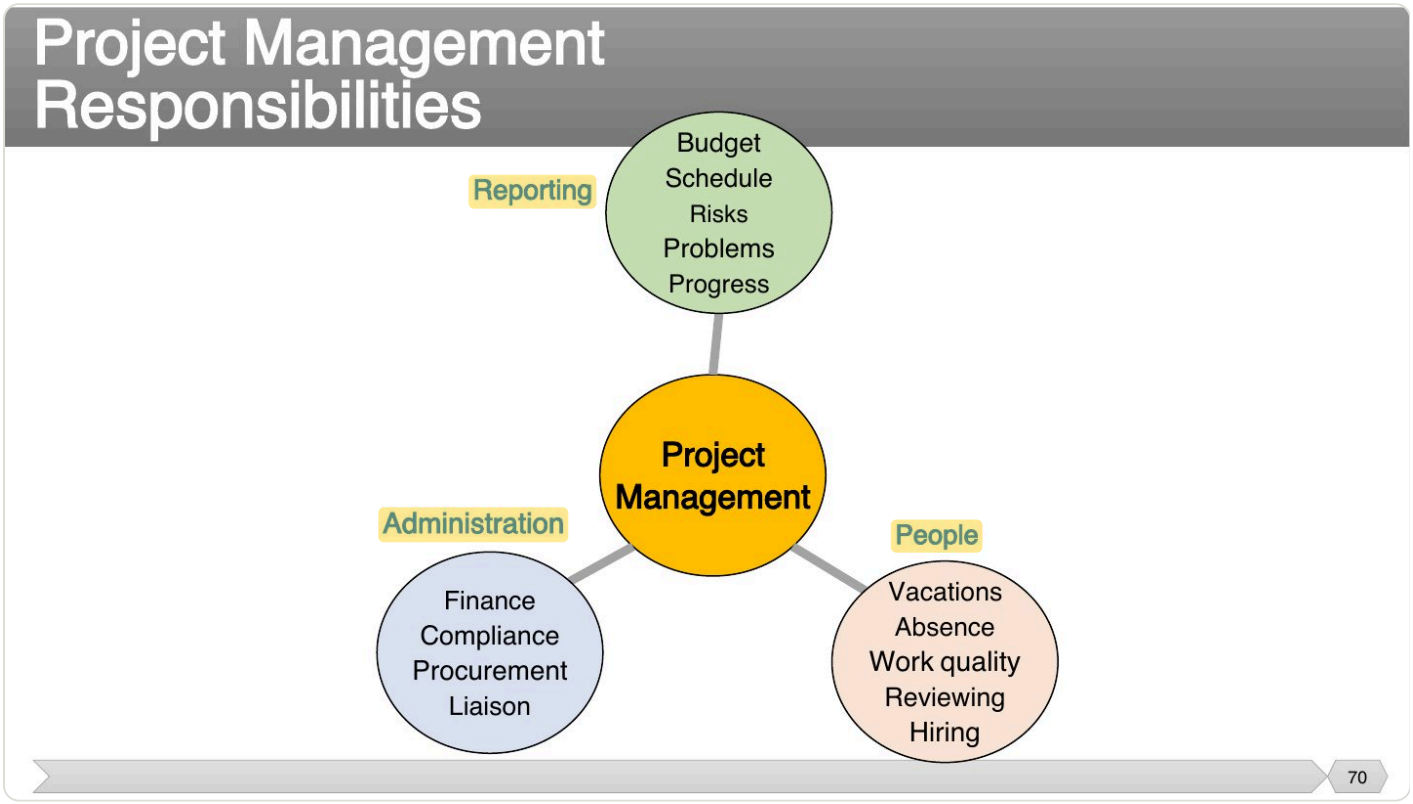
SLIDE 69 / 98 **Managing External Interactions**



NOTES

1. **Team-focused** external interactions (with management, other teams, HR) go through the **ScrumMaster**.
2. **Product-focused** ones (customers, marketing, sales) go through the **Product Owner**. Either way, the developers are shielded from distractions.

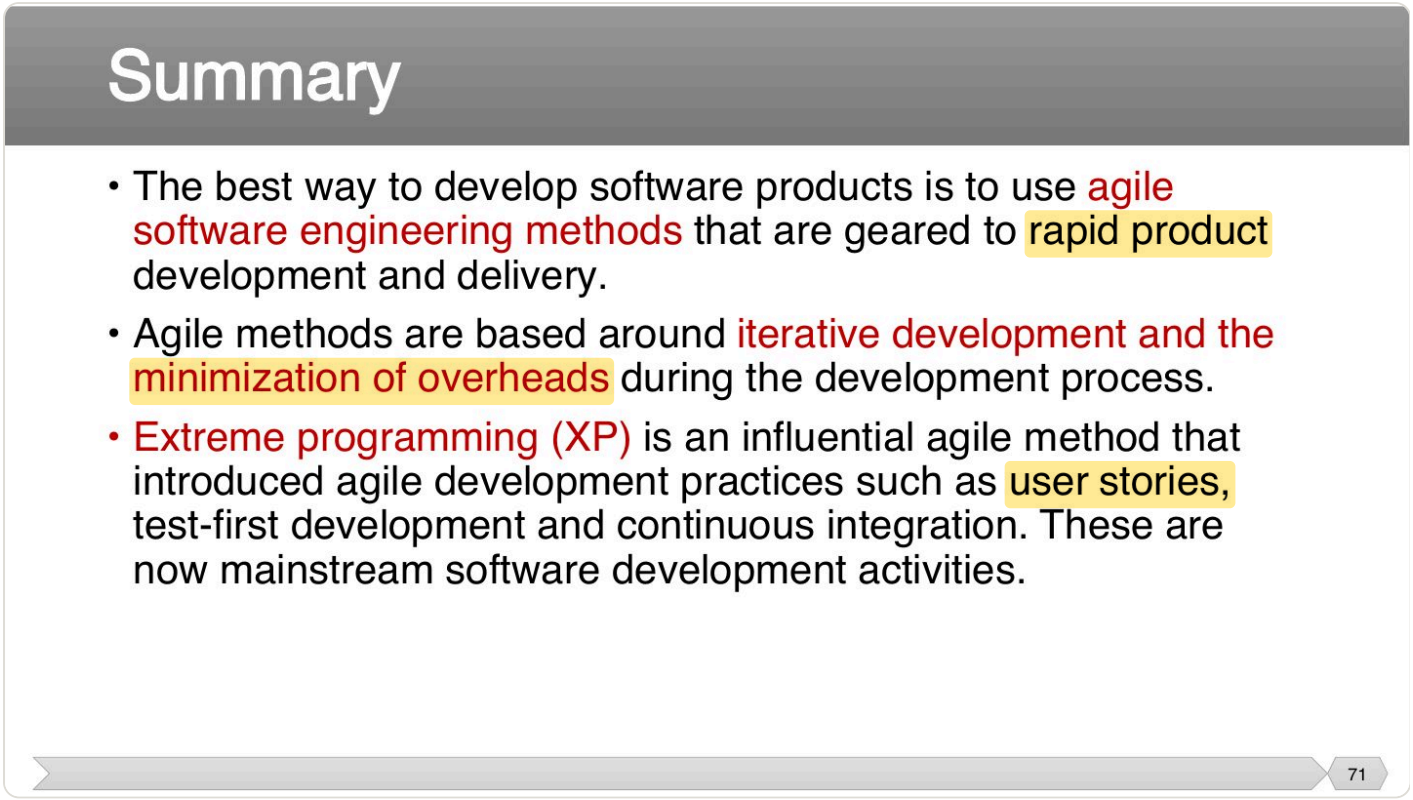
SLIDE 70 / 98 **Responsibilities**



NOTES

1. Even in Scrum, someone must do PM work. **Reporting**: budget, schedule, risks, problems, progress.
2. **Administration**: finance, compliance, procurement, liaison.
3. **People**: vacations, absence, work quality, reviewing, hiring. Often the ScrumMaster or a PM takes these on.

 Summary



NOTES

1. Agile methods suit **rapid product** development and delivery...
2. ...through iterative development and **minimal overhead**.
3. **XP** introduced practices now mainstream: **user stories**, test-first development and continuous integration.

Summary

- Scrum is an agile method that focuses on agile planning and management. Unlike XP, it does not define the engineering practices to be used. The development team may use any technical practices that they believe are appropriate for the product being developed.
- In Scrum, work to be done is maintained in a product backlog – a list of work items to be completed. Each increment of the software implements some of the work items from the product backlog.

NOTES

1. **Scrum** focuses on **agile planning and management** and, unlike XP, **does not define engineering practices**; the team picks its own.
2. Work is kept in a **product backlog**, and each increment implements some of its items.

Summary

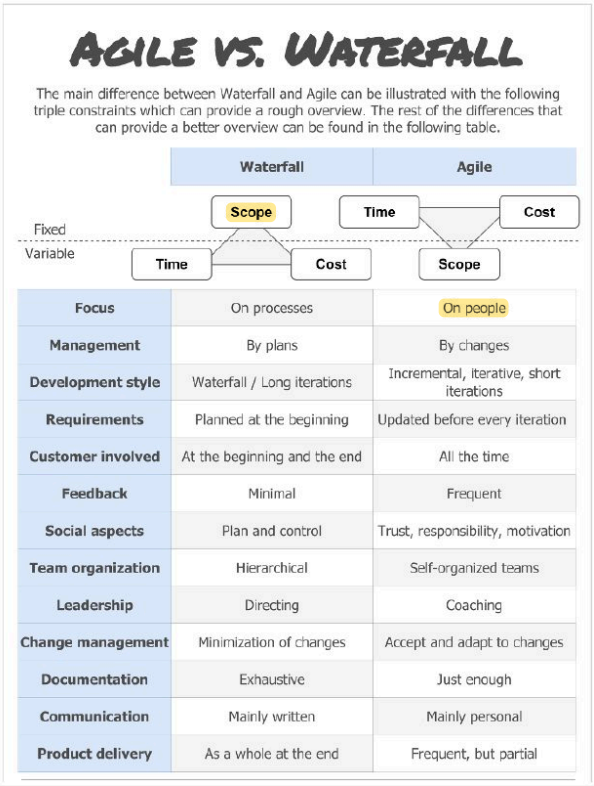
- Sprints are fixed-time activities (usually 2–4 weeks) where a product increment is developed. Increments should be 'potentially shippable' i.e. they should not need further work before they are delivered.
- A self-organizing team is a development team that organizes the work to be done by discussion and agreement amongst team members.
- Scrum practices such as the product backlog, sprints and self-organizing teams can be used in any agile development process, even if other aspects of Scrum are not used.

NOTES

1. Sprints are fixed-time (2–4 weeks), and their increments must be **potentially shippable** without further work.
2. A **self-organizing team** organizes its work by **discussion and agreement**.
3. Scrum practices (backlog, sprints, self-organizing teams) **can be used in any agile process**, even without the rest of Scrum.

SLIDE 74 / 98

Agile vs. Waterfall



AGILE VS. WATERFALL

The main difference between Waterfall and Agile can be illustrated with the following triple constraints which can provide a rough overview. The rest of the differences that can provide a better overview can be found in the following table.

	Waterfall	Agile
Focus	On processes	On people
Management	By plans	By changes
Development style	Waterfall / Long iterations	Incremental, iterative, short iterations
Requirements	Planned at the beginning	Updated before every iteration
Customer involved	At the beginning and the end	All the time
Feedback	Minimal	Frequent
Social aspects	Plan and control	Trust, responsibility, motivation
Team organization	Hierarchical	Self-organized teams
Leadership	Directing	Coaching
Change management	Minimization of changes	Accept and adapt to changes
Documentation	Exhaustive	Just enough
Communication	Mainly written	Mainly personal
Product delivery	As a whole at the end	Frequent, but partial

74

NOTES

1. The key line in the triangle: **waterfall fixes scope** and lets time and cost vary; **agile fixes time and cost** (the timebox and team) and lets scope vary.
2. The table compares the rest: waterfall focuses **on processes**, manages by plans, plans requirements up front, involves the customer at the start and end, and delivers everything at the end; agile focuses **on people**, manages by change, updates requirements every iteration, involves the customer all the time, uses self-organized teams and coaching leadership, and delivers frequently in parts.

EXAM ANGLE

Prediction Q1

In agile, which of the triple constraints is typically allowed to vary?

A Scope

B None of them

C Cost

D Time

Past exam · Midterm 251 · V1 · Case Q2 (also V2) Q2

Given the evolving requirements (student usability, advisor requests, IT integration rules), which development approach (Agile, Waterfall, or Hybrid) would you recommend, and why would it be the best fit for this project?

Answer: Agile (Scrum) (0.5), because it handles the evolving requirements from students, advisors and IT, delivers iteratively so each sprint gets feedback, and integrates with DevOps pipelines for frequent tested releases (0.5). Justify it against the slide criteria: the requirements are reasonably clear but not stable, so waterfall's "clear, unambiguous, stable requirements" condition fails.

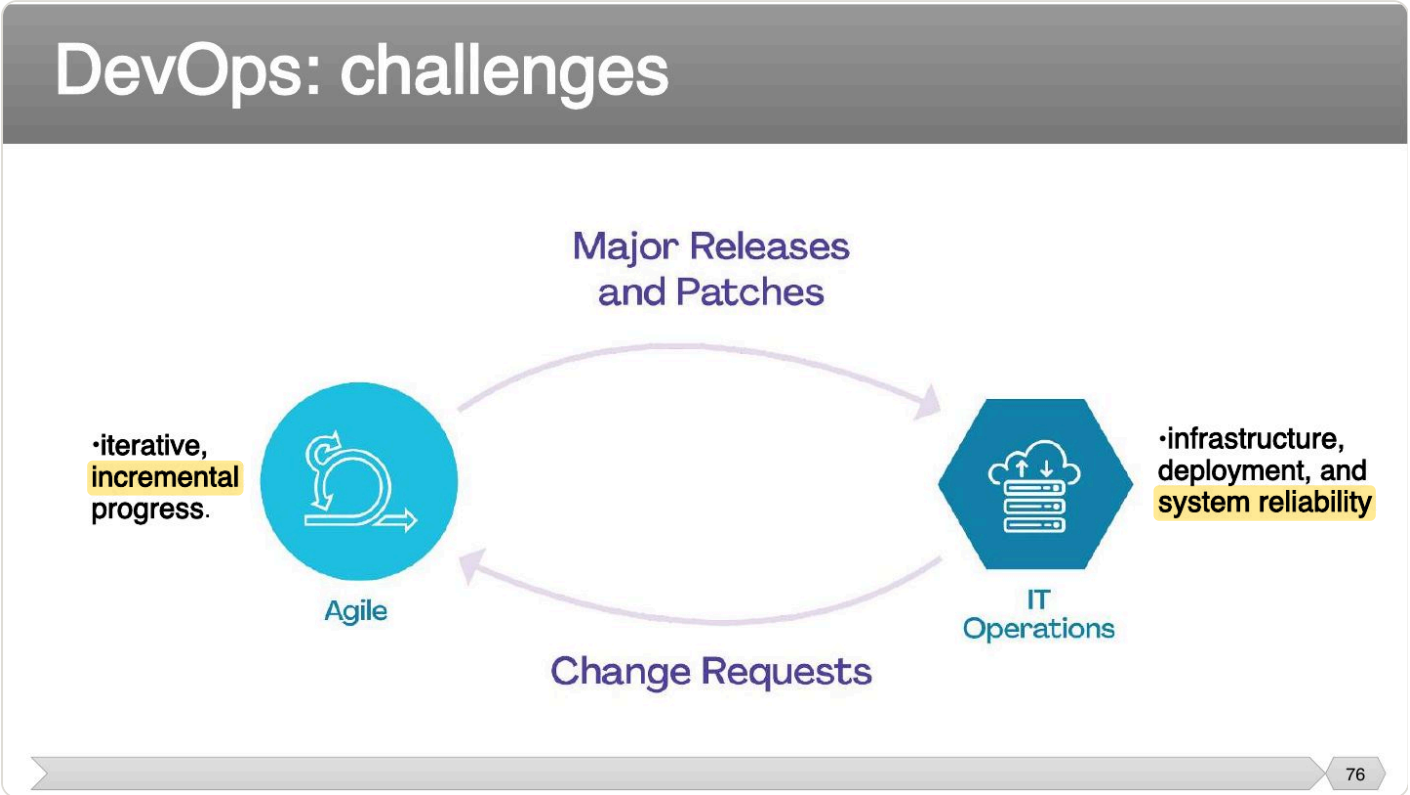
SLIDE 75 / 98 **DevOps**



NOTES

1. Section divider: **DevOps** = **dev**elopment and **op**eration**s**.

SLIDE 76 / 98 DevOps: challenges



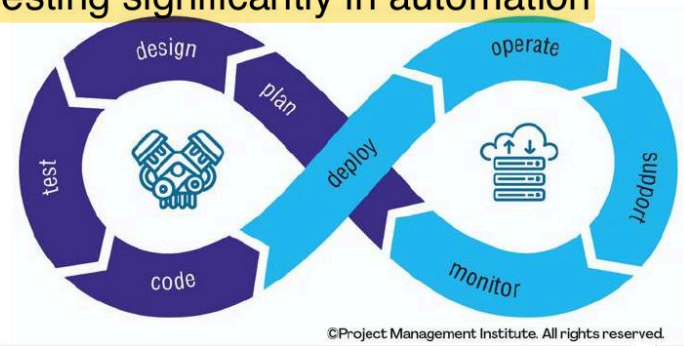
NOTES

1. The **DevOps gap**. Agile development delivers **iterative, incremental** progress and keeps sending change requests...
2. ...while IT operations care about infrastructure, deployment and **system reliability**, and resist frequent major releases and patches. Fast change and stability pull against each other.

DevOps

h

- When organizations address these challenges by removing the barriers that inhibit effective collaboration (operations <-> dev), they close the DevOps gap.
- Organizations do this by: adopting a mindset that promotes collaborative; learning-centric ways of working that are supported by agile practices; and very often investing significantly in automation
- On the development side they tend to adopt a continuous delivery (CD) approach and their operations efforts become streamlined and more automated.



©Project Management Institute. All rights reserved.

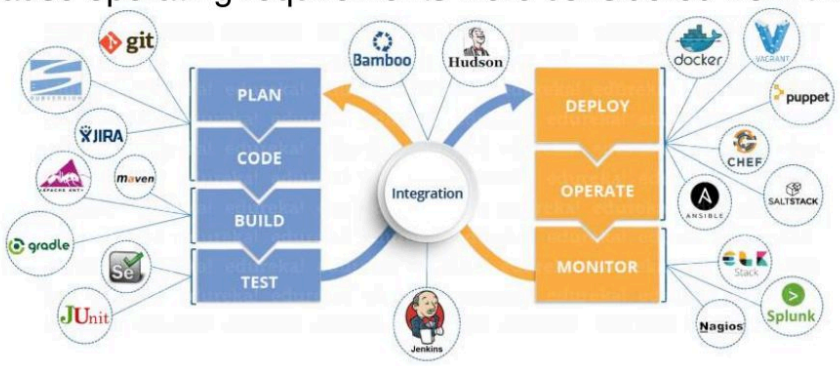
NOTES

1. Organizations close the DevOps gap by **removing the barriers** to collaboration between operations and development...
2. ...adopting a collaborative, learning-centric mindset supported by agile practices, and often **investing heavily in automation**.
3. Development adopts **continuous delivery**, and operations become streamlined and automated.

 DevOps

DevOps

- DevOps is an approach to software development that extends the principles of agile development by integrating both the development and operations units of the IT team into one homogenous unit.
- This integration enables a more seamless flow between the traditionally siloed sides of the software development lifecycle and leads to fewer defects in production because operating requirements were considered from the outset of the project.



78

NOTES

1. **DevOps** extends agile principles by joining development and operations into **one homogeneous unit**.
2. The flow between the formerly siloed sides becomes seamless, and there are **fewer production defects** because operating requirements were considered from the start.

EXAM ANGLE

Prediction Q1

DevOps extends agile principles by:

A Removing testing

B Separating developers from operations

C Fixing scope up front

D **Integrating development and operations into one unit**

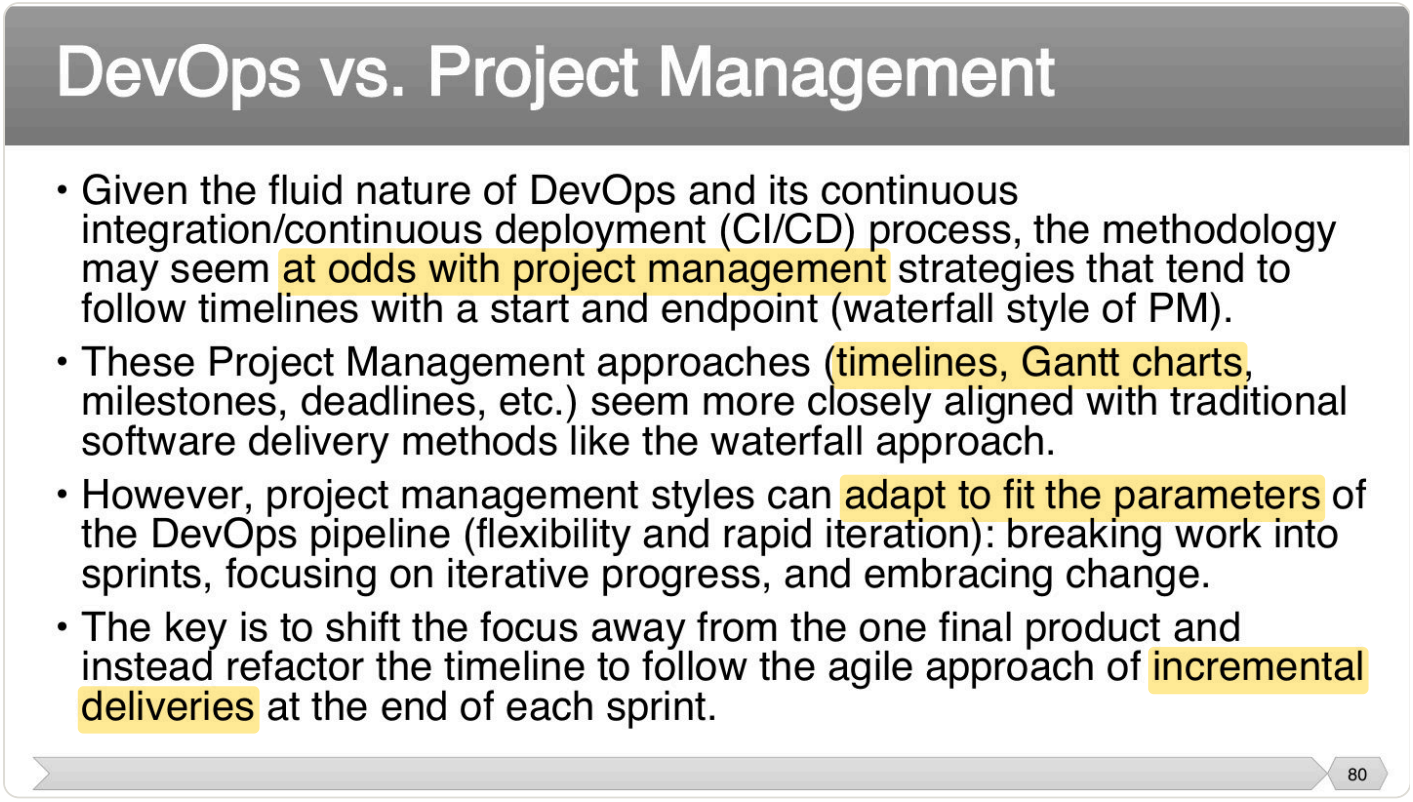
DevOps

- According to a study, the global DevOps market size was $7.398 billion in 2021, and it is anticipated to increase to $37.227 billion by 2030, with a significant CAGR (compound annual growth rate) of 20% from 2022 to 2030.
- DevOps project management is essential to any significant commercial venture to complete the project on schedule and under budget.
- It entails careful planning, resource allocation, and coordination between numerous stakeholders.

NOTES

1. The market figures are context only (not exam material): DevOps is growing fast.
2. DevOps still needs project management to finish **on schedule and under budget**...
3. ...through **careful planning, resource allocation** and coordination between many stakeholders.

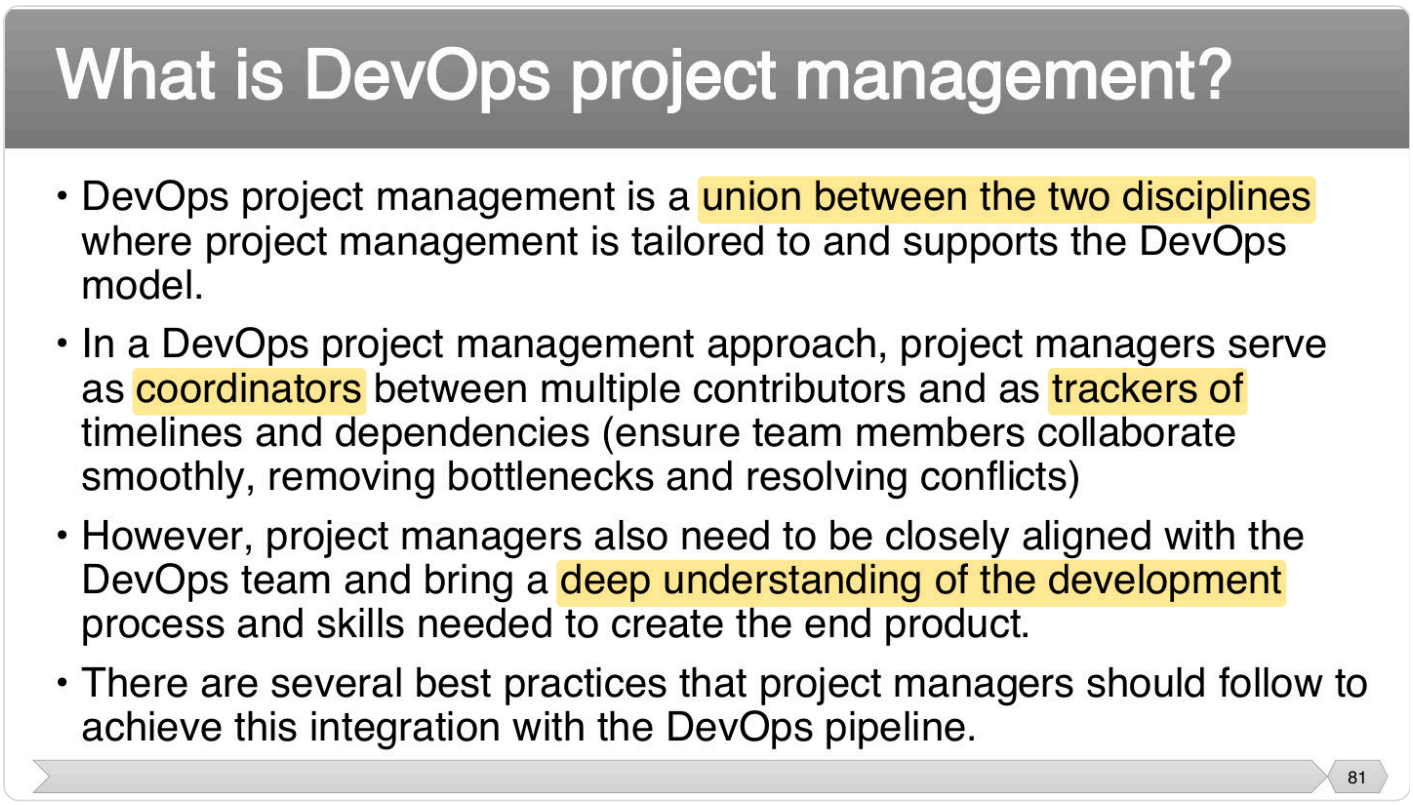
 DevOps vs. Project Management



NOTES

1. With continuous CI/CD, DevOps can seem **at odds with project management**, which follows timelines with a start and end.
2. Classic PM tools (**timelines, Gantt charts, milestones**) fit waterfall delivery more naturally.
3. But PM can **adapt**: break work into sprints, focus on iterative progress, embrace change.
4. The key: stop centering on one final product and plan **incremental deliveries** at the end of each sprint.

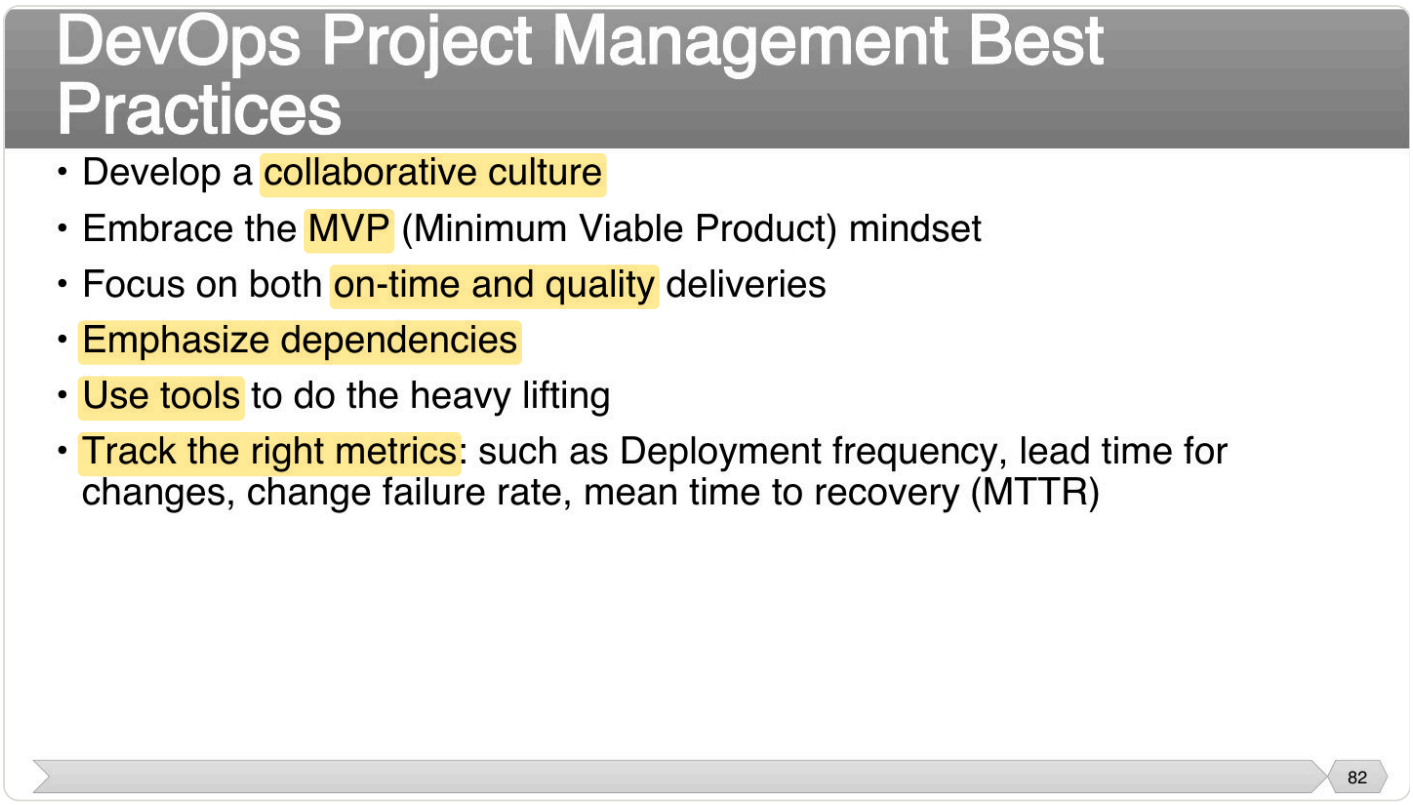
 What is DevOps project management?



NOTES

1. **DevOps project management** is a **union** of the two: PM tailored to support the DevOps model.
2. PMs act as **coordinators** between contributors and **trackers** of timelines and dependencies: making collaboration smooth, removing bottlenecks, resolving conflicts.
3. They must also be closely aligned with the DevOps team and have a **deep understanding of the development process**.

 Practices



NOTES

1. Six best practices (detailed on slides 83–88): develop a **collaborative culture**...
2. ...embrace the **MVP** mindset...
3. ...focus on both **on-time and quality** delivery...
4. ...**emphasize dependencies**...
5. ...let **tools** do the heavy lifting...
6. ...and **track the right metrics**: deployment frequency, lead time for changes, change failure rate and mean time to recovery (MTTR), the four DORA metrics.

TIPS & TRICKS

- **COMMiTteD**: Collaborative culture, On-time and quality, MVP mindset, Metrics, Tools, Dependencies.

EXAM ANGLE

Past exam · Final 251 · Q1.4 Q1

Give an example of a project management tool or technique that you might use to manage a complex software development project, and explain how it could help you ensure project success.
Answer: A Work Breakdown Structure: it decomposes the complex system into smaller deliverables and work packages that can be estimated, assigned and tracked, so nothing is forgotten, ownership is clear, and it is the base for the schedule, budget and risk analysis. Other valid answers, each with its benefit explained: a Gantt chart, CPM, a risk register, or a Scrum board with a burndown chart and Jira/CI dashboards for real-time visibility (slide 87).

Prediction Q2

Which is one of the DevOps metrics listed on the slide?

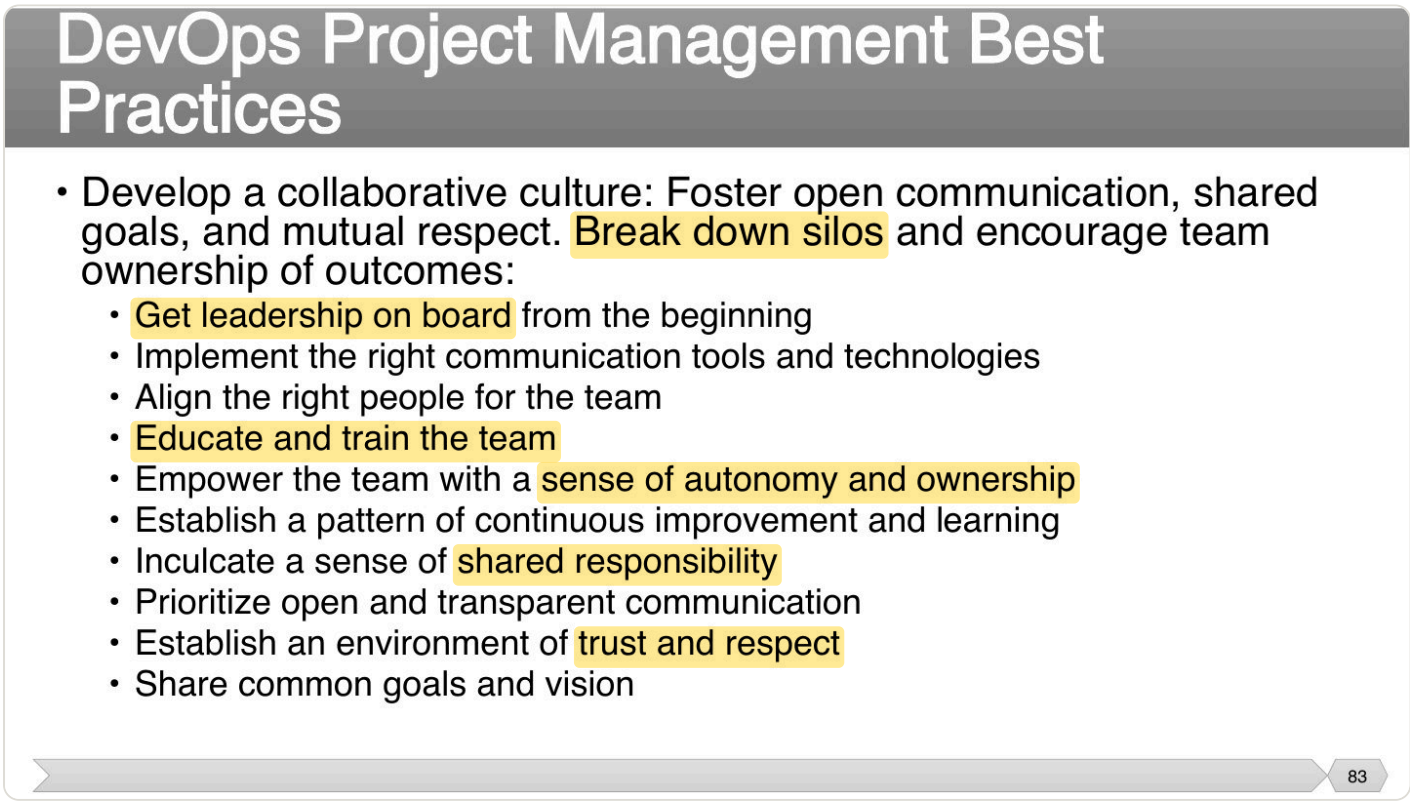
A **Mean time to recovery (MTTR)**

B Number of meetings

C Lines of code per day

D Cost per printed page

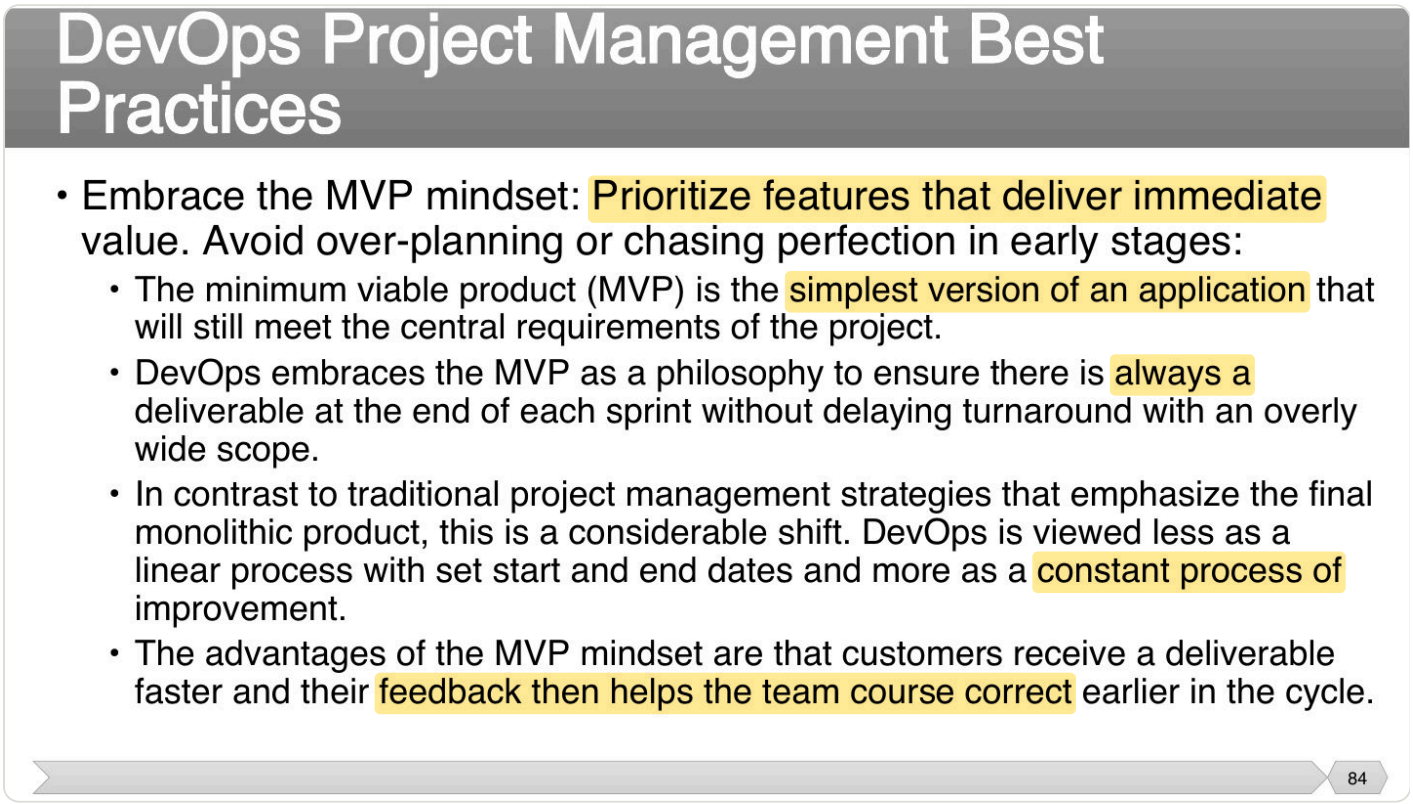
SLIDE 83 / 98 **Practices**



NOTES

1. A **collaborative culture**: open communication, shared goals and mutual respect; **break down silos** and share ownership of outcomes.
2. How: get **leadership on board** from the start, choose the right communication tools, align the right people, **educate and train** the team.
3. Give the team **autonomy and ownership**, build continuous learning and **shared responsibility**...
4. ...with open communication, **trust and respect**, and a common vision.

SLIDE 84 / 98 **Practices**



NOTES

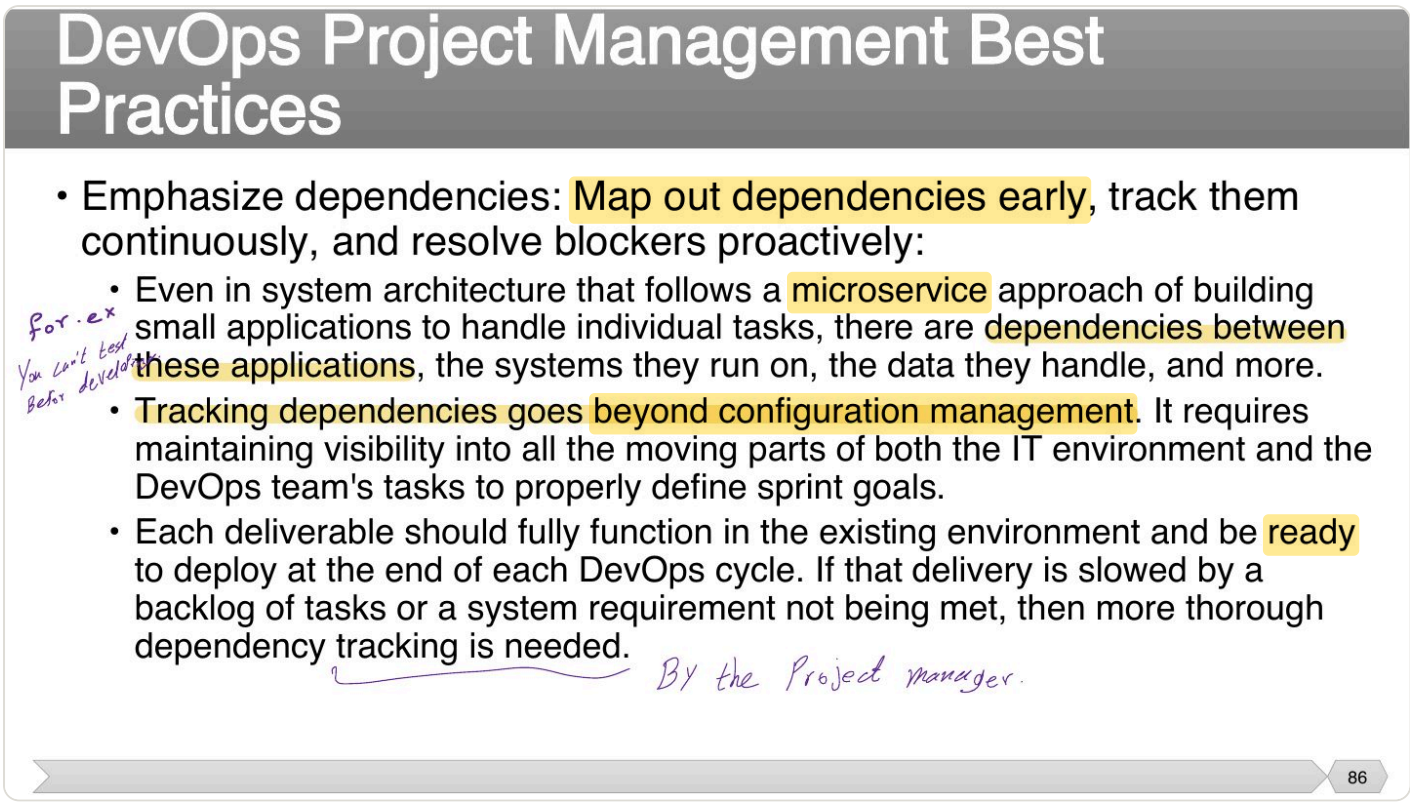
1. **MVP mindset**: **prioritize features with immediate value** and avoid over-planning or chasing perfection early.
2. The **minimum viable product** is the **simplest version** that still meets the central requirements.
3. DevOps uses it so there is **always a deliverable** at the end of each sprint, without an overly wide scope delaying it.
4. A big shift from traditional PM's final monolithic product: DevOps is a **constant process of improvement**, not a line with fixed start and end dates.
5. Customers get something sooner, and their **feedback lets the team correct course** early.

DevOps Project Management Best Practices

- Focus on both on-time and quality deliveries: Set realistic timelines, monitor quality metrics (e.g., defect rates, rollback frequency), and ensure testing is embedded in the pipeline:
  - Project management places a heavy emphasis on meeting deadlines. While this is important, many of the metrics DevOps teams use to measure success focus on more qualitative factors to gauge if the customer is meeting their goals with the application.
  - This means that quick turnaround times are important. So is allocating enough time and resources to ensure the deliverable quality meets a high standard. Project managers need to weigh both these factors when deciding on the goals for a sprint and assigning tasks.
  - Part of an effective delegation of responsibilities is understanding the dependencies for each task.

NOTES

1. **On-time and quality**: set **realistic timelines**, monitor quality metrics (**defect rates, rollback frequency**) and embed testing in the pipeline.
2. PM emphasizes deadlines, but many DevOps success metrics are **qualitative**: is the customer reaching their goals with the application?
3. Fast turnaround matters, and so does allocating **enough** time and resources for quality; the PM weighs both when setting sprint goals.
4. Effective delegation requires knowing the **dependencies for each task**.



NOTES

1. **Emphasize dependencies**: **map them early**, track them continuously and resolve blockers proactively.
2. Even in a **microservice** architecture there are dependencies between services, the systems they run on and the data they handle.
3. Tracking dependencies goes **beyond configuration management**: it needs visibility into the IT environment and the team's tasks to set sprint goals.
4. Each deliverable must work in the existing environment and be **ready to deploy** at the end of each cycle; if delivery is slowed by blocked tasks or unmet system requirements, dependency tracking needs to improve.

DevOps Project Management Best Practices

- Use tools to do the heavy lifting: Leverage dashboards, issue trackers, CI/CD platforms, and communication tools to streamline workflows and gain visibility:
  - Automation is a major focus in DevOps to scale the impact of a team's effort, and DevOps project management is not different.
  - PM tools provide a single source of truth for how work has been assigned, how the project is proceeding, and what issues have occurred along the way. This information is valuable for the entire team to provide greater context and foster more informed decision-making.
  - These applications will overlap and integrate with the tools the DevOps team uses in their day-to-day work so that both project managers and developers are on the same page (Example: A developer updates a ticket in Jira, and the PM sees the status instantly. A failed deployment in Jenkins triggers an alert that both teams can act on.)
  - Dashboards can provide real-time metrics on how the team is performing against their goals and help PMs detect potential issues to address earlier in the cycle.

87

NOTES

1. **Use tools**: **dashboards, issue trackers, CI/CD platforms** and communication tools streamline work and give visibility; automation scales the team's effort.
2. PM tools are a **single source of truth** for who is doing what, how the project is going and what issues occurred.
3. They integrate with the developers' tools so everyone sees the same thing: a developer **updates a Jira ticket** and the PM sees it instantly; a failed Jenkins deployment alerts both teams.
4. Dashboards give **real-time metrics** against goals so the PM spots problems early.

DevOps Project Management Best Practices

g

- Track the right metrics: such as Deployment frequency, lead time for changes, change failure rate, mean time to recovery (MTTR): Use these metrics to guide decisions, identify bottlenecks, and celebrate progress:
  - DevOps best practices recommend constant performance tracking.
  - You can use project management tools with built-in analytics reports to give you insight into data and performance tracking capabilities.
  - Tracking the right performance indicators, such as lead time, mean time to detect, and issue severity to assess the efficacy of a DevOps approach. Tracking these indicators is essential since it enables you to identify problems early and take swift corrective action.
  - The goals and standards of your enterprise will determine the DevOps metrics you need to monitor. Any engineering and development team can benefit from some indicators connected to profitability, like unit cost (the average cost to deliver a single unit of value, such as a feature, user story, bug fix, or deployment.).

88

NOTES

1. **Track the right metrics**: **deployment frequency**, lead time for changes, change failure rate and MTTR; use them to guide decisions, find bottlenecks and celebrate progress.
2. DevOps recommends **constant performance tracking**, using PM tools with built-in analytics.
3. Useful indicators: lead time, **mean time to detect** and issue severity, so problems are found early and corrected quickly.
4. The enterprise's goals decide which metrics; profitability-linked ones such as **unit cost** (average cost to deliver one feature, story, fix or deployment) help any team.

 outcomes.

DevOps project management leads to better outcomes.

- By combining the practice of DevOps (CD/CI, automation, etc.) with the principles of project management (planning, task definition, progress tracking, etc.), you achieve greater coordination between individual team members and the larger goals of the sprint.
- This leads to better outcomes by ensuring that task scopes are properly defined and dependencies are tracked so the application is ready to deploy.
- DevOps project management tools (Jira, github, gitlab, github actions, Jenkins, etc.) simplify this process and promote visibility between project managers and team members so everyone is on the same page and working toward the same goals.

NOTES

1. Combining DevOps practice (**CI/CD, automation**) with PM principles (**planning, task definition, progress tracking**) improves coordination between individuals and sprint goals.
2. Task scopes are well defined and **dependencies tracked**, so the application is ready to deploy.
3. Tools such as **Jira, GitHub, GitLab, GitHub Actions and Jenkins** make this visible to PMs and team members alike.

SLIDE 90 / 98 DevOps tools (Jenkins)

DevOps project management leads to better outcomes (Jenkins)





90

NOTES

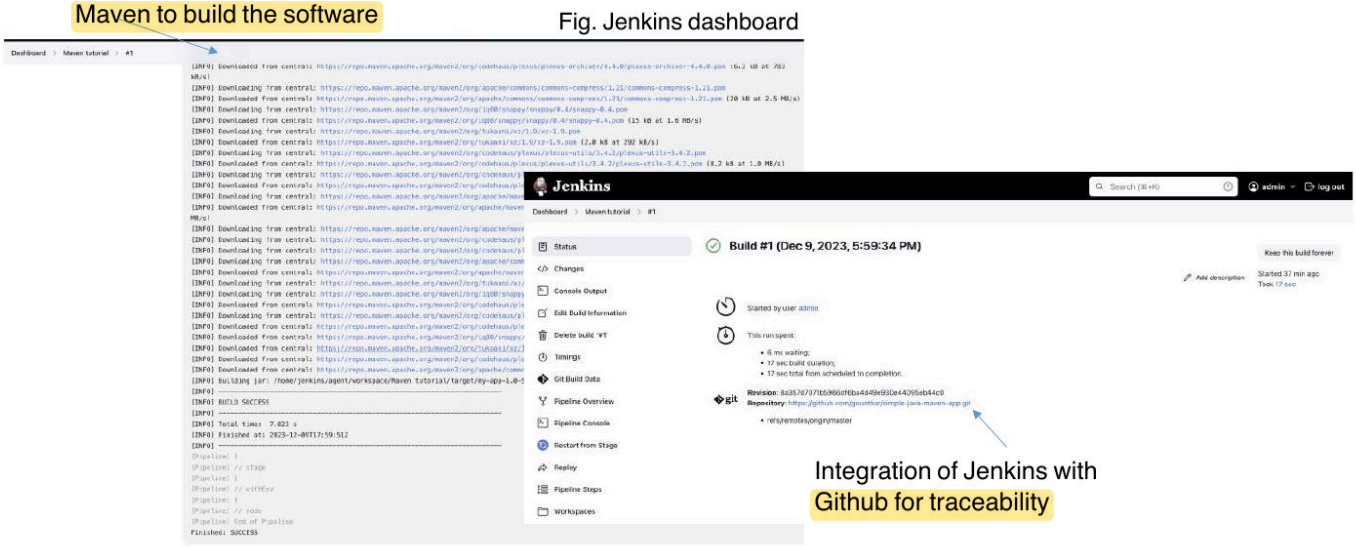
1 **Jenkins**, an open-source automation server, at the center of a DevOps toolchain (Docker, Kubernetes, AWS, Azure, Terraform, Git). It automates build, test and deployment.

SLIDE 91 / 98 outcomes (Jenkins)

DevOps project management leads to better outcomes (Jenkins)

Integration of Jenkins with Maven to build the software

Fig. Jenkins dashboard



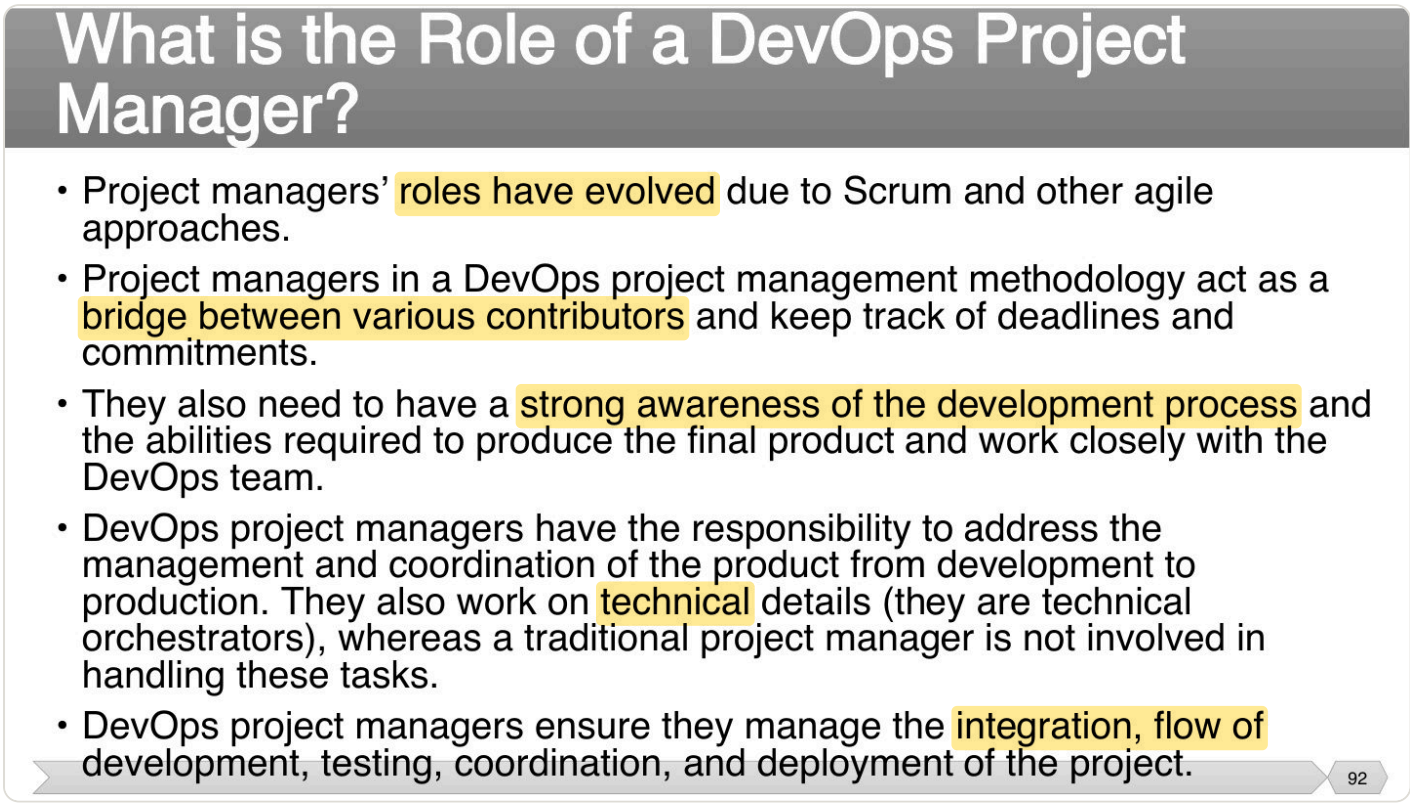
Integration of Jenkins with Github for traceability

91

NOTES

1. Jenkins integrates with **Maven to build** the software automatically...
2. ...and with **GitHub for traceability**: each build links back to the commits that triggered it.

SLIDE 92 / 98 What is the Role of a DevOps Project



NOTES

1. PM **roles have evolved** with Scrum and agile.
2. A DevOps PM is a **bridge between contributors** and tracks deadlines and commitments.
3. They need **strong awareness of the development process** and the skills required, working closely with the DevOps team.
4. They manage the product from development to production and also work on **technical** details as **technical orchestrators**, unlike a traditional PM.
5. They manage **integration, the flow of development, testing, coordination and deployment**.

EXAM ANGLE

Prediction Q1

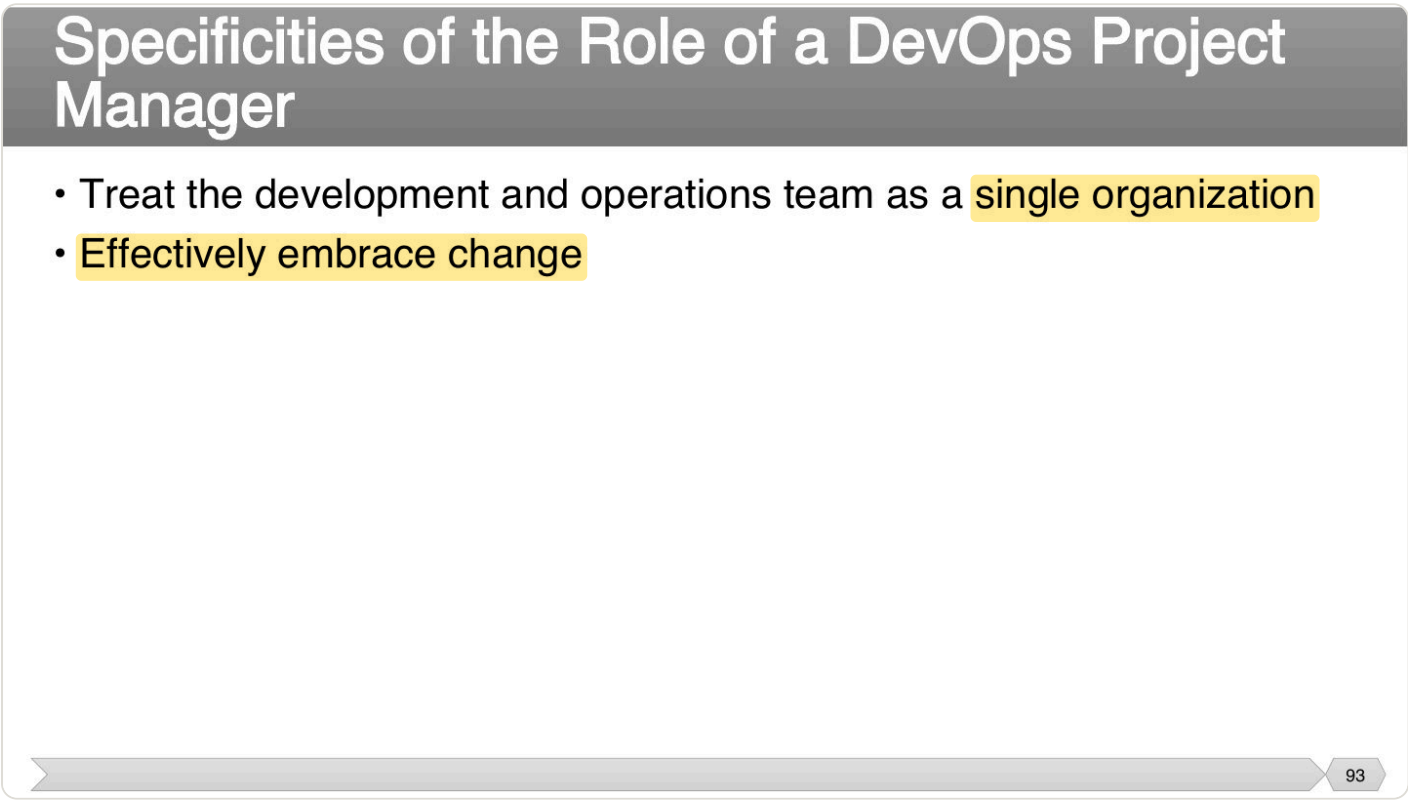
A DevOps project manager stays away from technical details, like a traditional project manager.

T True

F False

> DevOps PMs work on technical details as technical orchestrators.

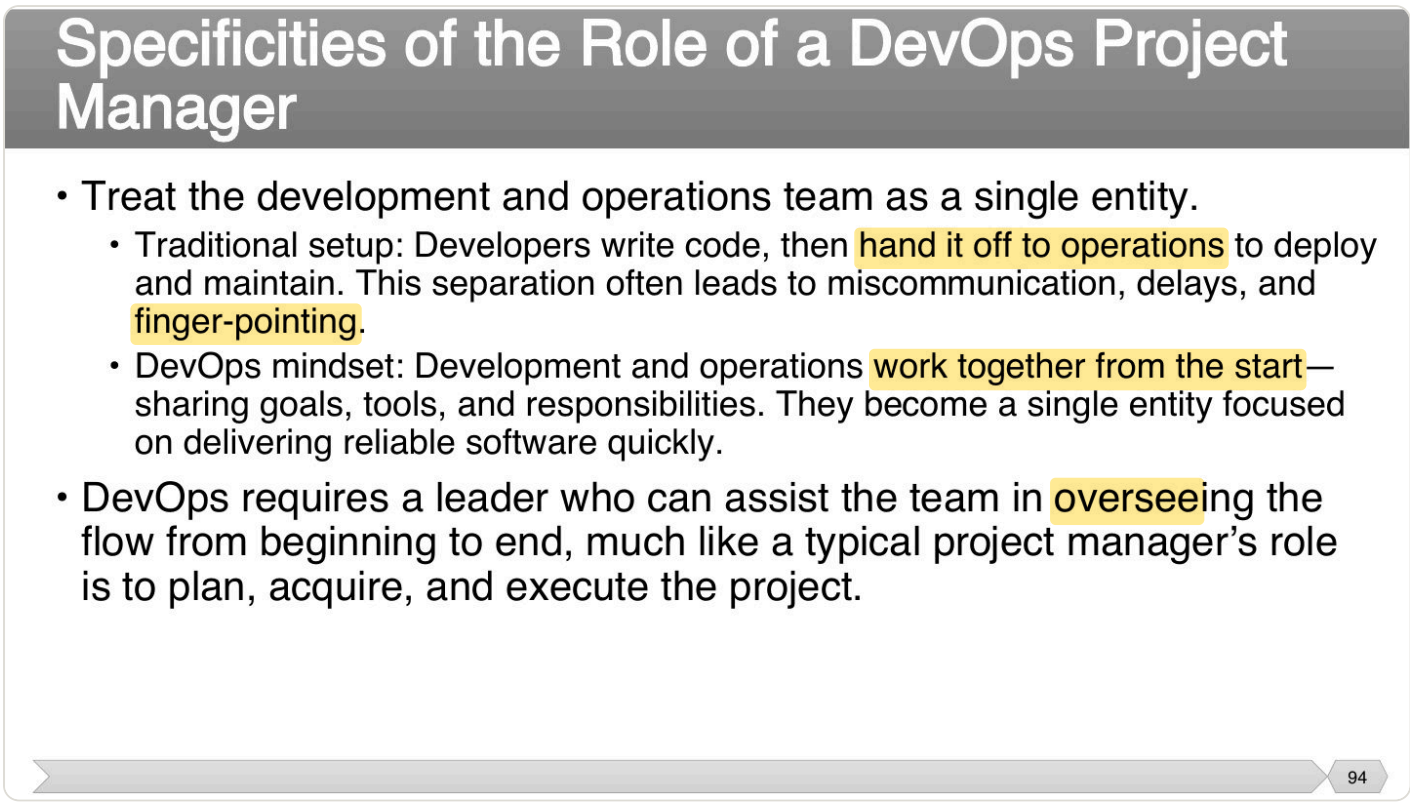
SLIDE 93 / 98 **Specificities of the Role of a DevOps Project**



NOTES

1. The DevOps PM's two specificities (detailed next): **treat development and operations as a single organization**...
2. ...and **embrace change** effectively.

 Specificities of the Role of a DevOps Project



NOTES

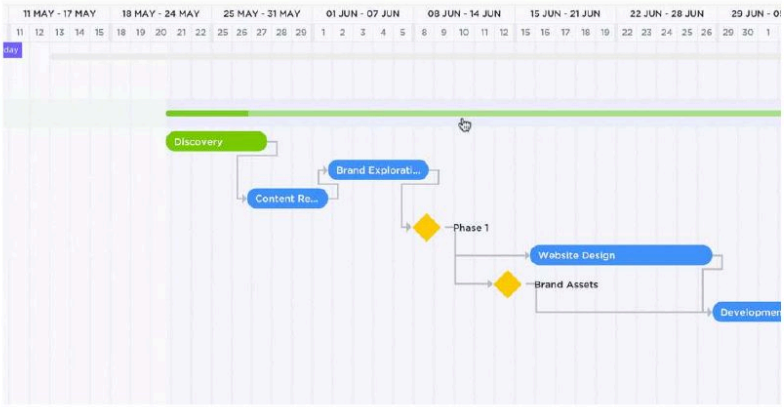
1. Traditionally developers write code and **hand it off to operations**, which causes miscommunication, delays and **finger-pointing**.
2. In DevOps, development and operations **work together from the start**, sharing goals, tools and responsibilities as one entity.
3. This needs a leader to **oversee** the flow end to end, like a typical PM plans, acquires and executes.

 Specificities of the Role of a DevOps Project

Specificities of the Role of a DevOps Project Manager

- Treat the development and operations team as a single entity
  - Creating value stream maps should be a natural extension of the project manager's Gantt Chart planning abilities.

•Traditional project managers use these to plan tasks, timelines, and dependencies.
•Value stream maps visualize the entire flow of work—from idea to deployment—highlighting where value is added and where waste or delays occur.
•For project managers familiar with Gantt charts, creating value stream maps is a logical next step. It shifts the focus from task scheduling to process optimization.



95

NOTES

1. **Value stream maps** are a natural extension of the PM's Gantt-chart skills: Gantt charts **plan tasks, timelines and dependencies**...
2. ...while value stream maps visualize the **entire** flow from idea to deployment, showing where value is added and where **waste or delays** occur.
3. It shifts the focus from **task scheduling to process optimization**.

 Specificities of the Role of a DevOps Project

Specificities of the Role of a DevOps Project Manager

- Treat the development and operations team as a single entity
  - Treating the Development and Operations team as a single organization is important to prevent work silos.
  - These two teams are merged into a single team to work across the entire application lifecycle and develop various skills, from development and testing to deployment to operations.
  - Attempts to use DevOps will be unsuccessful unless Dev and Ops teams collaborate effectively.
  - The development and operations teams have several impacts on task handling: educating and sharing responsibilities between Devs and Ops teams, improving productivity and collaboration, understanding deployment risks, awareness of tool quality, and managing added tasks.

NOTES

1. A single organization **prevents silos**.
2. The teams are **merged** to work across the whole lifecycle and gain skills from development and testing to deployment and operations.
3. DevOps is **unsuccessful unless** Dev and Ops truly collaborate.
4. Effects on task handling: shared education and responsibilities, better productivity and collaboration, **understanding deployment risks**, awareness of tool quality and managing added tasks.

 Specificities of the Role of a DevOps Project

Specificities of the Role of a DevOps Project Manager

- Effectively embrace change
  - The agile project management methodology clarifies that embracing change rather than sticking to the plan is crucial.
  - It takes a lot of communication and teamwork to manage change and release features incrementally. Communication is a project manager's primary skill. Enterprises ought to make use of this potential.

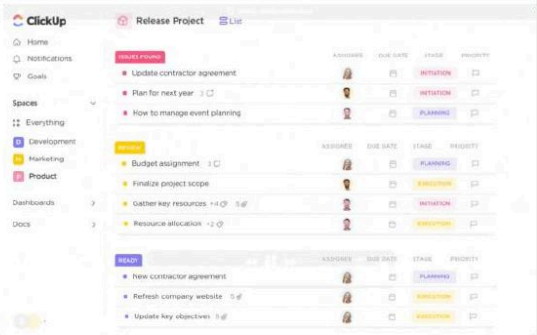
NOTES

1. Agile PM makes clear that **embracing change**, not sticking to the plan, is crucial.
2. Managing change and incremental releases takes a lot of communication and teamwork; **communication is the PM's primary skill**, and enterprises should use it.

 Specificities of the Role of a DevOps Project

Specificities of the Role of a DevOps Project Manager

- Effectively embrace change
  - Change isn't just inevitable—it's a catalyst for growth. To thrive in fast-paced DevOps environments, project managers must evolve their mindset and practices:
- Begin with small projects to test ideas and reduce risk
- using the MVP approach: to deliver value early and often
- using the right agile planning tools: to stay adaptive and transparent
- eliminating silos: by fostering cross-functional collaboration
- reducing project handoffs: to streamline workflows, minimize delays
- creating real-time project visibility: with shared dashboards and live updates
- reducing project overhead: by simplifying processes and focusing on value delivery
- managing change collaboratively: Involve your team in planning and adapting to change



NOTES

1. To thrive with change, PMs **start with small projects** to test ideas, and use the **MVP** to deliver value early and often, with agile planning tools.
2. **Eliminate silos** through cross-functional collaboration and **reduce hand-offs** to cut delays.
3. Create **real-time visibility** with shared dashboards, and reduce overhead by simplifying processes.
4. **Manage change collaboratively**: involve the team in planning and adapting.

EXAM ANGLE

Prediction Q1

Your organization wants to move a waterfall team to DevOps. As the PM, what would you change first and why?

Answer: Treat development and operations as one team with shared goals and tools, to stop hand-offs and finger-pointing. Start with a small project and an MVP so there is a deliverable every sprint; build a CI/CD pipeline with automated tests; map and track dependencies; use shared dashboards (Jira, Jenkins) for real-time visibility; and track metrics such as deployment frequency, lead time, change failure rate and MTTR to show improvement.

Original slides from the SE423 lecture deck, with study notes. Check exact wording against the original deck.
Notes and questions are study aids. Made by Shoug Alomran.